

UNIVERSIDAD DE ALMERIA

ESCUELA SUPERIOR DE INGENIERÍA

“Espejo inteligente
para la monitorización
de hábitos de vida
saludable.”

Curso 2024/2025

Alumno/a:

Esther Ibáñez Mingorance

Director/es:

Marcos Lupión Lorente
Vicente González Ruíz

Agradecimientos

En primer lugar, quiero agradecer a mis tutores, Marcos y Vicente, por haberme acompañado durante todo este proyecto y por su orientación en los momentos en que más lo necesité.

Además, quisiera mencionar a mis amigos. Aquellos que forman parte de mi círculo más cercano y que, la mayoría, llevan conmigo desde la infancia. Aunque ellos no tienen un gran conocimiento de la informática y, en ocasiones, no entendían del todo mis problemas por ser muy específicos de la carrera, siempre han estado ahí en mis momentos de estrés, frustración y ansiedad, dispuestos a escucharme y dejar que me desahogara. Un amigo vale más que un tesoro y yo tengo la suerte de no tener suficientes dedos en la mano para contarlos.

Por otro lado, también quiero agradecer a las personas que esta carrera me ha permitido conocer, y con quienes compartí clase. Sin ellos, la experiencia universitaria no hubiera sido la misma. Éramos el apoyo unos de otros, el empuje que necesitábamos para seguir hacia adelante y los cómplices de locuras y viajes.

En concreto, me gustaría mencionar a Isa, quien se ha convertido en mi pilar fundamental, tanto dentro como fuera de la universidad. Por ser mi base de apoyo en los malos momentos y mi punto de partida en los buenos. Mi vida no sería tan buena como lo es hoy si aquel día en clase no me hubiera dejado sentarme a su lado.

Por último, pero no menos importante, quiero expresar mi eterno agradecimiento a mi familia. A mis padres y hermana por guiarme siempre que han podido y por haber hecho todo lo posible para que pudiera estudiar aquello que me gustaba. Y a mi familia no tan cercana, como tíos y primos, por estar siempre pendientes de cómo me iba en la carrera y celebrar conmigo las buenas noticias tanto como si fueran tuyas.

La vida no siempre es justa y, mi tía “Queti”, falleció recientemente mientras finalizaba este proyecto. Esté donde esté, junto a mi abuela Enriqueta, estoy segura de que celebran este logro con la misma alegría con la que lo hacían en vida. Este trabajo va también dedicado a ellas, por ser mujeres fuertes, luchadoras y motor de inspiración.

Igualmente, a riesgo de parecer presuntuosa, me quiero felicitar a mí misma. Aquella chica vergonzosa y tímida que empezó Ingeniería Informática no es la misma que hoy la termina. Nuevas experiencias, conocimientos, formas de afrontar la vida, personas que me han marcado y ganas de crecer han hecho que me convierta en una versión mejorada de mí misma. Parecía un camino muy largo hasta terminar, pero al final no estaba tan lejos como creía.

Índice general

1	Introducción	1
1.1	Internet de las cosas	1
1.1.1	Smart Homes	2
1.2	Reconocimiento de acciones humanas	3
1.2.1	Aplicaciones del reconocimiento de acciones	3
1.2.2	Desafíos	4
1.3	Problemas de memoria en la tercera edad	5
1.4	Motivación	6
1.5	Objetivos del proyecto	6
1.6	Estructura del documento	8
2	Estado del Arte	10
2.1	Trabajos relacionados	10
2.2	Contribuciones del proyecto	11
3	Fases del proyecto	13
3.1	Planificación	13
3.2	Ejecución	15
3.3	Comparativa	17
4	Herramientas	18
4.1	Lenguajes de programación	18
4.1.1	Python	18
4.1.1.1	Beneficios de usar Python	18
4.1.1.2	Bibliotecas utilizadas	19
4.1.2	L ^A T _E X	19
4.1.2.1	Overleaf	20
4.2	Comunicación	20
4.2.1	Correo electrónico	20
4.2.2	Reuniones	21
4.3	Desarrollo	21
4.3.1	Visual Studio Code	21
4.3.2	Git	21
4.4	Infraestructura	22
4.4.1	Google Colab	22
4.4.2	Jupyter Notebook	23

5	Desarrollo del sistema	24
5.1	Arquitectura del sistema	24
5.2	Sistema de adquisición de datos anotados	25
5.2.1	Modelos para la estimación de pose	25
5.2.1.1	MediaPipe	26
5.2.2	Creación del dataset	28
5.2.2.1	Obtención de las imágenes	29
5.2.3	Actividades a reconocer	32
5.2.3.1	Anotación de actividades	33
5.2.4	Generación de imágenes	35
5.2.4.1	Carga de keypoints desde CSV	35
5.2.4.2	Visualización de keypoints	36
5.3	Modelo IA de reconocimiento de actividades	37
5.3.1	Taxonomía	37
5.3.1.1	Redes neuronales totalmente conectadas	38
5.3.1.2	Redes neuronales convolucionales	39
5.3.1.3	Redes neuronales recurrentes	39
5.3.1.3.1	LSTM (Long Short-Term Memory)	40
5.3.1.3.2	GRU (Gated Recurrent Unit)	40
5.3.1.3.3	Comparación conceptual	41
5.3.2	Combinación de arquitecturas en el modelo propuesto	42
5.3.3	Pre-procesamiento	44
5.3.3.1	Normalización	44
5.3.3.2	División del dataset por actividades	46
5.3.3.3	División del dataset en train y test	46
5.3.4	Entrenamiento	50
5.3.4.1	Evolución del entrenamiento	51
5.3.4.2	Carga y preparación de los datos	51
5.3.4.3	Evolución de la arquitectura: de LSTM a GRU	51
5.3.4.4	Definición del modelo	52
5.3.4.5	Compilación y configuración del entrenamiento	52
5.3.4.6	Entrenamiento	53
5.3.5	Validación	53
5.3.5.1	Métricas	53
5.4	Aplicación Web	55
5.4.1	Espejo inteligente	55
5.4.1.1	Componentes	55
5.4.1.1.1	Raspberry Pi 4 Model B	55
5.4.1.1.2	Cámara Módulo 3 NoIR para Raspberry Pi	56
5.4.1.1.3	Cable Flex CSI	57
5.4.1.1.4	Tarjeta microSD de 16 GB	57
5.4.1.1.5	Monitor HP LE2101W	58
5.4.1.1.6	Espejo acrílico semitransparente	59
5.4.2	Modificaciones realizadas en el espejo	59
5.4.2.1	Configuración de hardware	60

5.4.2.2	Configuración de software	60
5.4.3	Análisis de requisitos	60
5.4.3.1	Requisitos funcionales	61
5.4.3.2	Requisitos no funcionales	61
5.4.3.3	Actores del sistema	62
5.4.3.4	Casos de Uso	62
5.4.4	Funcionamiento del sistema	64
5.4.4.1	Escaneo e identificación de actividades	67
5.4.4.2	Ejecución en tiempo real del modelo	67
5.4.5	Almacenamiento y recuperación de datos	68
5.4.6	Muestra de actividades en la pantalla	69
5.5	Presupuesto	70
6	Resultados	72
6.1	Desempeño del modelo	72
6.1.1	Análisis mediante matrices de confusión	72
6.1.2	Dificultades	76
6.2	Simulación de uso	76
6.3	Evaluación del modelo en diferentes escenarios	80
6.3.1	Precisión según el lado de ejecución	81
6.3.2	Influencia de obstáculos en el campo visual	81
6.3.3	Variación según condiciones de iluminación	82
6.4	Conclusión de la evaluación	83
6.4.1	Análisis por clase	83
7	Conclusiones y trabajo futuro	85
7.1	Conclusiones	85
7.2	Trabajo futuro	86
	Bibliografía	88

Índice de figuras

1.1	Aplicaciones principales del IoT.	1
1.2	SmartHome.	2
1.3	Prevalencia estimada de deterioro cognitivo leve (DCL) según grupo de edad. . .	5
3.1	Cronograma estimado de la realización del proyecto.	14
3.2	Cronograma real de la realización del proyecto.	16
3.3	Gráfica comparativa entre tiempos.	17
5.1	Arquitectura del sistema.	24
5.2	Diagrama de la creación del dataset.	28
5.3	Archivo de la cara.	32
5.4	Archivo del cuerpo.	32
5.5	Archivo de las manos.	32
5.6	Formato del archivo “acciones_ anotadas”.	35
5.7	Acción <i>lavarse los dientes</i> con <i>keypoints</i>	37
5.8	Redes neuronales totalmente conectadas.	38
5.9	Redes neuronales convolucionales.	39
5.10	Redes neuronales recurrentes LSTM.	41
5.11	Redes neuronales recurrentes GRU.	41
5.12	Modelo CNN y GRU.	44
5.13	Cepillarse el pelo. Imagen normalizada vs imagen no normalizada.	45
5.14	Lavarse las manos. Imagen normalizada vs imagen no normalizada.	45
5.15	División del dataset en carpetas según actividades.	46
5.16	División del dataset se carpetas según train y test.	47
5.17	Dataset, ejemplo de la acción <i>Cepillarse el pelo</i>	47
5.18	Dataset, ejemplo de la acción <i>Echarse colonia</i>	48
5.19	Dataset, ejemplo de la acción <i>Echarse desodorante</i>	48
5.20	Dataset, ejemplo de la acción <i>Lavarse la cara</i>	49
5.21	Dataset, ejemplo de la acción <i>lavarse las manos</i>	49
5.22	Dataset, ejemplo de la acción <i>lavarse los dientes</i>	50
5.23	Dataset, ejemplo de la acción <i>tomarse las pastillas</i>	50
5.24	División entre train, validation y test.	53
5.25	Pérdida durante el entrenamiento.	54
5.26	Precisión durante el entrenamiento.	54
5.27	Espejo.	55
5.28	Raspberry Pi 4 Model B.	56
5.29	Cámara Módulo 3 NoIR para Raspberry Pi.	56

5.30	Cable Flex CSI para cámara Raspberry Pi.	57
5.31	Tarjeta microSD de 16 GB.	58
5.32	Monitor HP LE2101W.	58
5.33	Espejo acrílico semitransparente.	59
5.34	Diagrama de casos de uso.	62
5.35	Diagrama de flujo del funcionamiento del sistema.	66
5.36	Ejemplo imagen normalizada.	68
5.37	Almacenamiento en archivo CSV de las acciones detectadas.	69
5.38	Visualización del historial de acciones detectadas.	69
5.39	Interfaz web con lista de acciones y stream en vivo.	70
6.1	Matriz de confusión - conjunto de entrenamiento.	73
6.2	Matriz de confusión - conjunto de validación.	73
6.3	Similitudes entre lavarse las manos y lavarse la cara.	74
6.4	Matriz de confusión - conjunto de test.	75
6.5	Similitudes entre cepillarse el pelo y echarse colonia.	75
6.6	Prueba real del funcionamiento del espejo.	77
6.7	Acción de cepillarse el pelo en la web.	78
6.8	Acción de echarse colonia en la web.	78
6.9	Acción de echarse desodorante en la web.	78
6.10	Acción de lavarse la cara en la web.	79
6.11	Acción de lavarse las manos en la web.	79
6.12	Acción de lavarse los dientes en la web.	79
6.13	Acción de tomarse las pastillas en la web.	80
6.14	Todas las acciones realizadas vista desde la web.	80
6.15	Porcentaje de acierto según la mano de ejecución.	81
6.16	Porcentaje de acierto según los obstáculos en la habitación.	82
6.17	Porcentaje de acierto según la iluminación.	82

Índice de tablas

5.1	Especificaciones técnicas de la Raspberry Pi 4 Model B.	56
5.2	Especificaciones técnicas de la cámara NoIR.	57
5.3	Especificaciones técnicas del cable Flex CSI.	57
5.4	Especificaciones técnicas de la tarjeta microSD.	58
5.5	Especificaciones técnicas del monitor HP LE2101W.	59
5.6	Especificaciones técnicas del espejo.	59
5.7	Caso de uso 1: Detectar acción en tiempo real.	63
5.8	Caso de uso 2: Registrar acción realizada.	63
5.9	Caso de uso 3: Consultar historial de acciones.	64
5.10	Resumen del presupuesto del proyecto.	71

Índice de Códigos

5.1	Configuración de cámara.	29
5.2	Inicio de cámara.	29
5.3	Detección de keypoints.	30
5.4	Almacenar los datos en archivos CSV.	30
5.5	Captura, procesamiento y almacenamiento de imágenes.	30
5.6	Almacenamiento de keypoints.	31
5.7	Acciones recogidas.	34
5.8	Extracción de características espaciales.	42
5.9	Reducción de los mapas de activación a vectores.	43
5.10	Procesamiento secuencial mediante una capa GRU.	43
5.11	Capa de salida para clasificación de acciones.	43
5.12	Centrado y escalado de puntos clave.	44
5.13	Carga y normalización de una secuencia.	51
5.14	Arquitectura CNN-GRU final.	52
5.15	Entrenamiento del modelo.	53
5.16	Captura de vídeo en tiempo real.	67
5.17	Normalización de puntos clave.	67
5.18	Inferencia del modelo.	68
5.19	Registro de actividad detectada.	68
5.20	Almacenamiento en archivo CSV.	68
5.21	Detección y confirmación de acción en el cliente.	69

Glosario

CNN Red neuronal convolucional, por sus siglas en inglés *Convolutional Neural Network*.

CPU Unidad central de procesamiento, por sus siglas en inglés *Central Processing Unit*.

CSI Interfaz serie de cámara, por sus siglas en inglés *Camera Serial Interface*.

CSV Valores separados por comas, por sus siglas en inglés *Comma-Separated Values*.

DCL Deterioro Cognitivo Leve.

FPS Imágenes por segundo, por sus siglas en inglés *Frames Per Second*.

GPU Unidad de procesamiento gráfico, por sus siglas en inglés *Graphics Processing Unit*.

GRU Unidad recurrente simplificada, por sus siglas en inglés *Gated Recurrent Unit*.

HTML Lenguaje de marcado de hipertexto, por sus siglas en inglés *HyperText Markup Language*.

IA Inteligencia Artificial.

IDE Entorno de desarrollo integrado, por sus siglas en inglés *Integrated Development Environment*.

IoT Internet de las cosas, por sus siglas en inglés *Internet of Things*.

JSON Notación de objetos de JavaScript, por sus siglas en inglés *JavaScript Object Notation*.

LSTM Memoria a largo y corto plazo, por sus siglas en inglés *Long Short-Term Memory*.

ReLU Unidad lineal rectificada, por sus siglas en inglés *Rectified Linear Unit*.

RGB-D Color y profundidad, por sus siglas en inglés *Red Green Blue + Depth*.

RNN Red neuronal recurrente, por sus siglas en inglés *Recurrent Neural Network*.

Sigmoid Función sigmoide, por sus siglas en inglés *Sigmoid Function*.

SMC Quejas subjetivas de memoria, por sus siglas en inglés *Subjective Memory Complaints*.

Softmax Función softmax, por sus siglas en inglés *Softmax Function*.

Tanh Tangente hiperbólica, por sus siglas en inglés *Hyperbolic Tangent*.

TCP Protocolo de control de transmisión, por sus siglas en inglés *Transmission Control Protocol*.

TFG Trabajo de Fin de Grado.

1 Introducción

1.1 Internet de las cosas

Internet de las cosas, por sus siglas en inglés *Internet of Things* (IoT) es un paradigma tecnológico que permite la interconexión de objetos cotidianos mediante redes, dotándolos de la capacidad de recopilar, transmitir y procesar datos de forma autónoma. Esta tecnología convierte objetos físicos en elementos inteligentes que pueden interactuar entre ellos y con el entorno, facilitando la automatización y mejorando la toma de decisiones basada en datos [1].

El IoT se ha convertido en un pilar fundamental en sectores como la industria, la salud, la logística o el hogar (Figura 1.1), gracias a su capacidad para optimizar recursos, reducir costes y mejorar la calidad de vida. Una arquitectura típica IoT incluye sensores, actuadores, una red de comunicación, una unidad de procesamiento local, como una Raspberry Pi, y, en muchos casos, servicios en la nube para el almacenamiento o análisis de datos en tiempo real [2].



Figura 1.1: Aplicaciones principales del IoT.

1.1.1 Smart Homes

Uno de los principales campos de aplicación del IoT es el de las viviendas inteligentes o *smart homes*, entendidas como entornos residenciales que integran tecnologías interconectadas para proporcionar confort, eficiencia energética, seguridad y asistencia personalizada a los usuarios [3].

Las primeras aproximaciones al concepto de *smart home* surgieron en los años noventa, con la incorporación de sistemas programables que automatizaban funciones básicas del hogar [4]. Desde entonces, han evolucionado hasta convertirse en entornos dinámicos que aprenden de los hábitos del usuario e integran sistemas de Inteligencia Artificial (IA), interfaces naturales y redes de sensores distribuidos [5].

En la actualidad, una de las aplicaciones más prometedoras de las *smart homes* es su uso en la asistencia a personas mayores o con enfermedades cognitivas. Estos sistemas permiten una monitorización continua de las actividades diarias del usuario, facilitando la detección de anomalías, caídas o situaciones de riesgo, y fomentando así el envejecimiento activo e independiente [6, 7].

A continuación, en la Figura 1.2, se muestra un ejemplo del SmartHome de la Universidad de Almería, con sensores conectados y recopilando datos.



Figura 1.2: SmartHome.

1.2 Reconocimiento de acciones humanas

El reconocimiento automático de actividades humanas en entornos domésticos es un campo clave dentro del Internet de las Cosas (IoT) y la inteligencia artificial (IA), especialmente en aplicaciones orientadas a la asistencia de personas mayores o dependientes. Este problema consiste en identificar, de forma no intrusiva, qué tareas está realizando una persona en su día a día, como asearse, comer o tomar la medicación, con el objetivo de supervisar rutinas, detectar anomalías o fomentar la autonomía personal [8].

Para abordar este reto, existen distintas tecnologías y enfoques. Por un lado, se han utilizado sensores binarios (como contactos magnéticos en puertas o sensores de movimiento por infrarrojos), que ofrecen información muy limitada y no permiten una inferencia precisa sobre la actividad concreta que se está realizando [9]. Otra alternativa son los dispositivos *wearables*, como pulseras o relojes inteligentes, que permiten capturar patrones de movimiento a través de acelerómetros y giroscopios, aunque requieren que el usuario los lleve puestos de forma constante [10].

Una solución más versátil y menos intrusiva es el uso de cámaras combinadas con algoritmos de visión por computador. Gracias al avance de técnicas de aprendizaje profundo, hoy es posible detectar y seguir puntos clave del cuerpo humano (*keypoints*) en tiempo real, permitiendo analizar la postura, los gestos y la interacción con objetos. Esta información resulta especialmente útil para tareas de reconocimiento de acciones, ya que permite trabajar con secuencias de movimientos sin necesidad de procesar directamente imágenes completas, lo que mejora tanto la eficiencia como la privacidad [11].

1.2.1 Aplicaciones del reconocimiento de acciones

Dentro de las muchas ramas de la IA, la visión por computador resulta fundamental para que las máquinas interpreten y comprendan el contenido visual que nos rodea. Concretamente, esta tecnología es especialmente útil en tareas como la detección de objetos o el análisis de movimientos, permitiendo su aplicación en campos como la medicina, la robótica o el hogar.

Utilizando algoritmos de aprendizaje profundo y redes neuronales, hoy en día es posible identificar no solo objetos estáticos, sino también acciones complejas en movimiento. Estos avances aportan gran valor en contextos asistenciales, pudiendo determinar las acciones que un usuario realiza.

El reconocimiento de acciones humanas se puede aplicar en una gran cantidad de ámbitos, especialmente cuando se combina con tecnologías accesibles como cámaras y sensores de bajo costo. Algunas de las más destacadas incluyen:

- **Asistencia en el hogar:** En contextos de envejecimiento o deterioro cognitivo, poder detectar si una persona ha realizado correctamente acciones como lavarse la cara o cepillarse los dientes permite alertar a cuidadores o familiares en caso de olvido, sin invadir

la privacidad del usuario [12].

- **Salud y bienestar:** El seguimiento de la rutina diaria puede ofrecer indicadores sobre el estado anímico o físico de una persona. Cambios en la frecuencia o la forma en la que se realizan ciertas acciones pueden reflejar alteraciones en la salud [13].
- **Interacción con dispositivos inteligentes:** Este tipo de detección puede integrarse con otros elementos domóticos del hogar (luces, música, recordatorios), activando funciones automáticamente cuando se identifica una determinada acción [14].
- **Entrenamiento y rehabilitación:** El análisis de la postura y los movimientos también puede utilizarse en el ámbito deportivo o fisioterapéutico, proporcionando retroalimentación objetiva sobre la ejecución de gestos o ejercicios [15].
- **Supervisión discreta:** Detectar situaciones atípicas, como que una persona permanezca frente al espejo durante un periodo prolongado sin realizar ninguna actividad, puede ser útil para generar alertas tempranas ante posibles incidentes, como desorientación, falta de iniciativa o incluso una caída [16].

1.2.2 Desafíos

A pesar de su potencial, el reconocimiento de acciones humanas mediante *keypoints* aún presenta diversos desafíos técnicos:

- **Condiciones variables del entorno:** Cambios en la iluminación, el fondo o la ropa pueden afectar a la detección de *keypoints*, introduciendo ruido y distorsiones en los datos.
 - **Variabilidad entre usuarios:** Una misma acción puede ejecutarse de muchas formas distintas dependiendo de la persona, lo que complica la generalización de los modelos.
 - **Acciones similares entre sí:** Algunos gestos comparten movimientos parciales o posturas intermedias muy parecidas, dificultando la diferenciación entre acciones como lavarse la cara y peinarse.
 - **Limitaciones del hardware:** Ejecutar modelos de aprendizaje profundo en tiempo real desde un dispositivo embebido requiere optimización y, en muchos casos, simplificación del modelo sin perder precisión.
 - **Calidad del dataset:** Un dataset pequeño o sesgado puede generar modelos que no sean representativos, lo que disminuye su rendimiento en condiciones reales.
-

1.3 Problemas de memoria en la tercera edad

El envejecimiento suele venir acompañado de ciertos cambios en las capacidades mentales, siendo la memoria una de las funciones más sensibles al paso del tiempo. Aunque es normal experimentar olvidos puntuales en la edad avanzada, no siempre resulta sencillo distinguir entre un deterioro asociado al envejecimiento típico y los primeros indicios de un trastorno cognitivo más grave [17].

Entre los distintos tipos de memoria, la memoria episódica (aquella que nos permite recordar experiencias personales situadas en un tiempo y lugar concretos) es la que presenta una mayor vulnerabilidad con la edad [18]. También son frecuentes la disminución de la velocidad de procesamiento mental, la pérdida de concentración y las dificultades para recordar nombres o hechos recientes.

Una de las señales de alerta más comunes en etapas tempranas es la presencia de quejas subjetivas de memoria (Quejas subjetivas de memoria, por sus siglas en inglés *Subjective Memory Complaints* (SMC)), es decir, la percepción personal de que la memoria ya no funciona como antes. Aunque estas quejas no siempre se corresponden con un deterioro medible, se ha demostrado que pueden asociarse a un mayor riesgo de desarrollar deterioro cognitivo leve (Deterioro Cognitivo Leve (DCL)) o enfermedades neurodegenerativas como el Alzheimer [19].

En la Figura 1.3 se muestra cómo la prevalencia del deterioro cognitivo leve aumenta de forma significativa a medida que avanza la edad, especialmente a partir de los 70 años. Esta tendencia evidencia la necesidad de contar con mecanismos de detección temprana que permitan identificar cambios sutiles en el comportamiento antes de que el deterioro sea más evidente.

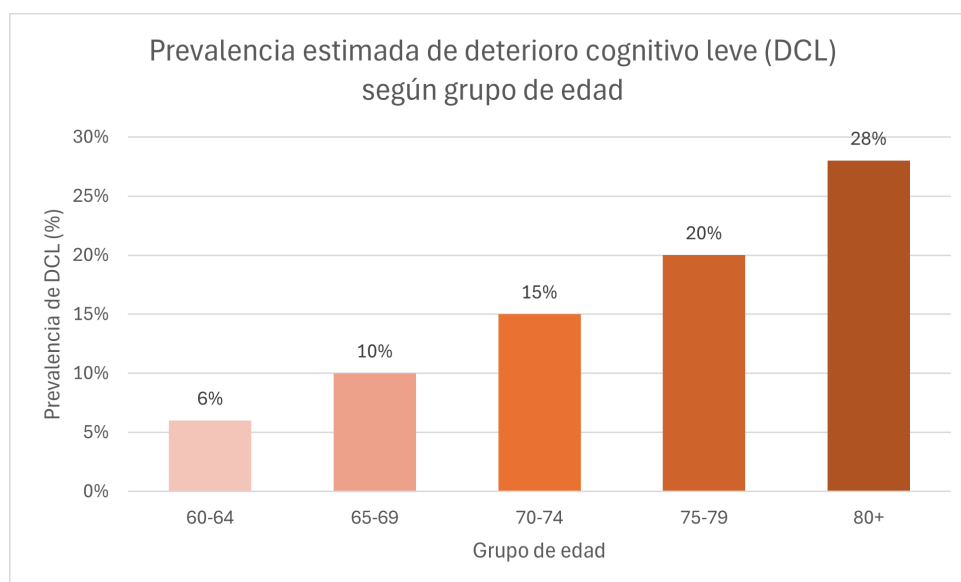


Figura 1.3: Prevalencia estimada de deterioro cognitivo leve (DCL) según grupo de edad [20].

Ante este contexto, el seguimiento de rutinas diarias puede ofrecer información relevante sobre el estado cognitivo de una persona. Cambios en la frecuencia o la omisión de actividades básicas de higiene, por ejemplo, podrían actuar como indicadores tempranos de un posible deterioro funcional. Por ello, resulta especialmente útil explorar soluciones tecnológicas que permitan monitorizar dichas actividades de forma continua, no invasiva y en el propio hogar.

1.4 Motivación

El envejecimiento de la población es una realidad creciente en la mayoría de sociedades desarrolladas. Cada vez son más las personas mayores que viven solas en su domicilio, muchas de ellas con algún tipo de dependencia física o cognitiva que condiciona su autonomía. Si bien algunas cuentan con el apoyo puntual de cuidadores, familiares o personal asistencial, lo cierto es que gran parte del tiempo permanecen sin supervisión directa, lo que incrementa el riesgo ante cualquier incidente o situación de deterioro no detectado.

En este contexto, uno de los indicadores más relevantes del estado funcional de una persona mayor es su capacidad para mantener rutinas básicas de autocuidado. Acciones como lavarse los dientes, peinarse, ducharse o tomar la medicación no solo son esenciales para la salud, sino que también reflejan el grado de conservación de habilidades cognitivas y ejecutivas. La omisión continuada o el abandono de estas tareas puede ser un síntoma precoz de un deterioro cognitivo leve o el inicio de una enfermedad neurodegenerativa, y por tanto debe ser identificado a tiempo.

La motivación principal de este trabajo es, por tanto, ofrecer una solución tecnológica que permita detectar de forma no invasiva si una persona está llevando a cabo determinadas actividades de la vida diaria relacionadas con el aseo personal. Monitorizar este tipo de rutinas mediante visión por computador e IA puede proporcionar información valiosa a familiares y profesionales, facilitando intervenciones tempranas y mejorando la seguridad y calidad de vida de las personas mayores.

Además, esta herramienta podría integrarse en un entorno doméstico o residencial adaptado, fomentando una mayor independencia sin renunciar al seguimiento del estado funcional del usuario. Identificar patrones de cambio en el comportamiento cotidiano no solo ayuda a prevenir situaciones de emergencia, sino que también permite personalizar la atención y adaptar el entorno a las necesidades reales de cada persona.

1.5 Objetivos del proyecto

El presente Trabajo de Fin de Grado (TFG) tiene como objetivo principal el desarrollo de un sistema inteligente capaz de detectar y reconocer determinadas actividades básicas de la vida diaria realizadas frente a un espejo. Esta solución está orientada especialmente a personas mayores que, debido al envejecimiento o al deterioro cognitivo, pueden presentar dificultades para mantener rutinas esenciales de autocuidado. La finalidad última es contribuir a su autonomía y

seguridad, permitiendo la monitorización pasiva de comportamientos relevantes mediante visión por computador e IA.

Para alcanzar este objetivo general, se establecen los siguientes objetivos específicos:

1. Desarrollar un sistema de captura de vídeo en tiempo real mediante una cámara integrada en un espejo inteligente.
2. Crear un conjunto de datos personalizado compuesto por secuencias de imágenes etiquetadas, obtenidas a partir de capturas reales de las acciones a reconocer.
3. Aplicar técnicas de visión por computador para extraer puntos clave (*keypoints*) del cuerpo humano, concretamente de la cara, torso, brazos y manos, utilizando herramientas como MediaPipe.
4. Entrenar y optimizar un modelo de aprendizaje automático capaz de reconocer acciones concretas como lavarse los dientes, peinarse, lavarse las manos o la cara, entre otras.
5. Implementar un sistema de inferencia en tiempo real que procese secuencias de imágenes para predecir la acción que se está realizando.
6. Desarrollar una aplicación web con una interfaz sencilla que permita visualizar en tiempo real la acción detectada y gestionar el funcionamiento general del sistema.
7. Registrar y almacenar las acciones detectadas junto con su duración, con el fin de generar un histórico útil para familiares o cuidadores.

Con este proyecto se busca sentar las bases para un sistema no invasivo de apoyo al cuidado personal, que pueda ser implantado tanto en hogares como en entornos residenciales, y que favorezca la detección temprana de posibles alteraciones funcionales en personas mayores.

Además de los objetivos principales anteriormente descritos, el proyecto contempla una serie de objetivos secundarios que, si bien no son estrictamente necesarios para el funcionamiento básico del sistema, aportan funcionalidades complementarias que enriquecen su utilidad, su escalabilidad futura y su aplicación en entornos reales:

1. Estudiar y aplicar técnicas de normalización espacial de los puntos clave obtenidos, con el fin de mejorar la robustez del modelo ante variaciones de tamaño, orientación y posición del usuario frente a la cámara.
 2. Implementar mecanismos de limpieza automática del conjunto de datos (como el descarte de capturas inválidas o incompletas) para garantizar la calidad del entrenamiento.
 3. Validar el modelo con usuarios no presentes en el conjunto de entrenamiento, con el fin de comprobar su capacidad de generalización ante sujetos nuevos.
 4. Registrar estadísticas de uso del sistema, tales como frecuencia y duración de las
-

actividades detectadas, con vistas a posibles análisis longitudinales o comportamiento evolutivo del usuario.

5. Optimizar el rendimiento del sistema para que funcione de forma fluida en dispositivos con recursos limitados, como una *Raspberry Pi*.
6. Diseñar el sistema de forma modular, que permita la incorporación futura de nuevas actividades o comportamientos a detectar sin necesidad de reconstruir la arquitectura completa.
7. Documentar el sistema con instrucciones claras de uso, instalación y mantenimiento, dirigidas tanto a perfiles técnicos como no técnicos.
8. Registrar eventos relevantes con marcas temporales y exportarlos a un archivo en formato Valores separados por comas, por sus siglas en inglés *Comma-Separated Values* (CSV) o Notación de objetos de JavaScript, por sus siglas en inglés *JavaScript Object Notation* (JSON) para su posterior análisis o revisión externa.
9. Investigar y analizar las principales técnicas de entrenamiento de modelos de IA, con especial atención a arquitecturas recurrentes como Memoria a largo y corto plazo, por sus siglas en inglés *Long Short-Term Memory* (LSTM) o Unidad recurrente simplificada, por sus siglas en inglés *Gated Recurrent Unit* (GRU), y su aplicabilidad en la detección de acciones humanas a partir de *keypoints*.

1.6 Estructura del documento

Este documento está estructurado en siete capítulos, cada uno de los cuales aborda una parte fundamental del desarrollo del sistema propuesto.

En el capítulo uno se presenta una introducción al proyecto, contextualizando su desarrollo dentro del ámbito de las smart homes y la inteligencia artificial aplicada a la asistencia de personas mayores. Se expone la motivación personal, el problema a resolver y los objetivos principales y secundarios que guían el TFG.

El capítulo dos está dedicado al estado del arte. Se analizan diferentes trabajos previos relacionados con espejos inteligentes, reconocimiento de acciones mediante visión por computador y aplicaciones similares en entornos asistenciales. Además, se detallan las contribuciones propias del proyecto con respecto a estas propuestas existentes.

En el capítulo tres se describen las distintas fases en las que se ha dividido el desarrollo del proyecto, incluyendo tanto la planificación inicial como la ejecución real y una comparativa entre ambas. Se detallan las tareas realizadas, los tiempos empleados y las desviaciones surgidas a lo largo del proceso.

El capítulo cuatro recoge las herramientas utilizadas en el desarrollo, abarcando lenguajes de

programación, bibliotecas, entornos de desarrollo y componentes hardware del espejo inteligente. Se justifica su elección y se explican sus características más relevantes para la implementación del sistema.

En el capítulo cinco se expone el desarrollo técnico del sistema. Se describe la arquitectura general, el proceso de captura y anotación de datos, el preprocesamiento y normalización de los mismos, el diseño y entrenamiento del modelo de IA, así como su integración en una aplicación web funcional que permite el uso del sistema en tiempo real. También se incluye el presupuesto estimado del proyecto.

El capítulo seis está dedicado a los resultados. Se evalúa el rendimiento del modelo en distintos escenarios, se comentan las dificultades encontradas y se analiza el comportamiento del sistema ante condiciones reales de uso, como variaciones de iluminación, obstáculos en el entorno o lateralidad de la acción.

Finalmente, en el capítulo siete se recogen las conclusiones extraídas del proyecto y se plantean posibles líneas de trabajo futuro para continuar mejorando el sistema, tanto en términos de precisión como de usabilidad y conectividad con otras tecnologías asistenciales.

2 Estado del Arte

2.1 Trabajos relacionados

El progresivo envejecimiento de la población ha impulsado en los últimos años el desarrollo de tecnologías orientadas a mejorar la calidad de vida y la autonomía de las personas mayores. En este contexto, los sistemas de asistencia basados en IA y visión por computador han cobrado una especial relevancia en aplicaciones como la detección de caídas, el reconocimiento de rutinas o la supervisión de actividades cotidianas. Uno de los dispositivos más investigados es el espejo inteligente, que combina sensores, cámaras, pantallas y algoritmos de procesamiento para interactuar con el usuario y proporcionarle información personalizada.

Entre los desarrollos más destacados se encuentra *CareMirror*, diseñado para motivar a personas mayores a realizar ejercicios físicos mediante retroalimentación visual en tiempo real [21]. De forma similar, *FitMirror* analiza la postura del usuario durante la actividad física y ofrece correcciones visuales para mejorar la ejecución de los movimientos [22]. Otra propuesta relevante es *Wize Mirror*, centrada en la salud metabólica y facial, que emplea sensores de imagen y análisis de voz para generar alertas relacionadas con riesgos cardiovasculares [23].

Más recientemente, el proyecto europeo *SHAPES Smart Mirror* ha abordado la asistencia a personas mayores desde una perspectiva más integral, incluyendo detección de caídas, control de actividad, conexión con cuidadores y recordatorios de salud, con especial énfasis en la accesibilidad de la interfaz y la privacidad de los datos [24]. Otros trabajos han explorado aspectos más técnicos, como el diseño adaptativo de espejos inteligentes para su integración en el entorno doméstico o su uso como interfaz médica no invasiva [25].

Fuera del ámbito del espejo, pero estrechamente relacionado, se han desarrollado sistemas específicos para la monitorización de la higiene personal. En el ámbito hospitalario, se han propuesto soluciones basadas en sensores de profundidad y visión por computador para verificar el uso de dispensadores de gel, alcanzando una precisión superior al 96% [26]. Del mismo modo, en entornos clínicos inteligentes, se han usado sistemas basados en IA y cámaras Color y profundidad, por sus siglas en inglés *Red Green Blue + Depth* (RGB-D) para detectar patrones de higiene y alertar ante conductas inadecuadas o potencialmente peligrosas [27]. Las revisiones más recientes señalan que muchas de estas tecnologías, aunque efectivas, presentan limitaciones en cuanto a su aplicabilidad doméstica, ya que suelen depender de sensores invasivos, requieren interacción manual o están diseñadas para entornos controlados [28].

Además, en el campo de los sistemas inteligentes basados en IoT, algunos trabajos han

abordado la detección de rutinas de aseo mediante sensores de movimiento, vibración o sonido, alcanzando buenas tasas de precisión pero sin contar con un componente visual ni capacidad de identificación específica de acciones como lavarse los dientes o la cara [29].

Por otro lado, algunos enfoques emergentes han comenzado a explorar el uso de técnicas de aprendizaje profundo aplicadas directamente a secuencias de imágenes, como el análisis de acciones humanas mediante modelos LSTM o GRU en el contexto de actividades cotidianas. Sin embargo, estos trabajos se centran mayoritariamente en datasets públicos de acciones genéricas como levantar objetos o sentarse, y rara vez abordan rutinas específicas de aseo personal. Esto limita su aplicabilidad a contextos domésticos reales, donde las variaciones de iluminación, ángulo de cámara o comportamiento del usuario son mucho más impredecibles. Tampoco suelen considerar la integración del sistema en un dispositivo físico como un espejo, ni aspectos de bajo consumo o usabilidad para usuarios con limitaciones cognitivas.

Asimismo, la mayoría de los estudios revisados no ofrecen una solución completa que integre captura, procesamiento, inferencia y visualización de resultados en un único dispositivo accesible. Por el contrario, tienden a fragmentar el sistema en distintas capas que dependen de recursos externos o infraestructura técnica avanzada. Esto plantea dificultades de escalabilidad y adopción en hogares reales, especialmente cuando se trata de usuarios mayores que requieren tecnologías invisibles, no intrusivas y completamente automatizadas.

2.2 Contribuciones del proyecto

En respuesta a este vacío identificado en la literatura, el presente proyecto propone el desarrollo de un espejo inteligente centrado específicamente en la monitorización de la higiene personal de personas mayores, como el cepillado de dientes, el lavado de cara o el peinado. A diferencia de la mayoría de los trabajos existentes, este sistema no requiere interacción activa por parte del usuario, ni depende de sensores portables ni de comandos de voz. En su lugar, emplea una cámara integrada que capta imágenes en tiempo real y, mediante la biblioteca *MediaPipe*, extrae puntos clave del cuerpo, manos y rostro que son posteriormente analizados por un modelo de aprendizaje profundo.

Una de las principales aportaciones del proyecto es la integración de un modelo Red neuronal convolucional, por sus siglas en inglés *Convolutional Neural Network* (CNN) combinado con GRU entrenado a partir de secuencias de 10 imágenes por acción, lo que permite captar la dinámica temporal de cada rutina de higiene. Este enfoque mejora sustancialmente la capacidad del sistema para distinguir acciones similares entre sí y proporciona una robustez superior frente a movimientos ambiguos o ruidos contextuales. Además, a diferencia de muchos proyectos que entrenan sus modelos con bases de datos públicas o imágenes genéricas, este trabajo ha creado un conjunto de datos propio, grabado con distintos usuarios simulando rutinas reales en un cuarto de baño doméstico. Este hecho incrementa la validez del modelo al ajustarse mejor al entorno y a la variabilidad natural de las acciones humanas.

Otra contribución relevante es la portabilidad del sistema. El modelo ha sido optimizado para

su ejecución en una Raspberry Pi utilizando TensorFlow Lite, lo que permite su despliegue en entornos reales sin necesidad de grandes recursos computacionales. Esto representa una ventaja respecto a otros desarrollos que requieren infraestructuras más costosas o entornos controlados. Junto a ello, el sistema incorpora un historial consultable desde una interfaz web sencilla, que permite verificar de manera rápida y accesible si las tareas de higiene se han realizado, facilitando el seguimiento por parte del propio usuario o de sus cuidadores.

Este proyecto, en cambio, ha sido concebido desde su inicio como una solución integral y realista, pensada para funcionar de forma autónoma en un entorno doméstico con los mínimos requisitos técnicos. La arquitectura modular del sistema permite que, una vez instalado, funcione de manera continua sin necesidad de configuración ni supervisión técnica, lo cual lo hace especialmente adecuado para personas con deterioro cognitivo o dependencia parcial. Además, al tratarse de un espejo (un objeto cotidiano en el espacio del baño), la integración en el entorno del usuario se realiza de forma natural, sin generar rechazo ni sensación de vigilancia.

Finalmente, la posibilidad de registrar el historial de actividades y de acceder a él desde una interfaz web sencilla supone un valor añadido respecto a otras soluciones. Esta característica no solo permite realizar un seguimiento de las rutinas de higiene, sino que también puede alertar de desviaciones en los hábitos, facilitando la intervención temprana por parte de familiares o personal sanitario. De este modo, el sistema no solo actúa como asistente pasivo, sino que también se convierte en una herramienta proactiva para el mantenimiento de la autonomía personal.

3 Fases del proyecto

El desarrollo del proyecto se ha dividido en diferentes fases con el objetivo de organizar de forma eficiente las tareas a realizar, estimar el esfuerzo requerido y analizar posteriormente la ejecución real del proyecto. Esta planificación ha permitido controlar el progreso del trabajo y ajustar la dedicación según las necesidades que fueron surgiendo durante su implementación.

3.1 Planificación

En este proyecto, se estimaron inicialmente un total de 300 horas. A continuación, se describen las fases que se definieron al comienzo del desarrollo, junto con la estimación de horas asignadas a cada una de ellas:

- **Inicio técnico y planificación documental (20 horas):** Tareas de configuración del entorno, planificación de la estructura del proyecto e inicio de la documentación técnica.
- **Obtención de *keypoints* y representación (30 horas):** Evaluación y puesta en marcha de MediaPipe para la extracción de puntos clave de manos, rostro y cuerpo. Se realizaron pruebas iniciales para validar la calidad de los datos obtenidos.
- **Captura y preparación del conjunto de datos (45 horas):** Grabación de sesiones de acciones de aseo, normalización de *keypoints*, representación gráfica sobre fondo blanco y organización del conjunto de datos por clases.
- **Desarrollo y entrenamiento del modelo de IA (50 horas):** Diseño de una arquitectura híbrida CNN-GRU, preparación de la red neuronal y entrenamiento con los datos recopilados.
- **Implementación del sistema en tiempo real (40 horas):** Integración del modelo en una aplicación en vivo que procesa imágenes en tiempo real, ejecuta la predicción y muestra el resultado sin interrupciones.
- **Desarrollo de la interfaz web y sistema de historial (35 horas):** Creación de una interfaz sencilla que permite visualizar la acción detectada en directo, así como almacenar y consultar el historial de actividades registradas por el sistema.
- **Redacción de la documentación y memoria del proyecto (50 horas):** Elaboración del documento final del TFG, estructurando y detallando todos los aspectos técnicos y

metodológicos del desarrollo.

- **Tutorías (30 horas):** Reuniones periódicas con el tutor académico para resolver dudas, validar decisiones y revisar la estructura y el contenido de la memoria.

En la Figura 3.1 se puede observar un desglose detallado de las tareas desarrolladas en cada fase del proyecto y el mes en el que se realizaron.

Tareas	Nov24	Dic24	Ene25	Feb25	Mar25	Abr25	May25	Jun25
Inicio técnico y planificación documental								
Pruebas con MediaPipe								
Creación del proyecto en Overleaf								
Configuración del entorno de trabajo								
Selección de librerías necesarias								
Obtención de keypoints y representación								
Integración de MediaPipe								
Extracción de keypoints y almacenamiento								
Función de visualización								
Evaluación desde distintos ángulos								
Desarrollo y entrenamiento del modelo de IA								
Diseño e implementación de arquitectura de entrenamiento								
Pruebas con distintos parámetros								
Implementación del sistema en tiempo real								
Captura de vídeo en vivo desde Raspberry Pi								
Normalización en tiempo real								
Control de cambio de acción								
Optimización del rendimiento								
Captura y preparación del conjunto de datos								
Grabación de acciones de aseo								
Almacenamiento de acciones por secuencias								
Normalización de coordenadas								
Generación de imágenes								
Estructuración de carpetas								
Limpieza y validación del dataset								
Desarrollo de la interfaz web y sistema de historial								
Desarrollo Web								
Generación de un historial de acciones								
Integración con sistema de ejecución del modelo en Raspberry Pi								
Documentación final								
Introducción								
Historia del Arte								
Fases								
Herramientas								
Desarrollo								
Resultado								
Conclusiones								
Tutorías								
Reuniones								

Figura 3.1: Cronograma estimado de la realización del proyecto.

3.2 Ejecución

El desarrollo del proyecto se llevó a cabo siguiendo la planificación inicial como referencia. No obstante, durante su ejecución surgieron diversos desafíos técnicos y ajustes que modificaron la distribución del tiempo entre tareas. Finalmente, el proyecto tuvo una duración total de 330 horas, distribuidas de la siguiente forma:

- **Inicio técnico y planificación documental (25 horas):** Configuración del entorno de desarrollo (Python, Flask, MediaPipe), estructuración de carpetas, control de versiones y primeros borradores de la memoria. Se evaluaron cámaras y se establecieron criterios para la captura de movimientos.
- **Obtención de *keypoints* y representación (35 horas):** Integración con MediaPipe para capturar los *keypoints* de interés. Se desarrollaron scripts para visualizar y exportar los puntos clave, ajustando parámetros para lograr una representación estable.
- **Captura y preparación del conjunto de datos (55 horas):** Se grabaron distintas acciones, se etiquetaron correctamente y se normalizaron los datos. Se automatizó la generación de imágenes a partir de *keypoints* sobre fondo blanco.
- **Desarrollo y entrenamiento del modelo de IA (60 horas):** Diseño, implementación y ajuste del modelo CNN-GRU. Se realizaron múltiples entrenamientos, análisis de métricas, cambios de modelos y ajustes de hiperparámetros para mejorar su precisión.
- **Implementación del sistema en tiempo real (45 horas):** Desarrollo del sistema que captura imágenes, normaliza los *keypoints* y pasa las secuencias al modelo para realizar predicciones. Se incluyó el registro automático de acciones detectadas.
- **Desarrollo de la interfaz web y sistema de historial (35 horas):** Creación de una interfaz web con Flask para mostrar la acción detectada y el historial almacenado. Se generó un sistema de almacenamiento accesible desde el navegador.
- **Redacción de la documentación y memoria del proyecto (50 horas):** Redacción estructurada de la memoria, incluyendo capturas, fragmentos de código y análisis de resultados. Se utilizó Overleaf como plataforma colaborativa.
- **Tutorías (25 horas):** Reuniones periódicas para seguimiento, revisión del enfoque y validación técnica de los avances realizados.

En la Figura 3.2 se puede observar el tiempo real desglosado por meses en el que se desarrolló cada tarea de las distintas fases del proyecto.

Tareas	Nov24	Dic24	Ene25	Feb25	Mar25	Abr25	May25	Jun25
Inicio técnico y planificación documental								
Pruebas con MediaPipe								
Creación del proyecto en Overleaf								
Configuración del entorno de trabajo								
Selección de librerías necesarias								
Obtención de keypoints y representación								
Integración de MediaPipe								
Extracción de keypoints y almacenamiento								
Función de visualización								
Evaluación desde distintos ángulos								
Desarrollo y entrenamiento del modelo de IA								
Diseño e implementación de arquitectura de entrenamiento								
Pruebas con distintos parámetros								
Implementación del sistema en tiempo real								
Captura de video en vivo desde Raspberry Pi								
Normalización en tiempo real								
Control de cambio de acción								
Optimización del rendimiento								
Captura y preparación del conjunto de datos								
Grabación de acciones de aseo								
Almacenamiento de acciones por secuencias								
Normalización de coordenadas								
Generación de imágenes								
Estructuración de carpetas								
Limpieza y validación del dataset								
Desarrollo de la interfaz web y sistema de historial								
Desarrollo Web								
Generación de un historial de acciones								
Integración con sistema de ejecución del modelo en Raspberry Pi								
Documentación final								
Introducción								
Historia del Arte								
Fases								
Herramientas								
Desarrollo								
Resultado								
Conclusiones								
Tutorías								
Reuniones								

Figura 3.2: Cronograma real de la realización del proyecto.

3.3 Comparativa

La Tabla 3.3 se muestra una comparativa entre las horas inicialmente planificadas y el tiempo real dedicado a cada fase del proyecto. Este análisis permite evaluar la precisión de la planificación inicial, identificar desviaciones y entender mejor qué tareas requirieron un mayor esfuerzo del previsto.

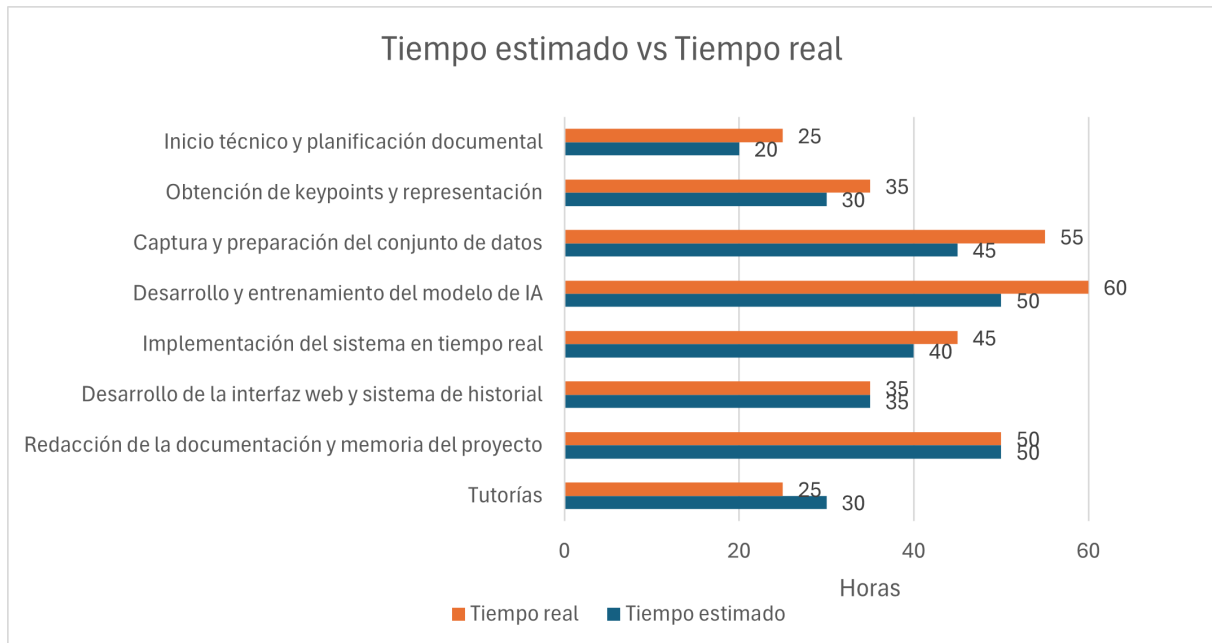


Figura 3.3: Gráfica comparativa entre tiempos.

Este análisis revela que, aunque algunas fases del proyecto se ajustaron razonablemente a las horas inicialmente previstas, otras presentaron desviaciones significativas. El mayor incremento de tiempo se produjo durante el desarrollo y entrenamiento del modelo de IA, debido a la necesidad de realizar múltiples pruebas para mejorar la precisión del sistema, así como a la recolección y depuración de datos adicionales no contemplados en la planificación inicial.

También se invirtió más tiempo del esperado en la implementación del sistema en tiempo real y en la elaboración de la documentación final, especialmente en lo relativo a la redacción técnica y la generación de gráficos y tablas ilustrativos. En cambio, tareas como la redacción de la documentación y memoria del proyecto o el desarrollo de la interfaz web y sistema de historial se mantuvieron dentro del margen previsto.

El desfase global se debe en parte a la naturaleza exploratoria del proyecto, en el que surgieron imprevistos técnicos que requirieron reajustes y adaptaciones constantes, como la integración entre distintos módulos (MediaPipe, cámara, interfaz web) o la necesidad de normalizar correctamente los datos para su uso en el modelo.

4 Herramientas

En este capítulo se va a realizar una descripción de las herramientas que se han usado para desarrollar el proyecto. Cada una de las herramientas seleccionadas cumplía una serie de características y requisitos para satisfacer las necesidades del proyecto. A continuación, se detallan los lenguajes de programación, los medios de comunicación, los entornos de desarrollo y la infraestructura utilizados.

4.1 Lenguajes de programación

4.1.1 Python

Se trata de un lenguaje de programación muy usado en aplicaciones web, desarrollo de software y Machine Learning. Alcanzó su popularidad gracias a su eficiencia y facilidad para usarse, además de poder ser ejecutado en múltiples plataformas. Se puede descargar de forma gratuita y se integra en todos los sistemas, aumentando la velocidad de desarrollo.

Es el principal lenguaje de programación utilizado en este proyecto, ya que cuenta con diversas bibliotecas muy útiles para el procesamiento de imágenes, obtención de puntos del cuerpo humano y entrenamiento de IA.

4.1.1.1 Beneficios de usar Python

Entre los beneficios de usar Python como el principal lenguaje de programación del proyecto, podemos destacar su simplicidad, puesto que tiene una sintaxis más simple e intuitiva que otros lenguajes, facilitando su aprendizaje y uso. Gracias a esta simplicidad, también se acelera el desarrollo de código y la generación de proyectos [30].

Además, cuenta con una gran cantidad de recursos disponibles. Existen muchas bibliotecas y marcos de trabajo que han sido desarrollados y correctamente documentados, y permiten incorporar utilidades a nuestros proyectos rápidamente.

Por último, cabe destacar que su rendimiento ha mejorado notablemente en los últimos años gracias a bibliotecas que permiten manejar grandes volúmenes de datos de manera eficiente.

4.1.1.2 Bibliotecas utilizadas

Durante la elaboración del proyecto se han usado varias bibliotecas de Python fundamentales para la implementación de los modelos de IA, obtener las coordenadas de las distintas partes del cuerpo y el manejo de datos:

- **MediaPipe:** La biblioteca MediaPipe fue diseñada para procesar datos a través de imágenes. Proporciona aprendizaje automático para detectar la pose del cuerpo y sus extremidades a través de puntos clave de las articulaciones en tiempo real. En este proyecto, se usó para detectar y seguir la posición de las manos, pose corporal y la cara. Obtiene 21 *keypoints* de las manos, 6 del tronco superior del cuerpo y 478 de la cara, extrayendo sus coordenadas para su posterior uso. Además, gracias a otra función que incorpora esta biblioteca, podremos visualizar esos puntos y sus conexiones sobre la imagen prácticamente al instante [31].
- **OpenCV:** Esta biblioteca es ampliamente usada en el procesamiento de imágenes y vídeos. Permite capturar, ver, preprocesar y transformar imágenes. En este proyecto, se usó OpenCV para obtener vídeo en tiempo real desde la cámara, convertir imágenes de formato BGR a RGB, mostrar los resultados procesados y guardar imágenes de este vídeo [32].
- **CSV:** Es una biblioteca que ayuda a crear, leer y escribir en archivos de formato CSV. Se utilizó para almacenar los *keypoints* detectados junto con las coordenadas correspondientes y otros datos de interés [33].
- **Os:** La biblioteca os permite interactuar con el sistema operativo. En este proyecto, se utilizó para gestionar las rutas de archivos y crear directorios si no existían previamente, facilitando la organización de imágenes y datos [34].
- **Time:** Time es una biblioteca que proporciona herramientas para la obtención del tiempo. Se utilizó para obtener la hora real cuando se capturaban los datos de *keypoints* con sus imágenes asociadas [35].

4.1.2 L^AT_EX

L^AT_EX es una herramienta para componer documentos de aspecto profesional. A diferencia de otros procesadores de texto, L^AT_EX está basado en comandos, donde el usuario define la estructura del documento mediante etiquetas, capítulos, secciones, tablas y figuras. Es muy usado en la comunidad científica, ya que este enfoque garantiza una presentación uniforme y agiliza la incorporación de contenido complejo como fórmulas, gráficos o redactar la bibliografía [36].

A diferencia de otros procesadores de texto como Word o Libre Office, en L^AT_EX el documento es un archivo sin formato y se aplican comandos propios para dar la apariencia deseada. Se basa

en el lenguaje de programación $\text{\TeX}$ y separa el contenido del aspecto visual. El usuario sólo tiene que enfocarse en el contenido, mientras que, a través de $\text{\TeX}$ y sus comandos, se formará el aspecto visual.

Una de sus grandes ventajas es que está disponible para la mayoría de los sistemas operativos con Windows, Linux o Mac OS, siendo mucho más portátil que su competencia. Los documentos creados mantienen la misma apariencia en diferentes dispositivos y plataformas. Otro aspecto a favor de $\text{\LaTeX}$ es que se trata de un programa de código abierto, en el que los usuarios pueden acceder y modificar sus fuentes libremente. Esta característica ha generado una comunidad de usuarios y desarrolladores que contribuyen a su mejora constantemente.

Por último, cabe destacar que existen muchos recursos y foros donde los principiantes pueden preguntar y obtener ayuda de otros usuarios más expertos, facilitando el aprendizaje y el uso de este sistema.

4.1.2.1 Overleaf

Overleaf es una herramienta de escritura que usa el editor de $\text{\LaTeX}$. Permite la colaboración en línea de documentos, haciendo que sea mucho más fácil y rápido la escritura, edición y publicación de archivos. Es fácil de usar y se obtiene una salida automáticamente compilada a medida que se escribe.

Al ser una plataforma Web, no requiere instalación o actualizaciones, funciona en todos los entornos y con cualquier dispositivo, es una herramienta colaborativa y fácil de compartir documentos. Además, dado que el compilador está integrado, la visualización del resultado es inmediata, con posibilidad de proyectos ilimitados, historial completo de cambios en el documento y con posibilidad de restaurar versiones anteriores.

4.2 Comunicación

En este apartado se describen los medios empleados para mantener la comunicación entre los tutores y la alumna durante el desarrollo del proyecto. La coordinación ha jugado un papel esencial para garantizar el correcto avance del trabajo, permitiendo un control constante y resolviendo de forma eficiente cualquier incidencia o duda que pudiera surgir.

4.2.1 Correo electrónico

El correo electrónico ha constituido uno de los principales canales de interacción entre la alumna y el tutor. A través de él se han compartido archivos relevantes, entregas de avances y se han resuelto consultas puntuales. También ha servido para enviar informes sobre el progreso del proyecto y recibir observaciones o recomendaciones. Al tratarse de un medio asíncrono,

ha permitido una comunicación eficaz incluso cuando no era posible coincidir en tiempo real, resultando muy útil en momentos de mayor carga de trabajo o incompatibilidad horaria.

4.2.2 Reuniones

A lo largo del proyecto se han realizado reuniones con una frecuencia variable, siendo mayormente mensuales durante las primeras fases y aumentando en intensidad a medida que se acercaba la fecha de entrega. Estas sesiones han sido fundamentales para mantener una dinámica de trabajo ordenada y dirigida hacia los objetivos planteados.

En las reuniones se han tratado temas técnicos, se han buscado soluciones a obstáculos detectados, y se ha reajustado el cronograma cuando ha sido necesario. Estas se han llevado a cabo tanto de forma presencial en la universidad como de manera virtual mediante la plataforma *Google Meet*. Esta flexibilidad ha permitido una comunicación constante sin que las limitaciones geográficas o de disponibilidad condicionaran el progreso del proyecto.

4.3 Desarrollo

En este apartado se describirán los entornos de desarrollo y herramientas utilizadas para la parte de desarrollo de código del proyecto. Además, se justificará su uso en lugar de otros con características similares.

4.3.1 Visual Studio Code

Visual Studio Code ha sido el principal editor de código utilizado para el proyecto. A pesar de que el único lenguaje de programación usado para este trabajo haya sido Python, este software ofrece una amplia gama de lenguajes, pudiendo adaptarse a las necesidades de todos los usuarios y proyectos.

Se trata de un Entorno de desarrollo integrado, por sus siglas en inglés *Integrated Development Environment* (IDE) cuya principal característica es su rapidez y capacidad de personalización. Una de sus funcionalidades más importantes recae sobre sus herramientas de depuración, ejecución de scripts desde el editor y el autocompletado de código. Incluye una gran cantidad de *plugins*, extensiones y complementos con el objetivo de hacer la experiencia de trabajo mucho más intuitiva y personalizada.

4.3.2 Git

Git se trata de un sistema de control de versiones de código abierto. GitHub es una plataforma donde almacenar, trabajar y compartir junto a otros usuarios código. Cuando se carga un archivo

en GitHub, lo estamos almacenando en un repositorio de Git. Al hacer cambios en los archivos de GitHub, Git se inicia automáticamente y realiza un seguimiento de los cambios [37].

Muchas de las acciones de Git se pueden hacer directamente en GitHub desde un navegador, como cargar y editar archivos y crear un repositorio o ramas dentro de él. Permite trabajar en archivos localmente, desde nuestro dispositivo, y mantenerlo sincronizado constantemente.

Esta herramienta ha sido especialmente útil para controlar el código y los cambios que se han ido realizando sobre el mismo. Gracias a ello, se han podido volver a versiones anteriores cuando ha sido necesario sin perder ningún dato o línea de código.

4.4 Infraestructura

El desarrollo del proyecto se ha apoyado en entornos de programación interactivos ampliamente utilizados en ciencia de datos y aprendizaje automático. Concretamente, se trabajó inicialmente en Google Colab, para posteriormente migrar a un entorno local basado en Jupyter Notebook. Esta decisión estuvo motivada por las necesidades crecientes de rendimiento, control y flexibilidad del sistema conforme avanzaba la fase de entrenamiento y validación.

4.4.1 Google Colab

Google Colab (Colaboratory) es una plataforma gratuita ofrecida por Google que permite ejecutar cuadernos Jupyter en la nube, sin necesidad de configuración local. Se integra fácilmente con Google Drive y proporciona acceso a aceleradores de hardware como Unidad de procesamiento gráfico, por sus siglas en inglés *Graphics Processing Unit* (GPU) y Unidad central de procesamiento, por sus siglas en inglés *Central Processing Unit* (CPU), lo que la convierte en una herramienta atractiva para el desarrollo rápido de proyectos de IA y aprendizaje profundo [38].

Durante las primeras etapas del proyecto, Colab fue utilizado para:

- Implementar y probar versiones iniciales del modelo con datasets reducidos.
- Visualizar los *keypoints* generados con MediaPipe.
- Generar gráficas de pérdida y precisión durante el entrenamiento.
- Exportar el modelo entrenado a formato TensorFlow Lite.

No obstante, el uso de Google Colab también presentó ciertas limitaciones:

- Las sesiones se desconectan automáticamente tras un período de inactividad o después de varias horas de uso.
-

- El acceso a GPU no está garantizado y depende de la disponibilidad del momento.
- La necesidad de cargar y descargar datos desde Google Drive ralentiza el preprocesamiento de grandes volúmenes de imágenes.

Estas restricciones supusieron un obstáculo para una experimentación más intensiva, lo que motivó el cambio de entorno.

4.4.2 Jupyter Notebook

Jupyter Notebook es una herramienta de código abierto que permite crear cuadernos interactivos con código, texto, fórmulas matemáticas y visualizaciones. Al ejecutarse en local, otorga al usuario un mayor control sobre el entorno de ejecución, incluyendo el uso de bibliotecas personalizadas, la gestión de archivos y la integración con otros sistemas externos [39].

El cambio a Jupyter permitió:

- Ejecutar el código sin las restricciones temporales de Colab.
- Leer y escribir imágenes directamente en el sistema de archivos, mejorando la velocidad de preprocesamiento.
- Integrar fácilmente componentes como Flask para la inferencia en tiempo real, OpenCV para visualización y MediaPipe para la detección de *keypoints*.
- Entrenar modelos más complejos y hacer ajustes más finos en los hiperparámetros sin preocuparse por los límites de uso de recursos en la nube.

Este entorno local facilitó un flujo de trabajo más ágil y estable en las fases críticas del proyecto, especialmente durante el entrenamiento completo del modelo con el conjunto de datos definitivo y su posterior validación.

5 Desarrollo del sistema

En este capítulo se describe el proceso seguido durante el desarrollo del sistema, especificando el uso de las herramientas mencionadas en secciones anteriores, así como los programas creados y las funcionalidades incorporadas.

5.1 Arquitectura del sistema

El objetivo principal es implementar un sistema de visión por computador integrado en un espejo inteligente, que permita el reconocimiento automatizado de actividades que el usuario realiza. Además, se almacenará la información obtenida, de tal forma que los usuarios puedan consultar si se realizó alguna actividad o cuánto tiempo transcurrió en su desarrollo.

Para reconocer los movimientos del usuario, se usarán puntos clave de la pose, cara y manos. Una cámara incorporada al espejo será la encargada de captarlos, obteniendo imágenes en tiempo real, para ejecutar el modelo de visión por computadora. En la Figura 5.1 se muestra el funcionamiento del sistema.

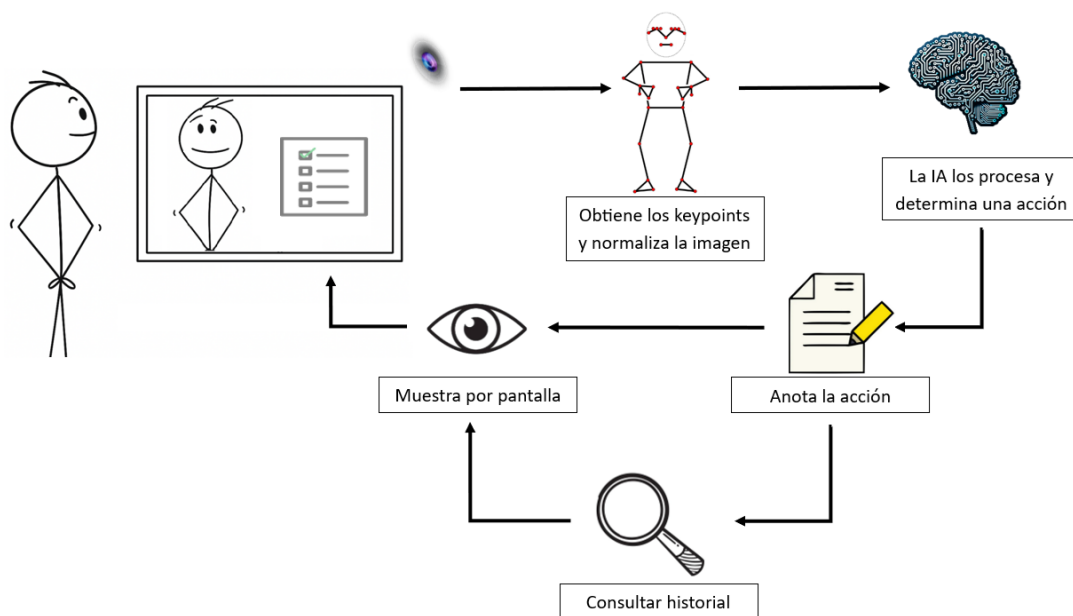


Figura 5.1: Arquitectura del sistema.

La arquitectura del sistema sigue un modelo de arquitectura cliente-servidor, donde el servidor procesa las imágenes capturadas y posteriormente las trata para identificar la acción que realiza a través de los puntos clave del cuerpo. El espejo, que actúa como cliente, permite al usuario consultar qué acciones ha realizado. Los datos son persistentes y se van actualizando para su almacenamiento.

5.2 Sistema de adquisición de datos anotados

Para que nuestro modelo sea capaz de detectar un cuerpo y la acción que está realizando, previamente tenemos que realizar una serie de pasos.

5.2.1 Modelos para la estimación de pose

La extracción de puntos clave es uno de los componentes fundamentales en el diseño del sistema de detección de acciones. Este proceso permite transformar las imágenes captadas por una cámara en una representación abstracta del cuerpo humano, compuesta por coordenadas que indican la posición de partes específicas como las manos, el rostro o las principales articulaciones. Esta representación simplificada permite centrar el análisis en los movimientos y posturas del cuerpo, ignorando aspectos irrelevantes como el fondo, la iluminación o el color de la ropa, que podrían interferir en la clasificación de acciones.

Este componente se sitúa en las primeras etapas del flujo del sistema, justo después de la captura de vídeo. Su principal función es reducir la complejidad de los datos visuales y generar una entrada mucho más estructurada y manejable para los algoritmos de IA. Si se trabajara directamente con imágenes, el volumen de información sería muy elevado y más sensible a las condiciones externas. En cambio, al extraer solo las posiciones clave del cuerpo, se obtiene una descripción más robusta y eficiente para detectar patrones de movimiento.

El resultado de esta fase no son imágenes procesadas ni píxeles, sino una lista de coordenadas (en dos o tres dimensiones) que representan, por ejemplo, la posición de la muñeca, los ojos, los dedos o la cadera. Estas coordenadas se pueden almacenar y analizar a lo largo del tiempo para reconstruir el movimiento realizado por la persona.

Este enfoque ofrece varias ventajas importantes:

- Elimina información visual innecesaria y reduce el ruido.
 - Disminuye notablemente el tamaño de los datos, lo que facilita el entrenamiento del modelo y mejora su velocidad.
 - Aumenta la capacidad del sistema para generalizar a distintos usuarios, entornos y condiciones.
-

- Permite detectar con mayor precisión secuencias de movimientos que caracterizan acciones concretas.

Existen diversas tecnologías para realizar esta extracción de puntos clave, pero en este proyecto se ha optado por utilizar una herramienta especializada en visión por computadora que permita identificar automáticamente las posiciones del rostro, las manos y el cuerpo entero a partir de cada imagen del vídeo.

5.2.1.1 MediaPipe

MediaPipe es un framework de código abierto desarrollado por Google Research para el procesamiento de medios en tiempo real, centrado principalmente en tareas de visión por computador y aprendizaje automático [40]. Su objetivo es ofrecer soluciones precisas, eficientes y multiplataforma que permitan detectar y seguir puntos clave del cuerpo humano, la cara o las manos, con una arquitectura fácilmente integrable tanto en dispositivos móviles como en ordenadores de escritorio o entornos embebidos.

El funcionamiento de MediaPipe se basa en una arquitectura modular estructurada en forma de grafo, donde cada nodo, denominado *calculator*, realiza una tarea específica: captura de imagen, preprocesamiento, inferencia de un modelo de Machine Learning, postprocesamiento o visualización. Esta estructura permite diseñar flujos de trabajo flexibles y altamente optimizados, denominados *pipelines*, que pueden ejecutarse de forma eficiente y en tiempo real incluso en dispositivos con recursos limitados [41].

Una de sus características clave es su compatibilidad con múltiples plataformas. Está diseñado para ejecutarse en sistemas operativos de escritorio (Linux, Windows, macOS), en dispositivos móviles (Android e iOS), en navegadores mediante WebAssembly, e incluso en dispositivos de bajo consumo como Raspberry Pi o chips Edge TPU. Además, proporciona interfaces en varios lenguajes de programación: C++, Python y JavaScript, lo cual facilita su adopción en todo tipo de proyectos.

Google ha desarrollado dentro de MediaPipe una serie de soluciones preconstruidas y optimizadas que permiten abordar tareas específicas de visión por computador sin necesidad de entrenar modelos desde cero. Las principales soluciones disponibles son:

- **MediaPipe Hands:** detecta y rastrea hasta 21 puntos clave por mano con alta precisión y robustez frente a oclusiones o deformaciones. Está optimizada para ejecutarse en tiempo real y es utilizada en interfaces gestuales, control sin contacto, realidad aumentada y educación interactiva [42].
 - **MediaPipe Pose:** permite detectar la pose completa del cuerpo humano mediante 33 puntos clave distribuidos entre cabeza, torso, extremidades y pies. Es especialmente útil en aplicaciones de análisis de movimiento, rehabilitación, deporte, danza y ergonomía [43].
 - **MediaPipe Face Mesh:** genera una malla tridimensional densa del rostro humano con
-

más de 468 puntos clave. Captura con precisión contornos, labios, ojos, cejas y mandíbula, lo que la convierte en una solución ideal para animación facial, avatares en tiempo real, filtros faciales o análisis emocional.

- **MediaPipe Face Detection:** realiza detección rápida de rostros en imágenes, devolviendo cajas delimitadoras y puntos clave básicos. Es útil para aplicaciones de desbloqueo facial, seguimiento de atención o filtros automáticos de cámara [44].
- **MediaPipe Holistic:** combina las soluciones de manos, pose y malla facial en un único pipeline, permitiendo la estimación integral de la postura y expresividad corporal del usuario. Esta integración completa es clave en contextos como avatares digitales, realidad virtual, monitorización biométrica o sistemas de asistencia.

MediaPipe fue creado inicialmente como una herramienta interna por parte de Google para acelerar el prototipado de soluciones de visión por computador, y más tarde fue liberado como proyecto de código abierto para facilitar su uso por parte de la comunidad investigadora y desarrolladora. Los modelos que utiliza han sido entrenados sobre grandes volúmenes de datos y, posteriormente, optimizados utilizando técnicas como:

- *Model Pruning:* eliminación de conexiones o nodos innecesarios en la red neuronal para reducir su complejidad.
- *Quantization:* conversión de pesos de precisión flotante a formatos más compactos (por ejemplo, int8) para acelerar la inferencia.
- *Knowledge Distillation:* transferencia de conocimiento desde modelos grandes y precisos a modelos más pequeños y rápidos.

Además, muchos de estos modelos han sido convertidos a TensorFlow, el cual está especialmente diseñado para ejecutarse de forma eficiente en dispositivos móviles o con recursos computacionales limitados. Esta combinación de modelos optimizados y ejecución eficiente hace que MediaPipe pueda operar incluso en tiempo real, sin requerir GPU dedicadas.

Las principales ventajas que ofrece MediaPipe incluyen:

- **Ejecución en tiempo real** con latencia muy baja.
- **Compatibilidad multiplataforma** y multilenguaje.
- **Facilidad de integración** gracias a su arquitectura modular y sus interfaces claras.
- **Disponibilidad de modelos preentrenados**, evitando la necesidad de entrenamiento propio.
- **Flexibilidad y personalización** de cada componente del pipeline.

Gracias a estas características, MediaPipe se ha convertido en una herramienta esencial para

el desarrollo de sistemas avanzados de visión artificial, como espejos inteligentes, asistentes virtuales, videojuegos interactivos, sistemas biométricos, entornos educativos con feedback gestual, análisis postural y monitorización de pacientes. Su capacidad para combinar precisión, eficiencia y facilidad de uso lo convierte en una de las soluciones más completas disponibles actualmente para el reconocimiento de acciones y comprensión del movimiento humano.

5.2.2 Creación del dataset

La creación de un conjunto de datos, también llamado dataset, es un paso fundamental en el desarrollo de modelos de reconocimiento de patrones basados en puntos clave del cuerpo humano. En este proyecto, el dataset está formado por un conjunto de coordenadas obtenidas a partir de los *keypoints* del cuerpo, las manos y la cara. Estos puntos se obtienen en tiempo real a través de la cámara y, posteriormente, se emplearán para entrenar un modelo de reconocimiento de actividades.

Para asegurar que los datos se recopilan correctamente, se ha implementado un sistema basado en la biblioteca *MediaPipe*, que permite procesar la imagen para detectar y extraer dichos puntos clave del cuerpo con alta precisión. Además, para facilitar el entrenamiento supervisado del modelo, se ha elaborado un etiquetado de datos a través de la asociación de cada captura con una acción específica.

Para poder elaborar el dataset, es necesaria la captura continua de datos mientras el sujeto realiza diversas acciones. Cada frame del vídeo es procesado para extraer las coordenadas de los *keypoints* importantes. Posteriormente, son almacenadas junto con una etiqueta de acción en un archivo CSV. Gracias a esto, podemos garantizar la creación de un dataset estructurado y correctamente etiquetado para su uso en próximos pasos del proyecto.

En la Figura 5.2 se muestra un diagrama del flujo completo del tratamiento de las imágenes.

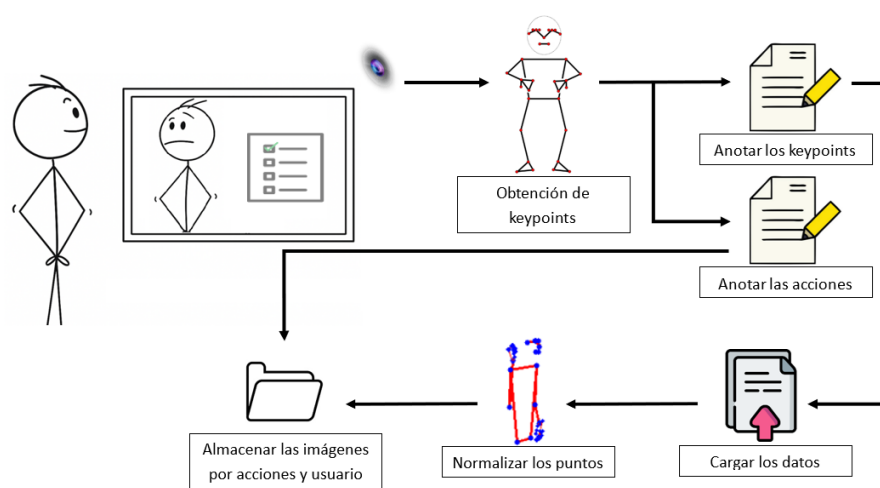


Figura 5.2: Diagrama de la creación del dataset.

5.2.2.1 Obtención de las imágenes

Uno de los principales desafíos a la hora de crear un modelo de reconocimiento de acciones es la imposibilidad de entrenar un sistema que sea capaz de identificar cualquier tipo de actividad, puesto que existe una gran cantidad. Es inviable desarrollar un modelo universal que reconozca todas las acciones de manera precisa.

Teniendo en cuenta esta limitación, se ha adoptado un enfoque práctico, centrado en un conjunto de acciones específicas que reflejan la rutina de aseo que un usuario puede realizar diariamente frente a un espejo. De esta manera, el sistema estará optimizado para reconocer acciones más frecuentes, sin la necesidad de abarcar una gama excesivamente amplia de opciones.

Dado que la cámara oficial de la Raspberry Pi no es accesible mediante la interfaz estándar, fue necesario utilizar *libcamera-vid* para realizar la captura de vídeo. El primer paso consiste en lanzar dicha biblioteca desde el propio script, configurando su salida como un *stream* Protocolo de control de transmisión, por sus siglas en inglés *Transmission Control Protocol (TCP)* que pueda ser leído por OpenCV.

Además, para solventar uno de los principales retos de incorporar este programa a la Raspberry Pi: el retardo (*delay*) en la visualización y procesamiento de las imágenes capturadas, fue necesario optimizar el proceso de captura de vídeo y la carga de procesamiento en la placa. Se añadieron varias configuraciones adicionales: desactivación de la ventana de previsualización, resolución a 1640x1232 píxeles, limitación del bitrate a 2000000 *Mbps* y reducción de la tasa de frames a 1 Imágenes por segundo, por sus siglas en inglés *Frames Per Second (FPS)*. Esta configuración se puede apreciar en el Código 5.1.

Código 5.1: Configuración de cámara.

```
1 libcamera_process = subprocess.Popen([
2     'libcamera-vid', '-t', '0', '--inline', '--listen', '--nopreview',
3     '-o', 'tcp://127.0.0.1:8888',
4     '--width', '1640', '--height', '1232',
5     '--bitrate', '2000000', '--framerate', '1'
6 ])
7 time.sleep(2)
```

El parámetro *-nopreview* evita que se abra una ventana duplicada con la imagen en crudo, es decir, sin los puntos clave señalados. La reducción de resolución y disminuye el volumen de datos transmitidos, facilitando su procesamiento. Se incluye una espera de dos segundos para asegurar que el *stream* esté disponible antes de continuar.

A continuación, como ya tenemos la configuración para la captura de vídeo especificada, se establece conexión desde OpenCV utilizando el *stream TCP* generado (Código 5.2).

Código 5.2: Inicio de cámara.

```
1 cap = cv2.VideoCapture('tcp://127.0.0.1:8888', cv2.CAP_FFMPEG)
2
```

```

3 if not cap.isOpened():
4     print("No se pudo abrir la cámara (libcamera-vid)")
5     libcamera_process.terminate()
6     exit()

```

Se fuerza el uso del *backend FFMPEG*, una biblioteca de software muy utilizada para gestionar audio, vídeos y *stream*, para garantizar la compatibilidad. Si la conexión falla, el proceso de *libcamera* se termina y el programa finaliza, evitando errores posteriores.

Una vez establecida la conexión, se inicializan los modelos de MediaPipe necesarios para la detección de *keypoints* de manos y de pose corporal, ilustrado en el Código 5.3.

Código 5.3: Detección de keypoints.

```

1 mp_hands = mp.solutions.hands
2 mp_pose = mp.solutions.pose
3 mp_drawing = mp.solutions.drawing_utils
4
5 hands = mp_hands.Hands(static_image_mode=False, max_num_hands=2,
6                        min_detection_confidence=0.5, min_tracking_confidence=0.5)
7 pose = mp_pose.Pose(static_image_mode=False, min_detection_confidence=0.5)

```

Posteriormente, se preparan los archivos CSV para almacenar los datos capturados. Estos ficheros se abren en modo *append* para poder añadir líneas de información, de tal forma que no se sobrescriban datos anteriores (Código 5.4).

Código 5.4: Almacenar los datos en archivos CSV.

```

1 hand_csv_path = os.path.join(current_dir, 'hand_keypoints.csv')
2 pose_csv_path = os.path.join(current_dir, 'pose_keypoints.csv')
3 acciones_csv_path = os.path.join(current_dir, 'acciones_tecclas.csv')
4
5 with open(hand_csv_path, 'a', newline='', encoding='utf-8-sig') as hand_file, \
6     open(pose_csv_path, 'a', newline='', encoding='utf-8-sig') as pose_file, \
7     open(acciones_csv_path, 'a', newline='', encoding='utf-8-sig') as ↵
8     ↵ acciones_file:

```

Dentro del bucle principal del script, se capturan las fotografías, se procesan con MediaPipe y se almacenan los resultados, como se muestra en el Código 5.5.

Código 5.5: Captura, procesamiento y almacenamiento de imágenes.

```

1 ret, frame = cap.read()
2 if not ret:
3     print("No se pudo capturar imagen")
4     break
5
6 timestamp = int(time.time() * 1000)
7
8 image_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
9 image_rgb.flags.writeable = False

```

```

10
11 hand_results = hands.process(image_rgb)
12 pose_results = pose.process(image_rgb)
13
14 image_rgb.flags.writeable = True
15 frame = cv2.cvtColor(image_rgb, cv2.COLOR_RGB2BGR)

```

Cada imagen capturada se convierte al formato RGB requerido por MediaPipe. Los resultados de la detección de manos y pose se procesan para extraer los *keypoints* y guardarlos en sus respectivos archivos (Código 5.6).

Código 5.6: Almacenamiento de keypoints.

```

1 if hand_results.multi_hand_landmarks:
2     for hand_idx, hand_landmarks in enumerate(hand_results.multi_hand_landmarks):
3         for kp_idx, landmark in enumerate(hand_landmarks.landmark):
4             hand_writer.writerow([timestamp, hand_idx, hand_keypoints_names[kp_idx]↵
5                                     ↵ ], landmark.x, landmark.y, landmark.z])
6             mp_drawing.draw_landmarks(frame, hand_landmarks, mp_hands.HAND_CONNECTIONS↵
7                                     ↵ )
8
9 if pose_results.pose_landmarks:
10     for kp_idx, landmark in enumerate(pose_results.pose_landmarks.landmark):
11         if kp_idx in pose_keypoints_names:
12             pose_writer.writerow([timestamp, pose_keypoints_names[kp_idx], landmark↵
13                                     ↵ .x, landmark.y, landmark.z])
14
15 mp_drawing.draw_landmarks(frame, pose_results.pose_landmarks, mp_pose.↵
16                             ↵ POSE_CONNECTIONS)

```

El almacenamiento de los puntos clave se realiza en archivos CSV estructurados, lo que permite una posterior organización y análisis más eficientes. Cada fila del archivo contiene una marca de tiempo única para cada captura (*timestamp*), el nombre del punto detectado (por ejemplo, “muñeca”, “codo izquierdo”, etc.) y sus coordenadas tridimensionales (X, Y, Z), que representan la posición del *keypoint*. En el caso de los datos de las manos, se incluye una columna adicional que indica si el punto pertenece a la mano derecha o izquierda, utilizando los valores 1 y 2, respectivamente.

Para facilitar el análisis, se han generado tres archivos CSV independientes: uno para la pose del cuerpo, otro para las manos y otro para la cara. Esto permite trabajar de forma segmentada según el tipo de movimiento que se desee estudiar. En la Figura 5.3, Figura 5.4 y Figura 5.5 se muestra un ejemplo del contenido de cada uno de estos archivos.

Además, para anotar la acción realizada por el usuario en cada instante, se implementó un sistema de etiquetado mediante teclado. Así, durante la captura de imágenes, se puede pulsar una tecla que asocia el *timestamp* con una acción concreta (por ejemplo, la tecla “1” para lavarse los dientes). Esta etiqueta se guarda en un archivo CSV adicional, separado de los archivos de puntos clave, lo que permite entrenar el modelo con supervisión explícita.

```

face_keypoints.csv U X
Código > Dataset > face_keypoints.csv
1  Timestamp,Face_Keypoint_Index,X,Y,Z
2  1742755227891,0,0.5092255473136902,0.5659316778182983,-0.02345515415072441
3  1742755227891,1,0.5123063921928406,0.5191307663917542,-0.044456783682107925
4  1742755227891,2,0.5104042291641235,0.5311037302017212,-0.02260485664010048
5  1742755227891,3,0.5057763457298279,0.47731566429138184,-0.034935321658849716
6  1742755227891,4,0.5129489898681641,0.5065650343894958,-0.0476420521736145
7  1742755227891,5,0.5132107138633728,0.4898092448711395,-0.04463472217321396
8  1742755227891,6,0.513206422328949,0.4483429789543152,-0.023107675835490227

```

Figura 5.3: Archivo de la cara.

```

pose_keypoints.csv U X
Código > Dataset > pose_keypoints.csv > data
1  Timestamp,Nombre_Keypoint,X,Y,Z
2  1742755227891,Hombro_Izquierdo,0.6857457756996155,0.8077876567840576,-0.1366874170303345
3  1742755227891,Hombro_Derecho,0.27735885977745056,0.8020980954170227,-0.276568740606308
4  1742755227891,Codo_Izquierdo,0.7738092541694641,1.1939254999160767,-0.12421576678752899
5  1742755227891,Codo_Derecho,0.18545496463775635,1.2260807752609253,-0.21675905585289001
6  1742755227891,Muñeca_Izquierda,0.7653536796569824,1.461167812347412,-0.3646475076675415
7  1742755227891,Muñeca_Derecha,0.20472517609596252,1.4795918464660645,-0.45062145590782166
8  1742755228706,Hombro_Izquierdo,0.6868695020675659,0.808035671710968,-0.15970614552497864

```

Figura 5.4: Archivo del cuerpo.

```

hand_keypoints.csv U X
Código > Dataset > hand_keypoints.csv > data
1  Timestamp,Mano,Nombre_Keypoint,X,Y,Z
2  1742755233120,0,Muñeca,0.4567255675792694,1.0133509635925293,-1.3691956723960175e-07
3  1742755233120,0,Pulgar_Base,0.4465816915035248,0.9714649319648743,-0.013677980750799179
4  1742755233120,0,Pulgar_Medio,0.44395357370376587,0.9288992285728455,-0.02889631688594818
5  1742755233120,0,Pulgar_Punta,0.4603124260902405,0.8993768095970154,-0.045042891055345535
6  1742755233120,0,Pulgar_Extremo,0.4831033945083618,0.8908300399780273,-0.061307765543460846
7  1742755233120,0,Indice_Base,0.395938515663147,0.9515606164932251,-0.027361147105693817
8  1742755233120,0,Indice_Medio,0.42788010835647583,0.8960078954696655,-0.05320659279823303

```

Figura 5.5: Archivo de las manos.

Por último, al finalizar la ejecución, se liberan todos los recursos utilizados, cerrando las conexiones de vídeo, las ventanas de visualización y deteniendo el proceso de *libcamera-vid* iniciado al principio. Este proceso proporciona un sistema eficiente para la captura y anotación de *keypoints* en la *Raspberry Pi*, resolviendo las limitaciones de acceso a la cámara y optimizando el rendimiento para su uso en tiempo real.

5.2.3 Actividades a reconocer

Para el entrenamiento y evaluación del modelo de detección de acciones desarrollado en este trabajo, se han seleccionado siete acciones representativas que se realizan habitualmente frente a un espejo de baño.

Esta selección responde tanto a criterios funcionales como a la viabilidad técnica de capturar

los movimientos asociados mediante visión por computador. Las acciones elegidas son las siguientes:

1. Cepillarse el pelo.
2. Lavarse los dientes.
3. Echarse colonia.
4. Echarse desodorante.
5. Lavarse la cara.
6. Lavarse las manos.
7. Tomarse las pastillas.

Estas acciones han sido seleccionadas por varias razones fundamentales. En primer lugar, todas ellas son actividades cotidianas vinculadas con el aseo personal y, por tanto, se realizan de forma natural y rutinaria en el entorno del baño, que es precisamente el lugar donde se plantea la instalación del sistema basado en espejo inteligente. Este entorno proporciona condiciones controladas de iluminación y posición corporal frente al espejo, lo que facilita la captura precisa de los movimientos mediante visión por computador.

En segundo lugar, se trata de acciones especialmente relevantes para el mantenimiento de la autonomía en personas mayores o con deterioro cognitivo, ya que forman parte de los hábitos de higiene diaria y autocuidado. La detección de estas actividades permite generar alertas o confirmar que el usuario ha seguido correctamente su rutina, aportando así un valor añadido desde el punto de vista asistencial.

Desde el punto de vista técnico, estas acciones presentan patrones de movimiento diferenciados y suficientemente representativos en términos de pose corporal, movimientos de las manos y gestos faciales, lo que favorece su detección mediante técnicas basadas en *keypoints* y modelos de aprendizaje automático. Por ejemplo, cepillarse el pelo implica un movimiento repetitivo del brazo por encima de la cabeza; lavarse los dientes se caracteriza por gestos rítmicos cerca de la boca; mientras que echarse desodorante o colonia suelen involucrar la elevación de uno de los brazos o movimientos hacia el cuello.

Finalmente, se ha buscado un equilibrio entre variedad gestual y coherencia contextual, seleccionando un conjunto de acciones que no solo tienen relevancia práctica, sino que también pueden ser diferenciadas de forma robusta a partir de los datos obtenidos mediante sensores visuales como los que ofrece MediaPipe. Esta combinación asegura que el sistema sea tanto útil como técnicamente viable dentro del entorno previsto.

5.2.3.1 Anotación de actividades

Para anotar los datos se desarrolló un programa cuyo objetivo principal, además de capturar y almacenar los puntos clave detectados por *MediaPipe*, es poder clasificar las acciones conforme se van realizando.

Cada *keypoint* que se detecta, se almacena en un archivo CSV previamente explicado. El

programa también permite etiquetar automáticamente cada muestra mediante la pulsación de teclas predefinidas, asignando así una acción específica a cada conjunto de *keypoints* capturados.

Durante la fase de captura de datos, es fundamental disponer de un sistema que permita al operador etiquetar en tiempo real la acción que está realizando la persona frente a la cámara. Para ello, se ha desarrollado un módulo auxiliar denominado *acciones_tecclas.py*, cuya función principal es establecer una correspondencia directa entre determinadas teclas del teclado y las distintas acciones que el sistema debe aprender a reconocer.

Este archivo contiene un diccionario en el que cada entrada asocia una tecla específica con una acción concreta, como se puede ver en el Código 5.7.

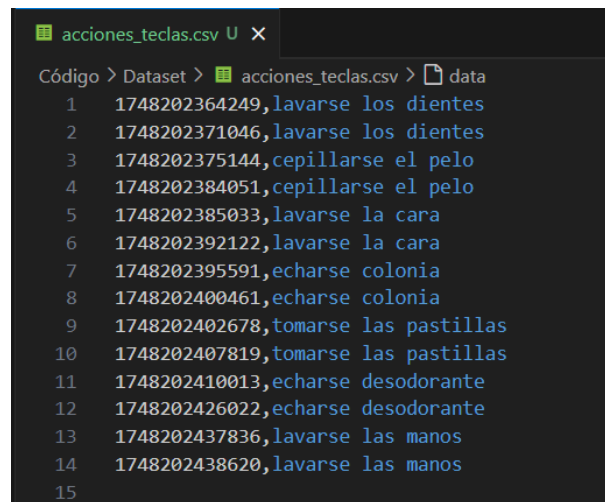
Código 5.7: Acciones recogidas.

```
1 acciones_tecclas = {  
2     '1': 'cepillarse_pelo',  
3     '2': 'lavarse_dientes',  
4     '3': 'echarse_colonia',  
5     '4': 'echarse_desodorante',  
6     '5': 'lavarse_cara',  
7     '6': 'lavarse_manos',  
8     '7': 'tomarse_pastillas'  
9 }
```

La lógica de funcionamiento es sencilla: cuando el sistema de captura de vídeo se encuentra en ejecución, permanece atento a la pulsación de teclas. Cuando el usuario pulsa una tecla registrada en el diccionario, el sistema interpreta dicha acción como la etiqueta correspondiente y anota en un CSV llamado *Acciones_tecclas* el de inicio junto con la acción correspondiente a la tecla pulsada.

Finalmente, cuando la acción ha terminado, el usuario volverá a pulsar la misma tecla asociada a la acción, registrándose su fin y anotándose en el mismo archivo que antes. Obteniendo el rango de tiempo que comprende una acción concreta, como se muestra en la Figura 5.6.

Esta solución resulta especialmente útil en contextos donde se requiere capturar múltiples secuencias de forma manual y rápida, permitiendo al operador alternar entre distintas acciones con un solo gesto.

The image shows a code editor window with a dark background. The title bar at the top reads 'acciones_teclas.csv U X'. Below the title bar, the breadcrumb navigation shows 'Código > Dataset > acciones_teclas.csv > data'. The main area contains a list of 15 lines of data, each consisting of a line number, a numeric ID, a comma, and a text description of an action. The text descriptions are color-coded: 'lavarse los dientes' is blue, 'cepillarse el pelo' is green, 'lavarse la cara' is blue, 'echarse colonia' is green, 'tomarse las pastillas' is blue, 'echarse desodorante' is green, and 'lavarse las manos' is blue.

```
1 1748202364249,lavarse los dientes
2 1748202371046,lavarse los dientes
3 1748202375144,cepillarse el pelo
4 1748202384051,cepillarse el pelo
5 1748202385033,lavarse la cara
6 1748202392122,lavarse la cara
7 1748202395591,echarse colonia
8 1748202400461,echarse colonia
9 1748202402678,tomarse las pastillas
10 1748202407819,tomarse las pastillas
11 1748202410013,echarse desodorante
12 1748202426022,echarse desodorante
13 1748202437836,lavarse las manos
14 1748202438620,lavarse las manos
15
```

Figura 5.6: Formato del archivo “acciones__ anotadas”.

5.2.4 Generación de imágenes

Además de todos los programas necesarios para la elaboración del dataset y entrenamiento del modelo, se crearon algunos scripts extras para comprobar que la generación de imágenes funcionara correctamente.

5.2.4.1 Carga de keypoints desde CSV

Para el correcto procesamiento y análisis de los *keypoints* obtenidos en el sistema, se requiere la carga de los datos previamente almacenados en archivos CSV.

El primer paso para su carga consiste en localizar los archivos almacenados. Para ello, se ha diseñado otro programa cuya ubicación será la misma que dichos CSV y podrá acceder a ellos sin importar desde qué entorno se ejecute. Esto garantiza portabilidad y facilita la organización de los datos dentro del directorio del proyecto.

Para obtener la información de estos archivos, usamos un proceso de lectura recorriendo las filas de los archivos en busca de la misma marca de tiempo. Como cada archivo sigue una estructura específica, es necesario contemplar las 3 opciones de archivos que podemos encontrar.

Antes que nada, el sistema debe verificar si el archivo está disponible y si hay datos en él, para evitar errores durante la ejecución. Si el proceso encuentra algún problema, el mismo parará e informará al usuario. Si, por el contrario, el archivo es accesible, se obtienen los puntos clave para almacenarlos en listas de datos.

Cuando ya se han obtenido los datos de cada archivo, es necesario implementar variables para su posterior análisis y representación. Como ya se ha mencionado previamente, cada archivo tiene

una estructura diferente y es necesario adaptar estas variables a dichos casos. Estas fluctuaciones se ven afectadas también por la cantidad de puntos que recopila cada parte en un instante de tiempo, ya que, por ejemplo, la pose recopila 6 puntos mientras que la cara 478. Por ello, es importante destacar que las estructuras de los *keypoints* faciales son almacenadas de diferente manera, usando listas para reducir lo máximo posible el tiempo de traspasar y leer los datos.

5.2.4.2 Visualización de keypoints

Tras la carga de datos, el siguiente paso en el procesamiento es la visualización de los *keypoints* obtenidos. Para ello, se genera una imagen base sobre la cual se representarán los puntos clave y sus conexiones.

Se inicia el proceso creando una imagen en negro que servirá como lienzo para la representación gráfica. Esta imagen tiene un tamaño fijo, y en caso de no encontrarse un fondo predefinido, se genera de forma dinámica una imagen de color blanco mediante OpenCV. Esto garantiza que la visualización pueda realizarse de manera autónoma sin depender de archivos externos.

A continuación, se verifica la disponibilidad de los *keypoints* cargados. Si existen datos para las manos, la pose o la cara, se procede a representarlos en la imagen. Para facilitar su interpretación visual, todos los puntos clave (*keypoints*) se dibujan en color azul, mientras que las conexiones entre ellos se representan en color rojo. Esta codificación cromática permite distinguir claramente la ubicación y la estructura de las distintas partes del cuerpo.

- **Manos:** Se representan tanto los puntos clave como las conexiones entre ellos, siguiendo la anatomía de la mano. Esto permite identificar gestos y posiciones relevantes para la detección de acciones.
- **Pose corporal:** Se dibujan los principales puntos de referencia de la parte superior del cuerpo (hombros, codos, muñecas, etc.) conectados entre sí para representar la postura del sujeto.
- **Cara:** Se muestran los numerosos puntos faciales que permiten esbozar los rasgos principales. Las conexiones más representativas ayudan a definir la estructura sin sobrecargar la imagen.

Para su representación, se convierten las coordenadas normalizadas, (que oscilan entre 0 y 1) a coordenadas de píxeles en la imagen. Con esto se consigue que, multiplicando las coordenadas por el tamaño de la imagen, los puntos se ubiquen correctamente dentro del fondo negro generado.

El sistema también utiliza las conexiones del cuerpo establecidas en la biblioteca *MediaPipe* para conectar los puntos clave de las manos y el cuerpo, haciendo la visualización más intuitiva y clara, ayudando al análisis de la postura y el movimiento por el ojo humano.

Esta función de visualización permite al usuario ver los datos obtenidos, tanto en tiempo real como una captura previa a partir de los archivos CSV.

Este programa permite la interpretación de forma gráfica de la información capturada, facilitando el análisis de la información en aplicaciones como el reconocimiento de gestos o el seguimiento de posturas para identificar actividades.

En la Figura 5.7 podemos ver un ejemplo de la acción *lavarse los dientes*, donde se han ido capturando diferentes posiciones del sujeto.

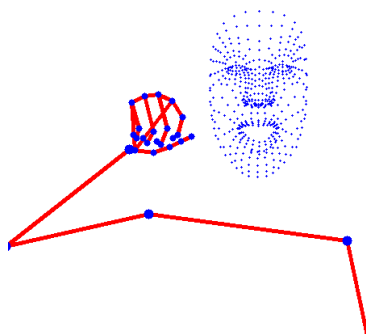


Figura 5.7: Acción *lavarse los dientes* con *keypoints*.

5.3 Modelo IA de reconocimiento de actividades

Una vez obtenidos y representados los puntos clave del cuerpo, manos y rostro mediante el sistema de detección, y tras almacenar correctamente todas las secuencias correspondientes a cada acción, se da paso a la fase de entrenamiento del modelo. Esta etapa tiene como objetivo enseñar a la IA a reconocer distintos patrones de movimiento y postura asociados a cada actividad registrada.

Cada clase está representada por múltiples secuencias formadas por 10 imágenes consecutivas, en las que se registran los *keypoints* del cuerpo, rostro y manos del usuario mientras ejecuta una acción determinada. Estas secuencias se organizan por usuario, lo que permite aplicar validación cruzada entre individuos. De este modo, se evalúa la capacidad del modelo para generalizar frente a personas no vistas durante el entrenamiento, simulando escenarios de uso real.

5.3.1 Taxonomía

Los datos utilizados en este proyecto están formados por secuencias de imágenes que contienen puntos clave del cuerpo, rostro y manos del usuario, capturados mientras realiza distintas acciones. Este tipo de información combina una dimensión espacial, presente en cada imagen individual, y una dimensión temporal, relacionada con la evolución del movimiento a lo largo del tiempo.

Por tanto, el problema requiere arquitecturas de Aprendizaje profundo, por sus siglas en inglés *Deep Learning* (DL) que sean capaces de procesar imágenes en secuencia, extrayendo patrones espaciales en cada fotograma y modelando su continuidad temporal. Este enfoque permite identificar no solo posturas aisladas, sino también transiciones entre ellas, lo que resulta esencial para distinguir acciones de forma precisa.

En esta sección se presentan las arquitecturas más adecuadas para abordar este tipo de problema, sirviendo como base para la selección del modelo implementado en el sistema.

5.3.1.1 Redes neuronales totalmente conectadas

Las redes neuronales conectadas, también conocidas como redes densas o *fully connected* son la forma más sencilla de red neuronal artificial. Cada neurona en una capa está conectada con todas las neuronas de la capa anterior, formando una red completamente entrelazada. Esto permite modelar relaciones complejas entre las variables de entrada y salida, lo que es ideal para tareas de clasificación sobre vectores de características.

Estas redes operan mediante una combinación de funciones lineales (productos matriciales entre pesos y entradas) y funciones de activación no lineales como Unidad lineal rectificada, por sus siglas en inglés *Rectified Linear Unit* (ReLU), Función sigmoide, por sus siglas en inglés *Sigmoid Function* (Sigmoid) o Tangente hiperbólica, por sus siglas en inglés *Hyperbolic Tangent* (Tanh). Su simplicidad las hace adecuadas para datos sin estructura espacial o secuencial, como vectores de atributos [45].

En la Figura 5.8 se muestra de forma gráfica el funcionamiento de estas redes neuronales.

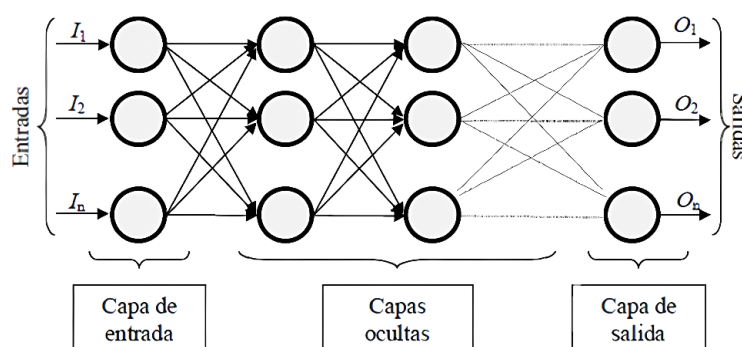


Figura 5.8: Redes neuronales totalmente conectadas.

Sin embargo, en contextos como la visión por computador o el análisis de secuencias temporales, su aplicación directa es limitada. Esto se debe a que no explotan la información local, como relaciones espaciales entre píxeles, ni tienen mecanismos internos para conservar información en el tiempo. Por tanto, aunque pueden usarse como parte final de una arquitectura más compleja, por sí solas no resultan adecuadas para el problema tratado en este proyecto.

5.3.1.2 Redes neuronales convolucionales

Las redes neuronales convolucionales, CNN, están diseñadas para el procesamiento eficiente de datos estructurados en forma de rejilla, como imágenes. Se caracterizan por el uso de filtros o kernels que se deslizan sobre la entrada, realizando operaciones de convolución, lo que permite detectar patrones locales como bordes, texturas o formas [46].

Cada capa convolucional genera mapas de activación que representan la presencia de características específicas en distintas ubicaciones. Al apilar varias capas convolucionales, la red aprende representaciones jerárquicas de la entrada, desde detalles finos hasta estructuras complejas. Las CNN suelen incluir capas de pooling para reducir la dimensionalidad y capas de normalización para estabilizar el entrenamiento.

En la Figura 5.9 se muestra de forma gráfica el funcionamiento de las redes neuronales convolucionales.

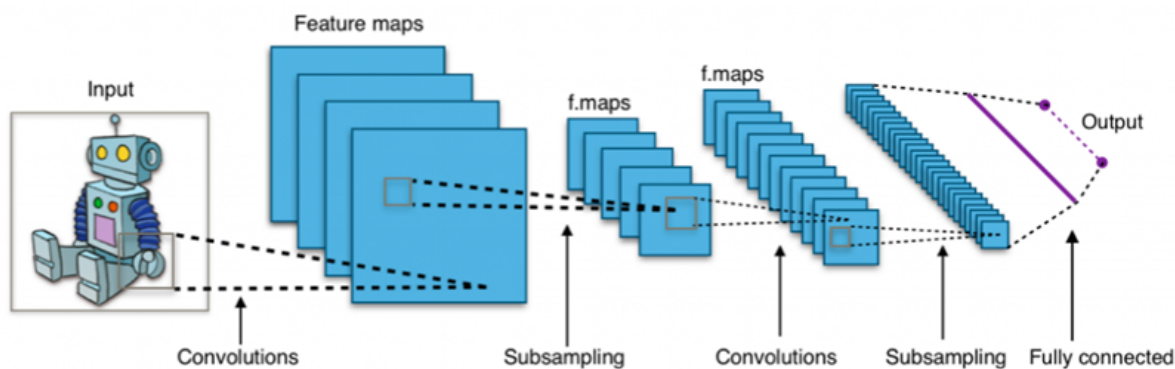


Figura 5.9: Redes neuronales convolucionales.

Estas redes son especialmente potentes para tareas de clasificación de imágenes, detección de objetos y reconocimiento de patrones espaciales. En muchos casos, se combinan con redes densas en las capas finales para realizar la predicción. En nuestro contexto, una CNN es una elección adecuada para procesar imágenes generadas a partir de *keypoints* normalizados [47].

5.3.1.3 Redes neuronales recurrentes

Las redes neuronales recurrentes, Red neuronal recurrente, por sus siglas en inglés *Recurrent Neural Network* (RNN), están orientadas al tratamiento de secuencias de datos, donde el orden y la dependencia temporal son relevantes. La principal característica de las RNN es la presencia de conexiones cíclicas que permiten a la red conservar un estado interno o memoria de pasos anteriores.

Esto las hace especialmente útiles en tareas como el análisis de texto, series temporales,

reconocimiento de voz y acción o vídeo. Sin embargo, las RNN tradicionales presentan dificultades para aprender dependencias a largo plazo debido al problema del desvanecimiento o explosión del gradiente durante el entrenamiento.

Para solucionar este problema, surgieron variantes como las redes LSTM y GRU, que incorporan mecanismos de compuertas para controlar el flujo de información relevante durante secuencias largas y olvidar aquellas que ya no son necesarias [48].

En el contexto del reconocimiento de acciones, las redes recurrentes son esenciales para captar la dinámica del movimiento: no basta con ver una postura, sino entender cómo esta evoluciona en el tiempo para diferenciar, por ejemplo, la acción de lavarse la cara con echarse colonia.

5.3.1.3.1 LSTM (Long Short-Term Memory)

Las redes LSTM son una arquitectura avanzada dentro del campo de las redes neuronales recurrentes (RNN), introducida por Hochreiter y Schmidhuber en 1997 [49]. Estas redes fueron diseñadas específicamente para superar una de las limitaciones fundamentales de las RNN clásicas: su incapacidad para aprender dependencias a largo plazo dentro de las secuencias de datos. En las redes recurrentes convencionales, la información de pasos anteriores se va debilitando progresivamente a medida que la secuencia avanza, debido a problemas como el desvanecimiento del gradiente [50]. Las LSTM resuelven este problema gracias a un mecanismo de memoria interna que permite retener información relevante durante períodos largos.

Esta arquitectura introduce un sistema de compuertas que actúan como filtros: una compuerta decide qué información debe mantenerse, otra controla qué datos deben olvidarse, y una tercera regula qué partes de la información serán utilizadas para generar la salida en cada instante. Gracias a esta lógica de control, las LSTM son capaces de aprender patrones temporales complejos y mantener el contexto incluso en secuencias largas. Esto las convierte en una herramienta eficaz para tareas como la traducción automática, la transcripción de voz, el modelado del lenguaje o el reconocimiento de acciones humanas en vídeo.

En la Figura 5.10 se muestra de forma gráfica el funcionamiento de las redes neuronales recurrentes LSTM.

5.3.1.3.2 GRU (Gated Recurrent Unit)

Las GRU constituyen una alternativa simplificada a las LSTM. Aunque persiguen el mismo objetivo —gestionar de forma eficiente la información relevante a lo largo del tiempo— lo hacen mediante una arquitectura más compacta. En lugar de tener múltiples compuertas, las GRU utilizan solo dos: una compuerta de actualización y otra de reinicio. Estas dos compuertas controlan de forma conjunta qué parte de la información pasada debe conservarse y cuál debe descartarse, y cómo combinarla con la información nueva.

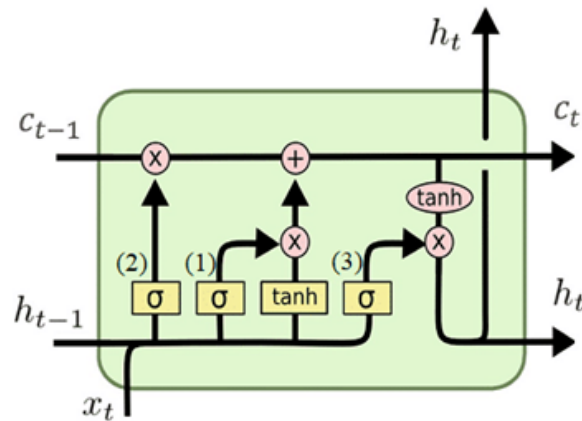


Figura 5.10: Redes neuronales recurrentes LSTM.

El menor número de parámetros que caracteriza a las GRU tiene varias implicaciones prácticas: en general, requieren menos tiempo de entrenamiento, consumen menos memoria y pueden ser más adecuadas para sistemas con recursos computacionales limitados. A pesar de su simplicidad, numerosos estudios empíricos han demostrado que las GRU pueden alcanzar un rendimiento comparable al de las LSTM en muchas aplicaciones reales, incluyendo procesamiento de lenguaje natural, reconocimiento de gestos y series temporales [51].

En la Figura 5.11 se muestra de forma gráfica el funcionamiento de las redes neuronales recurrentes GRU.

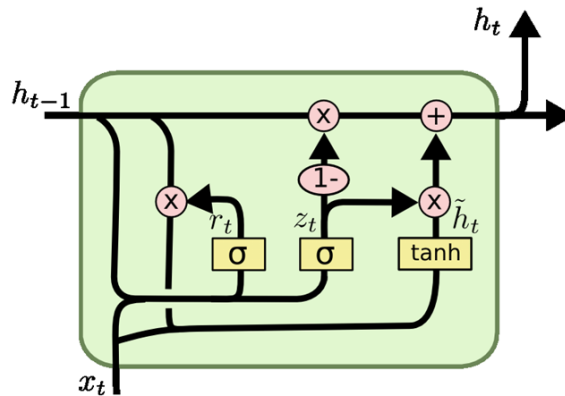


Figura 5.11: Redes neuronales recurrentes GRU.

5.3.1.3.3 Comparación conceptual

Tanto las LSTM como las GRU son arquitecturas diseñadas para resolver los mismos desafíos fundamentales que presentan las RNN tradicionales: la dificultad de capturar dependencias a largo plazo, la pérdida de información importante con el paso del tiempo y la inestabilidad

durante el entrenamiento. Ambas soluciones introducen mecanismos de memoria y control que permiten a la red “recordar” mejor información pasada y adaptarse dinámicamente a cada secuencia.

Desde un punto de vista práctico, la elección entre LSTM y GRU no tiene una respuesta universal. En problemas con secuencias muy largas o dependencias especialmente complejas, las LSTM pueden proporcionar un mayor control y rendimiento. En cambio, las GRU son más ligeras y rápidas, lo que puede ser ventajoso en escenarios donde el tiempo de procesamiento o la capacidad de cómputo son limitados. En general, la diferencia de rendimiento entre ambas suele ser mínima, y la selección depende del tipo de datos, el tamaño del conjunto de entrenamiento y los recursos disponibles [52].

5.3.2 Combinación de arquitecturas en el modelo propuesto

El modelo propuesto en este entrenamiento combina dos de las arquitecturas expuestas: redes convolucionales (CNN) y redes recurrentes, en una configuración híbrida que permite aprovechar tanto la información espacial como la temporal contenida en las secuencias de imágenes capturadas.

Inicialmente se planteó utilizar capas LSTM como componente recurrente del modelo, dada su capacidad para modelar dependencias temporales a largo plazo. No obstante, durante el proceso de ajuste se observó que las LSTM implicaban un mayor consumo de memoria y unos tiempos de entrenamiento más prolongados. Dado que las secuencias utilizadas en este trabajo son cortas (10 imágenes por acción) y que se trabaja en un entorno con recursos limitados, se optó finalmente por sustituir las LSTM por GRU. Esta alternativa más ligera ha demostrado ofrecer un rendimiento comparable con una arquitectura más sencilla, lo que resulta más adecuado en el contexto del proyecto [51].

Primero, cada imagen que representa un instante de la acción (formada por los *keypoints* proyectados sobre fondo blanco) es procesada por una red convolucional. Esta parte del modelo extrae patrones espaciales característicos de la postura del cuerpo y la posición de las manos y la cara. Las capas *Conv2D*, *MaxPooling2D* y *BatchNormalization* se encargan de capturar esta información relevante y de reducir la dimensionalidad de los datos (Código 5.8).

Código 5.8: Extracción de características espaciales.

```
1 cnn = TimeDistributed(Conv2D(4, (3,3), activation='relu'))(sequence_input)
2 cnn = TimeDistributed(MaxPooling2D((2,2)))(cnn)
3 cnn = TimeDistributed(BatchNormalization())(cnn)
4 cnn = TimeDistributed(Dropout(0.15))(cnn)
5
6 cnn = TimeDistributed(Conv2D(8, (3,3), activation='relu'))(cnn)
7 cnn = TimeDistributed(MaxPooling2D((2,2)))(cnn)
8 cnn = TimeDistributed(BatchNormalization())(cnn)
9 cnn = TimeDistributed(Dropout(0.15))(cnn)
10
11 cnn = TimeDistributed(Conv2D(8, (3,3), activation='relu'))(cnn)
```

```

12 cnn = TimeDistributed(MaxPooling2D((4,4)))(cnn)
13 cnn = TimeDistributed(BatchNormalization())(cnn)
14 cnn = TimeDistributed(Dropout(0.15))(cnn)
15
16 cnn = TimeDistributed(Conv2D(16, (5,5), activation='relu'))(cnn)
17 cnn = TimeDistributed(BatchNormalization())(cnn)
18 cnn = TimeDistributed(Dropout(0.15))(cnn)
19
20 cnn = TimeDistributed(Conv2D(16, (5,5), activation='relu'))(cnn)
21 cnn = TimeDistributed(MaxPooling2D((2,2)))(cnn)
22 cnn = TimeDistributed(BatchNormalization())(cnn)
23 cnn = TimeDistributed(Dropout(0.15))(cnn)

```

A continuación, se utiliza una capa *TimeDistributed(GlobalAveragePooling2D())* para reducir cada mapa de características a un vector independiente por instante (Código 5.9).

Código 5.9: Reducción de los mapas de activación a vectores.

```

1 cnn = TimeDistributed(GlobalAveragePooling2D())(cnn)

```

Estos vectores se concatenan en una secuencia temporal que es procesada por una red GRU. Esta unidad recurrente permite modelar la evolución de los movimientos y posturas a lo largo del tiempo, captando la dinámica de la acción y diferenciando entre aquellas que son visualmente similares pero se desarrollan de forma distinta a lo largo de la secuencia (Código 5.10).

Código 5.10: Procesamiento secuencial mediante una capa GRU.

```

1 gru = GRU(64)(cnn)

```

Finalmente, la salida de la GRU se conecta con una o varias capas densas (fully connected) que transforman la representación temporal en una predicción final: una probabilidad para cada clase de acción, como se refleja en el Código 5.11.

Código 5.11: Capa de salida para clasificación de acciones.

```

1 output = Dense(num_classes, activation='softmax')(gru)
2 model = Model(inputs=input_layer, outputs=output)

```

En la Figura 5.12 se muestra un diagrama esquemático del modelo CNN combinado con GRU.

Esta combinación se ha demostrado muy efectiva en tareas de clasificación de acciones humanas en vídeo y secuencias de imágenes, ya que conjuga la capacidad de detectar patrones espaciales complejos (CNN) con la habilidad de modelar relaciones temporales (GRU), todo ello con un coste computacional contenido.

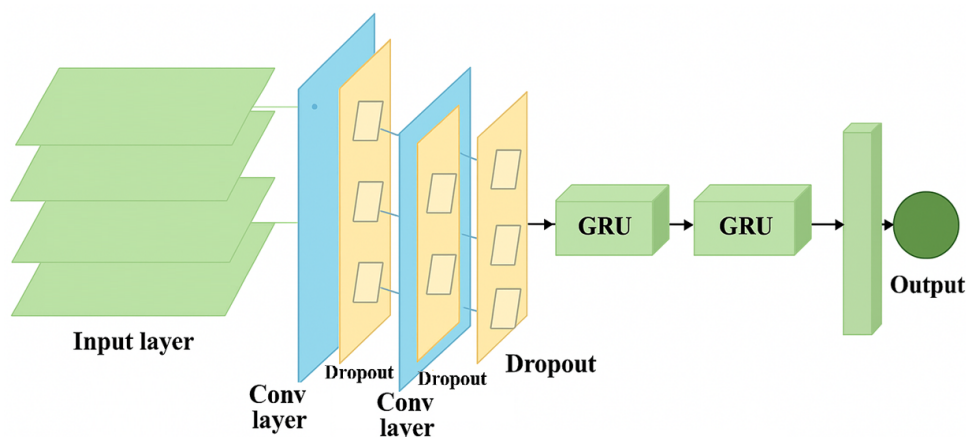


Figura 5.12: Modelo CNN y GRU.

5.3.3 Pre-procesamiento

En este trabajo se aplicaron una serie de transformaciones a los datos para garantizar que la información presentada al modelo sea consistente, estandarizada y relevante. Esto incluyó normalización espacial, organización por actividades, división en conjuntos de entrenamiento y prueba, y estrategias para aumentar la diversidad del dataset.

5.3.3.1 Normalización

Uno de los principales retos en la detección de acciones humanas es la variabilidad entre sujetos: altura, posición frente a la cámara, orientación corporal y distancia pueden afectar a la representación. Para reducir esta variabilidad, se desarrolló un sistema de normalización basado en el centrado de los puntos clave respecto a una referencia corporal (hombros y caderas), seguido de un escalado proporcional para ajustar la figura a un tamaño fijo.

A continuación, en la Figura 5.13 y Figura 5.14 se muestra una comparación entre un conjunto de *keypoints* originales y su versión normalizada, centrada y escalada. Dentro de cada Figura, podemos ver una clara diferencia de tamaño entre ambas imágenes. Esto ocurre porque están representadas a tamaño real: la 1, después de normalizar, cambia sus dimensiones y se hace más pequeña y la 2, es el tamaño con el que la cámara la captura.

Este proceso permite estandarizar todas las muestras independientemente del usuario y su posición relativa. El Código 5.17 ilustra el centrado y escalado aplicado.

Código 5.12: Centrado y escalado de puntos clave.

```

1centro = (min_xy + max_xy) / 2.0
2puntos_pose = np.array(pose_ordered) - centro
3size_box = max_xy - min_xy
4escala = POSE_BOX_SIZE / max(size_box)

```

```
px_pose = proyectar_a_imagen(puntos_pose.tolist(), escala)
```

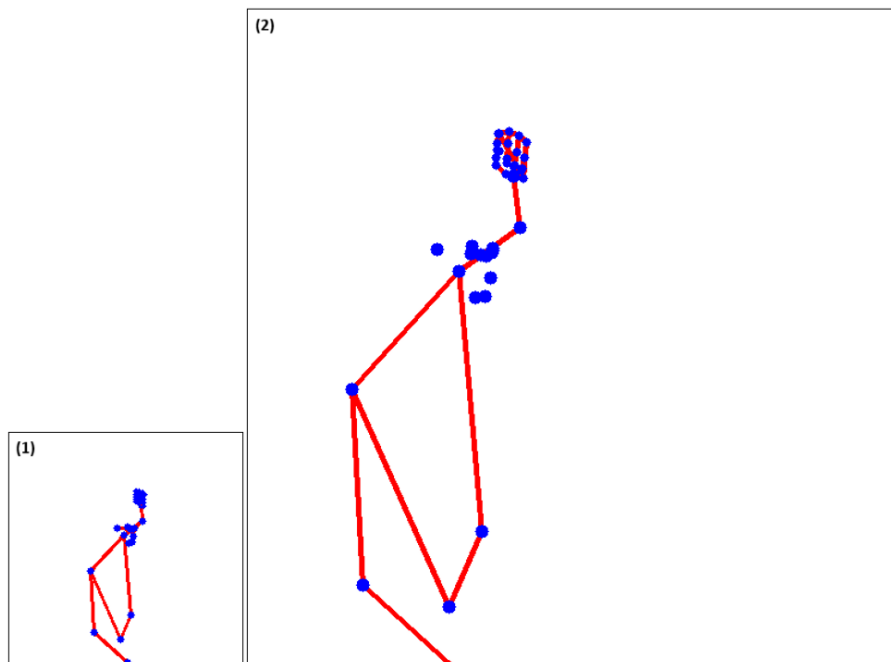


Figura 5.13: Cepillarse el pelo. Imagen normalizada (1) vs imagen no normalizada (2).

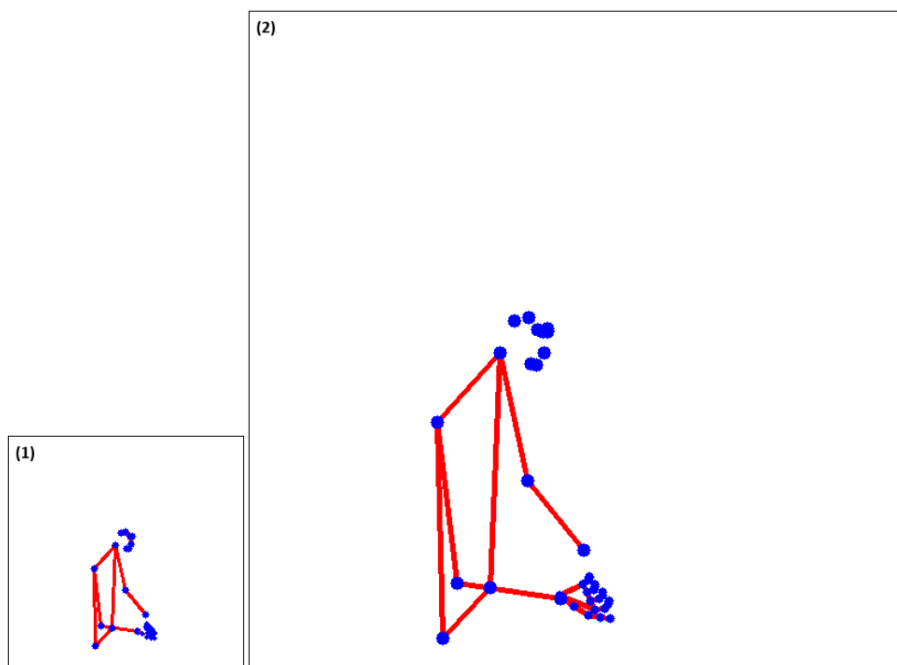


Figura 5.14: Lavarse las manos. Imagen normalizada (1) vs imagen no normalizada (2).

5.3.3.2 División del dataset por actividades

Una vez generadas las imágenes normalizadas, se organizó el dataset por tipo de acción. Esta información se extrajo del archivo *anotar_acciones.csv*, donde se recogen los intervalos de tiempo correspondientes a cada actividad anotada durante la grabación.

Las imágenes se agruparon en secuencias de 10 y se evaluó la acción dominante en cada grupo. Si al menos cinco imágenes correspondían a una misma acción, la secuencia completa se clasificaba bajo esa etiqueta. Esta estrategia permitió tolerar ligeras inconsistencias sin comprometer la coherencia del dataset.

En la Figura 5.15 se puede observar la estructura organizativa de las carpetas tras este paso.

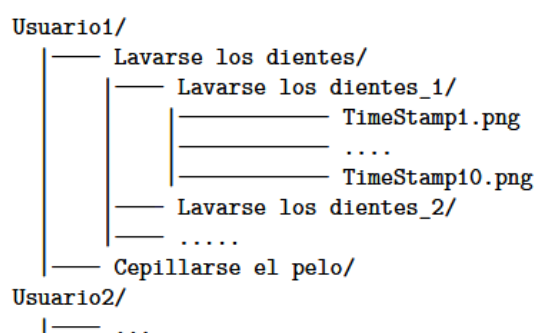


Figura 5.15: División del dataset en carpetas según actividades.

5.3.3.3 División del dataset en train y test

Para asegurar una evaluación justa del modelo, se dividió el dataset en dos subconjuntos: entrenamiento, del que se selecciona el 80% de las imágenes, y pruebas, del que se aparta el 20%.

Esta separación se hizo respetando la distribución de clases, evitando que las secuencias del subconjunto de prueba estuvieran presentes durante la fase del entrenamiento, para poder comprobar realmente la validez y capacidad de predicción y generalización del futuro modelo entrenado.

Partiendo de la estructura organizativa previa, el resultado tras esta nueva división se puede apreciar en la Figura 5.16.

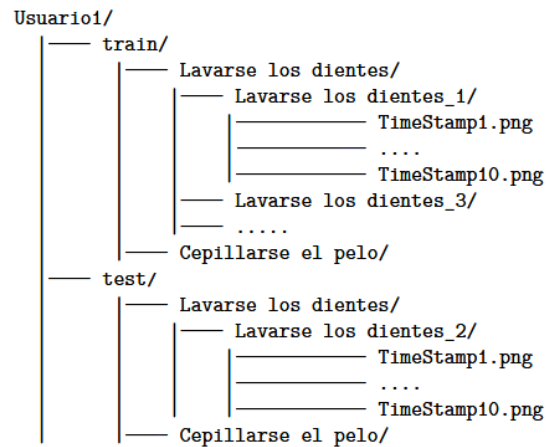


Figura 5.16: División del dataset se carpetas según train y test.

A continuación, en la Figura 5.17, Figura 5.18, Figura 5.19, Figura 5.20, Figura 5.21, Figura 5.22 y Figura 5.23, se muestra un ejemplo de 10 imágenes para cada tipo de acción.

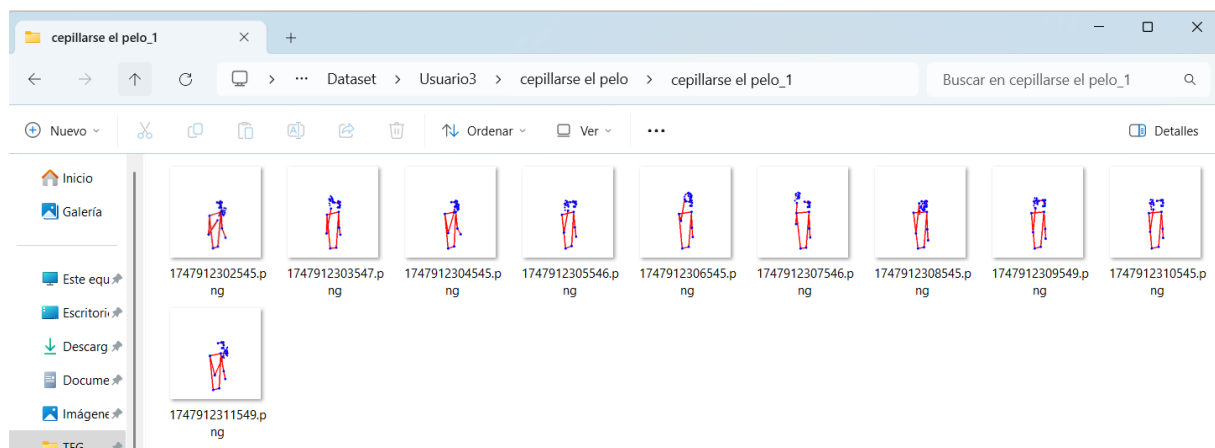


Figura 5.17: Dataset, ejemplo de la acción *Cepillarse el pelo*.

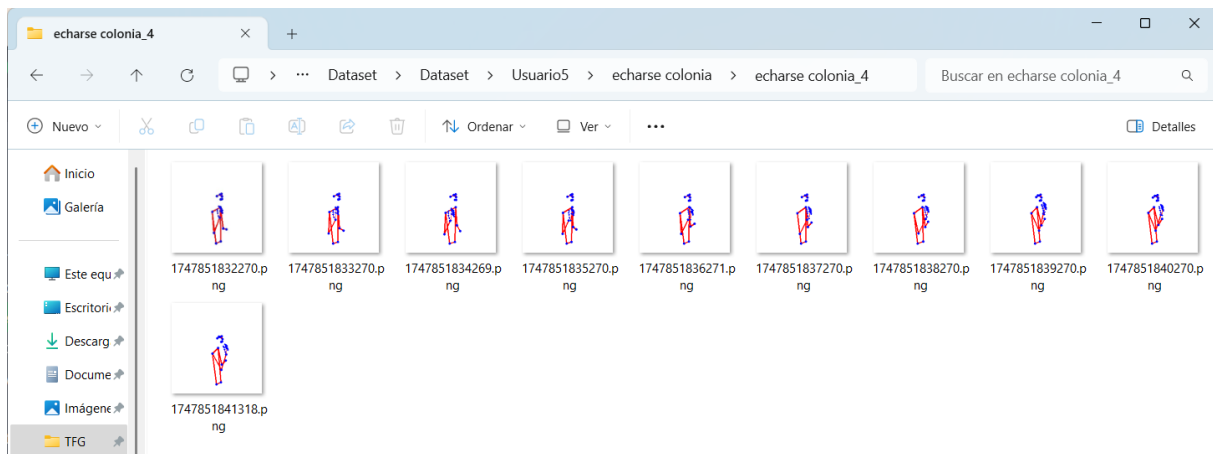


Figura 5.18: Dataset, ejemplo de la acción *Echarse colonia*.

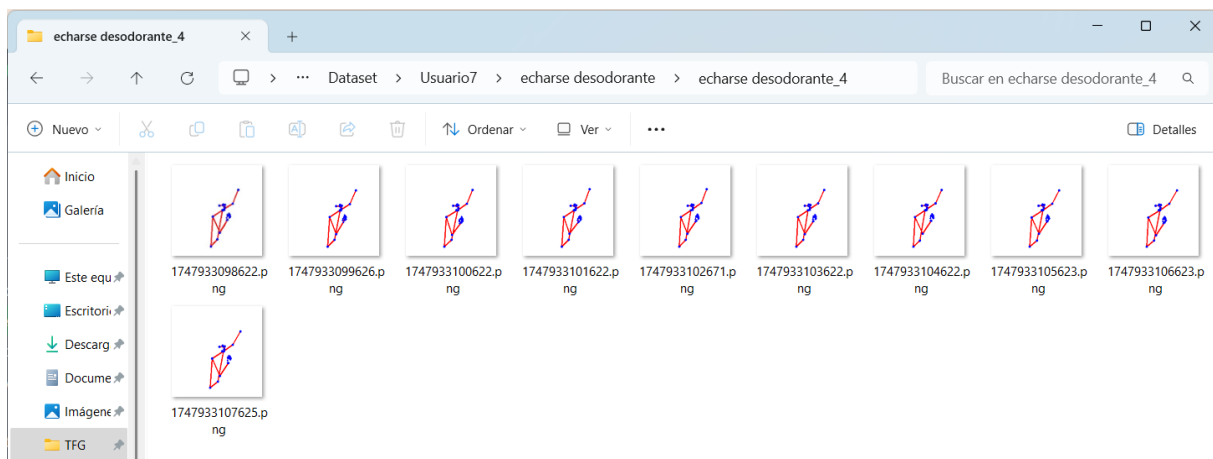


Figura 5.19: Dataset, ejemplo de la acción *Echarse desodorante*.

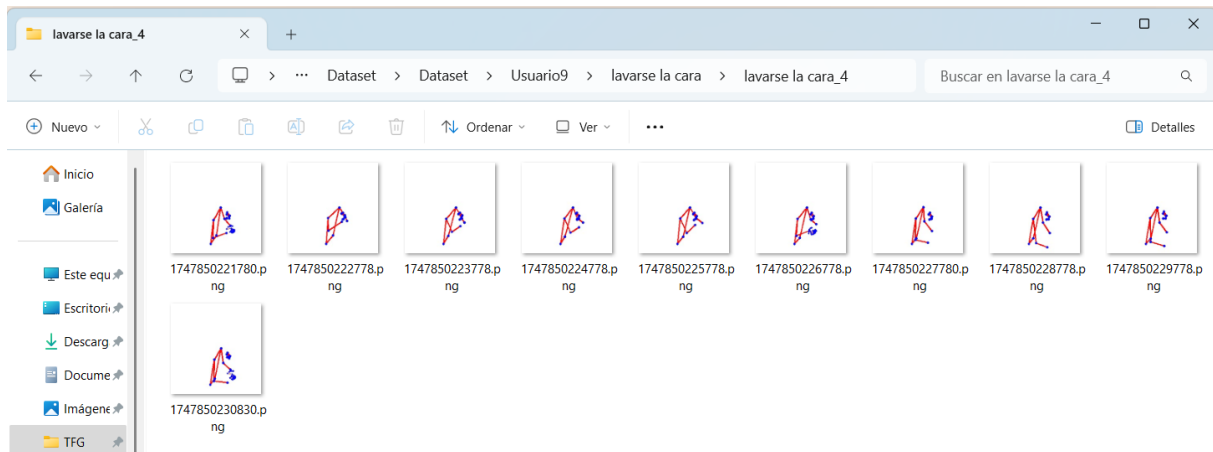


Figura 5.20: Dataset, ejemplo de la acción *Lavarse la cara*.

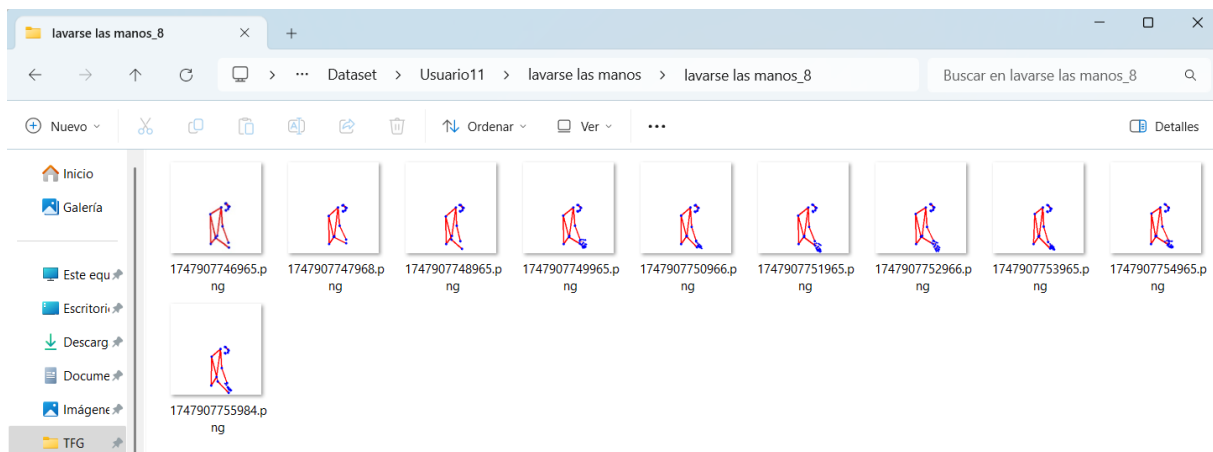


Figura 5.21: Dataset, ejemplo de la acción *lavarse las manos*.

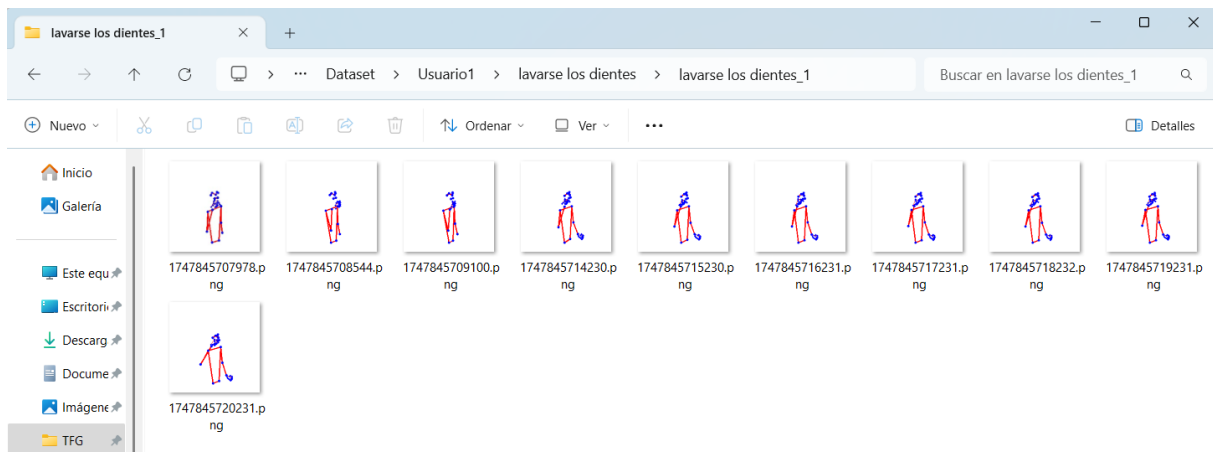


Figura 5.22: Dataset, ejemplo de la acción *lavarse los dientes*.

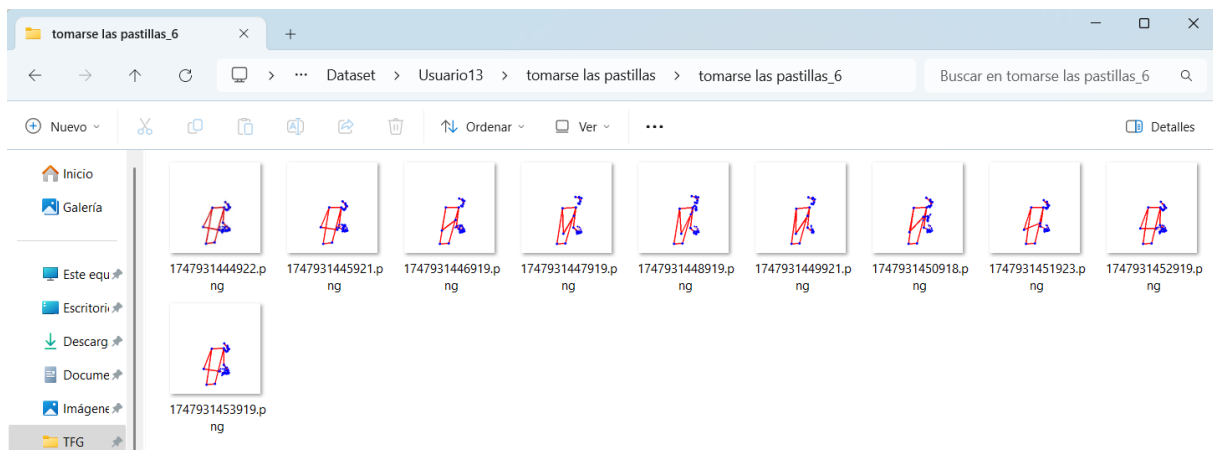


Figura 5.23: Dataset, ejemplo de la acción *tomarse las pastillas*.

5.3.4 Entrenamiento

El entrenamiento del modelo de IA ha sido la etapa más crucial y técnicamente exigente del proyecto. En esta fase se articulan de forma integrada la preparación de los datos, la arquitectura del modelo, la configuración del aprendizaje y la evaluación continua mediante gráficas e indicadores clave. A continuación, se describen con detalle cada una de las partes que conforman este complejo sistema de aprendizaje profundo.

5.3.4.1 Evolución del entrenamiento

Inicialmente, el entrenamiento del modelo se llevó a cabo en un entorno local con una política de *early stopping* muy restrictiva, que interrumpía el aprendizaje si la pérdida de validación no mejoraba en 5 épocas. Este enfoque provocó que el modelo se detuviera antes de alcanzar un nivel óptimo de generalización, obteniendo una precisión de tan solo 33.93%.

Ante este resultado, se migró el proceso a entornos más potentes: primero Google Colab y posteriormente un entorno Jupyter Notebook con acceso a GPU A30. En estos entornos se amplió la paciencia del *early stopping* a 50 épocas, lo que permitió realizar un entrenamiento más profundo y estable. Este cambio fue fundamental para que el modelo lograra una precisión del 95% en validación.

5.3.4.2 Carga y preparación de los datos

El modelo no trabaja con imágenes individuales, sino con secuencias de 10 imágenes por acción. Esta representación permite capturar la dinámica temporal de las actividades, lo cual es esencial para distinguir acciones similares, como *lavarse los dientes* frente a *lavarse la cara*.

Las secuencias se organizan por clase y usuario, y se normalizan al rango $[0, 1]$, redimensionándose a 224x224 píxeles. El tensor de entrada tiene forma (10, 224, 224, 3). El Código 5.13 muestra la función que realiza este preprocesamiento.

Código 5.13: Carga y normalización de una secuencia.

```
1 def load_sequence(sequence_path, img_shape):
2     num_frames, height, width, channels = img_shape
3     sequence_frames = np.zeros(shape=img_shape, dtype=np.float32)
4     files = sorted(os.listdir(sequence_path))[:num_frames]
5     for i, file_name in enumerate(files):
6         img_path = os.path.join(sequence_path, file_name)
7         if os.path.exists(img_path):
8             img = cv2.imread(img_path)
9             if img is None:
10                 continue
11             img = cv2.resize(img, (width, height))
12             img = img.astype(np.float32) / 255.0
13             sequence_frames[i] = img
14     return sequence_frames
```

5.3.4.3 Evolución de la arquitectura: de LSTM a GRU

Durante las primeras versiones del modelo se empleó una capa LSTM tras las convoluciones para modelar la secuencia temporal de cada acción. Sin embargo, en el momento de exportar el modelo a TensorFlow Lite para su despliegue en la Raspberry Pi, surgieron problemas de

compatibilidad debido al uso de LSTM.

Para solucionarlo, se sustituyó esta capa por una GRU, más ligera y totalmente compatible con dispositivos embebidos. Esta modificación no sólo resolvió los problemas de exportación, sino que también mantuvo un rendimiento competitivo en la tarea de clasificación.

5.3.4.4 Definición del modelo

El modelo final combina bloques convolucionales aplicados de forma independiente a cada imagen mediante *TimeDistributed*, con una capa GRU que capta el contexto temporal. A continuación, se añaden capas densas para la clasificación. La arquitectura completa se muestra en el Código 5.14.

Código 5.14: Arquitectura CNN-GRU final.

```
1 sequence_input = Input(shape=(10,224,224,3))
2 cnn = TimeDistributed(Conv2D(4, (3,3), activation='relu'))(sequence_input)
3 cnn = TimeDistributed(MaxPooling2D((2,2)))(cnn)
4 cnn = TimeDistributed(BatchNormalization())(cnn)
5 cnn = TimeDistributed(Dropout(0.15))(cnn)
6 ...
7 cnn = TimeDistributed(GlobalAveragePooling2D())(cnn)
8 gru = GRU(10, unroll=True)(cnn)
9 gru = Dropout(0.15)(gru)
10 dense = Dense(10, activation='relu')(gru)
11 dense = Dropout(0.15)(dense)
12 output = Dense(num_classes, activation='softmax')(dense)
13 model = Model(inputs=sequence_input, outputs=output)
```

5.3.4.5 Compilación y configuración del entrenamiento

El modelo se compiló con el optimizador Adam y una tasa de aprendizaje de 0.0003, utilizando la función de pérdida *categorical_crossentropy*. Para asegurar la convergencia y prevenir el sobreajuste, se configuraron dos *callbacks*:

- **EarlyStopping**: monitoriza la pérdida de validación y detiene el entrenamiento si no mejora en 50 épocas.
- **ModelCheckpoint**: guarda automáticamente el modelo con mejor rendimiento en validación.

5.3.4.6 Entrenamiento

Los datos de entrenamiento se dividieron en 80% para entrenamiento y 20% para validación. Se utilizó un tamaño de batch de 4 secuencias y un máximo de 1000 épocas, con parada anticipada en caso de estancamiento. El Código 5.15 muestra la instrucción de entrenamiento.

Código 5.15: Entrenamiento del modelo.

```
1 history = model.fit(  
2     train_ds,  
3     validation_data=val_ds,  
4     epochs=1000,  
5     callbacks=[early_stop, checkpoint]  
6 )
```

5.3.5 Validación

El proceso de validación permite evaluar la capacidad del modelo para generalizar sobre datos no vistos previamente. A través de diferentes estrategias, se analiza el comportamiento del sistema, se identifican posibles problemas de sobreajuste y se verifica la utilidad real del clasificador de acciones.

5.3.5.1 Métricas

Antes de iniciar el entrenamiento del modelo, el conjunto completo de datos fue dividido en dos subconjuntos: un 80% para entrenamiento y un 20% para prueba (test). Posteriormente, del conjunto de entrenamiento se extrajo un 20% para utilizarlo como conjunto de validación. En la Figura 5.24 se muestra esta disposición mencionada.

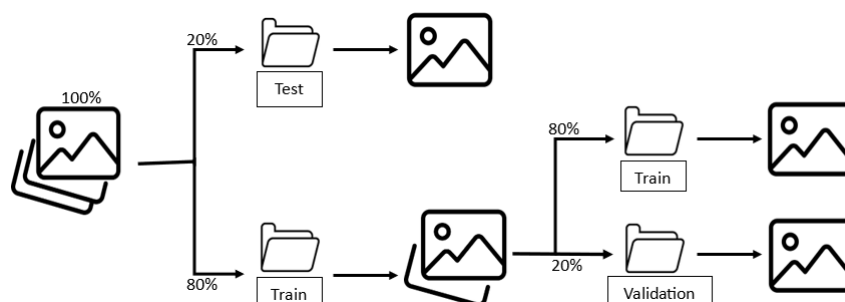


Figura 5.24: División entre train, validation y test.

Las métricas utilizadas para evaluar el rendimiento del modelo fueron la precisión (accuracy) y la función de pérdida categórica (categorical crossentropy). Estas métricas se calcularon durante cada época tanto sobre el conjunto de entrenamiento como sobre el conjunto de validación, y fueron registradas para su posterior análisis visual mediante gráficas.

La precisión mide el porcentaje de secuencias correctamente clasificadas respecto al total, mientras que la pérdida refleja el error del modelo en función de su desviación respecto a la salida esperada. El modelo alcanzó una precisión final del 95% en el conjunto de validación, lo que indica una alta fiabilidad.

A continuación, en la Figura 5.25 y Figura 5.26, se incluyen las gráficas generadas durante el entrenamiento, que ilustran la evolución de la precisión y la pérdida.

Estas gráficas permiten detectar posibles fases de sobreajuste, épocas óptimas de aprendizaje y analizar el comportamiento general del modelo.

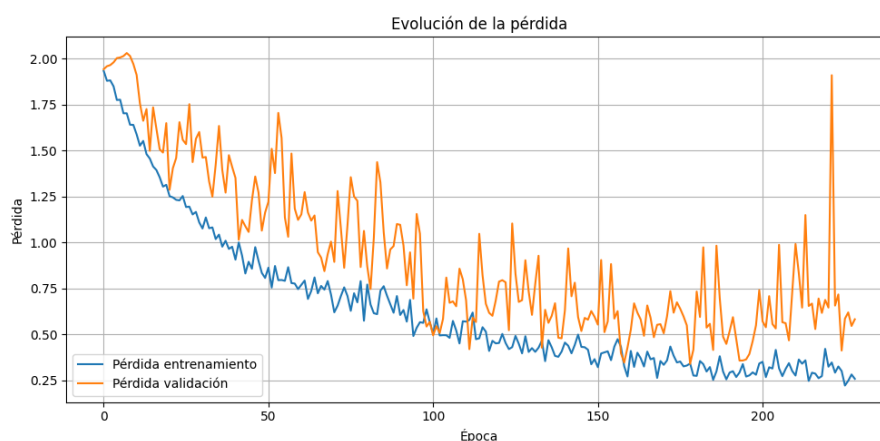


Figura 5.25: Pérdida durante el entrenamiento.

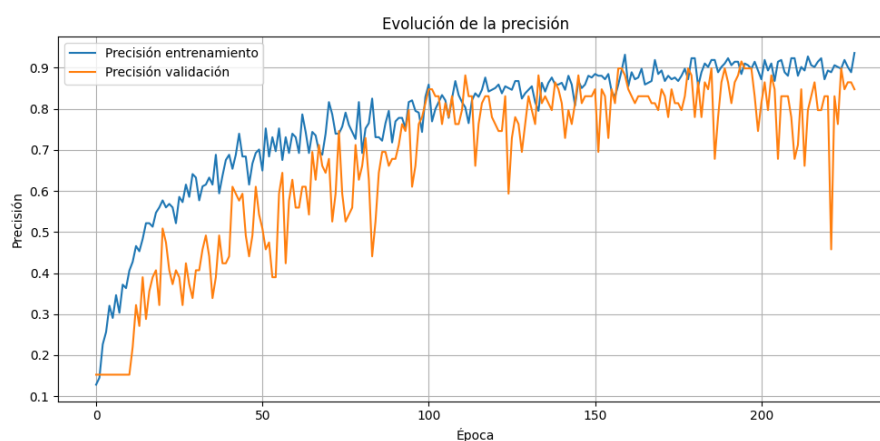


Figura 5.26: Precisión durante el entrenamiento.

5.4 Aplicación Web

Esta sección describe el funcionamiento y la estructura de la aplicación web desarrollada, cuyo objetivo es la detección automática de acciones humanas frente a un espejo inteligente. La aplicación integra captura de vídeo, procesamiento en tiempo real mediante un modelo de IA y una interfaz web sencilla para la visualización y consulta del historial.

5.4.1 Espejo inteligente

El espejo inteligente constituye el núcleo del sistema de asistencia desarrollado en este proyecto. Integra diversos componentes electrónicos y de hardware que permiten la captura, procesamiento y visualización de acciones humanas frente al espejo. En la Figura 5.27 se muestra el espejo.



Figura 5.27: Espejo.

5.4.1.1 Componentes

A continuación, se describen los componentes esenciales que lo conforman, junto con sus principales especificaciones técnicas.

5.4.1.1.1 Raspberry Pi 4 Model B

La Raspberry Pi 4 Model B (Figura 5.28) es una placa de desarrollo de bajo coste y alto rendimiento que actúa como el cerebro del sistema. Está equipada con un procesador de cuatro núcleos, puertos USB, HDMI, Ethernet y pines GPIO, lo que le permite realizar tareas de procesamiento de datos, inferencia de modelos de IA y control de periféricos como cámaras o sensores. Gracias a su versatilidad, tamaño reducido y eficiencia energética, esta placa ha sido

seleccionada para gestionar todo el sistema de visión por computador y la interfaz web del espejo inteligente. En la Tabla 5.1 se muestran sus especificaciones técnicas.

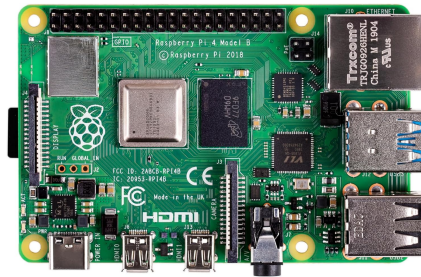


Figura 5.28: Raspberry Pi 4 Model B.

Característica	Valor
Procesador	Broadcom BCM2711, Quad-core Cortex-A72 (ARM v8) a 1.5GHz
Memoria RAM	4 GB LPDDR4
Conectividad	2 puertos micro-HDMI, 4 puertos USB (2x 3.0 y 2x 2.0), Gigabit Ethernet
Almacenamiento	Tarjeta microSD
GPIO	40 pines GPIO
Alimentación	5V/3A vía USB-C

Tabla 5.1: Especificaciones técnicas de la Raspberry Pi 4 Model B.

5.4.1.1.2 Cámara Módulo 3 NoIR para Raspberry Pi

La cámara Módulo 3 NoIR (Figura 5.29) permite capturar imágenes del usuario frente al espejo. A diferencia de otras versiones, carece de filtro infrarrojo, lo que le permite funcionar en condiciones de baja iluminación. Con sus 12 megapíxeles y enfoque automático, garantiza imágenes nítidas para una detección precisa de *keypoints* faciales y corporales. Se conecta a través de la interfaz CSI de la Raspberry Pi. En la Tabla 5.2 se muestran sus especificaciones técnicas.



Figura 5.29: Cámara Módulo 3 NoIR para Raspberry Pi.

Característica	Valor
Sensor	Sony IMX708
Resolución	12 megapíxeles
Enfoque	Automático
Sensibilidad IR	Sí (sin filtro infrarrojo)
Interfaz	CSI (Camera Serial Interface)
Compatibilidad	Raspberry Pi 4, 3B+, 3B

Tabla 5.2: Especificaciones técnicas de la cámara NoIR.

5.4.1.1.3 Cable Flex CSI

Este cable flexible (Figura 5.30) permite la conexión de la cámara al puerto CSI de la Raspberry Pi. Con una longitud de 1 metro y formato plano, se adapta fácilmente a la estructura del espejo, facilitando su montaje interno sin interferencias con el resto del hardware. En la Tabla 5.3 se muestran sus especificaciones técnicas.



Figura 5.30: Cable Flex CSI para cámara Raspberry Pi.

Característica	Valor
Tipo de conector	Cinta plana de 15 pines
Compatibilidad	Puerto CSI de Raspberry Pi
Longitud	1 metro
Uso recomendado	Conexión flexible de cámaras CSI

Tabla 5.3: Especificaciones técnicas del cable Flex CSI.

5.4.1.1.4 Tarjeta microSD de 16 GB

La tarjeta microSD (Figura 5.31) es el medio de almacenamiento principal de la Raspberry Pi. En ella se aloja el sistema operativo, el modelo de IA y el código del servidor web. Gracias a su velocidad de lectura y escritura, garantiza un rendimiento fluido durante la ejecución del sistema. En la Tabla 5.4 se muestran sus especificaciones técnicas.



Figura 5.31: Tarjeta microSD de 16 GB.

Característica	Valor
Capacidad	16 GB
Clase de velocidad	Clase 10 / UHS-I
Velocidad mínima de escritura	10 MB/s
Uso	Sistema operativo y almacenamiento del software del proyecto

Tabla 5.4: Especificaciones técnicas de la tarjeta microSD.

5.4.1.1.5 Monitor HP LE2101W

El monitor (Figura 5.32) es el encargado de mostrar la interfaz gráfica del sistema. Está montado justo detrás del espejo semitransparente, permitiendo superponer los elementos visuales (como la acción detectada) al reflejo del usuario. En la Tabla 5.5 se muestran sus características.



Figura 5.32: Monitor HP LE2101W.

Característica	Valor
Tamaño	20 pulgadas
Resolución	1600x900 píxeles
Relación de aspecto	16:9
Entrada de vídeo	VGA
Tiempo de respuesta	5 ms

Tabla 5.5: Especificaciones técnicas del monitor HP LE2101W.

5.4.1.1.6 Espejo acrílico semitransparente

El espejo (Figura 5.33) es el elemento frontal del sistema. Está fabricado con material acrílico semitransparente que permite reflejar la imagen del usuario mientras deja pasar parcialmente la luz del monitor trasero. Esta combinación permite superponer visualmente la información del sistema sobre la imagen reflejada del usuario. En la Tabla 5.6 se muestran sus características.

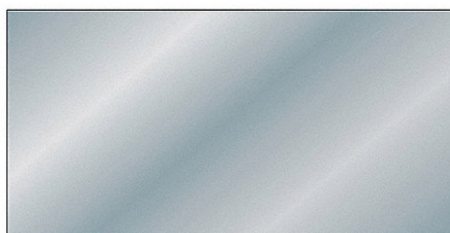


Figura 5.33: Espejo acrílico semitransparente.

Característica	Valor
Material	Acrílico semitransparente
Espesor	Aproximadamente 3 mm
Transparencia	Parcial (permite paso de luz desde el monitor)
Uso	Reflejo visual del usuario y proyección de la interfaz

Tabla 5.6: Especificaciones técnicas del espejo.

5.4.2 Modificaciones realizadas en el espejo

El espejo inteligente utilizado en este proyecto fue desarrollado previamente como parte del Trabajo de Fin de Grado de Elvira San José Amate, bajo la dirección de Juana López Redondo y Juan José Moreno Riado. A partir de esa base ya funcional, solo fue necesario realizar una serie de modificaciones tanto a nivel de hardware como de software para adaptar el dispositivo

al nuevo sistema de reconocimiento de actividades.

Estas configuraciones permitieron garantizar la compatibilidad con las bibliotecas utilizadas y asegurar el correcto funcionamiento del flujo de captura y procesamiento de datos.

5.4.2.1 Configuración de hardware

Respecto al hardware, era imprescindible conectar una cámara a la *Raspberry Pi*. Se utilizó la interfaz Interfaz serie de cámara, por sus siglas en inglés *Camera Serial Interface* (CSI), conectada a la placa por el puerto correspondiente. Además, se tuvo que realizar la habilitación de la cámara desde la configuración del sistema operativo. Esto se llevó a cabo mediante la herramienta *raspi-config*, donde se activó la opción correspondiente en el menú de interfaces. Sin esta activación, la *Raspberry Pi* no reconoce el periférico, imposibilitando su uso en el proyecto.

5.4.2.2 Configuración de software

En cuanto al software, se realizó la creación de un entorno virtual (*venv*) denominado *tfg*. El uso de entornos virtuales es una práctica recomendada en el desarrollo de proyectos en Python, ya que permite aislar las dependencias del proyecto respecto al sistema operativo u otros desarrollos. De este modo, se evitan conflictos entre versiones de bibliotecas y se facilita la portabilidad y capacidad de replicar el entorno de trabajo. Dentro de este nuevo entorno creado, se instalaron varias bibliotecas:

- **Mediapipe-rpi4==0.8.8:** Versión adaptada para *Raspberry Pi*, ya que las más recientes presentan incompatibilidades con la arquitectura ARM de la placa. Esta biblioteca es fundamental para la detección de puntos clave en las imágenes.
- **Protobuf==3.20.3:** Se desinstaló la versión instalada por defecto y se forzó la instalación de esta versión concreta, necesaria para garantizar la compatibilidad con la versión de MediaPipe seleccionada.
- **Tensorflow:** Utilizado para la carga y ejecución del modelo de reconocimiento de acciones. Se instaló la versión precompilada compatible con la arquitectura de la *Raspberry Pi*.

5.4.3 Análisis de requisitos

El análisis de requisitos permite identificar las necesidades y funcionalidades que debe cumplir el sistema, tanto desde el punto de vista del usuario como del desarrollador. Abarca los requisitos funcionales y no funcionales, los actores que interactúan con el sistema y los principales escenarios de uso.

5.4.3.1 Requisitos funcionales

Los requisitos funcionales describen las funcionalidades esenciales que debe cumplir el sistema para alcanzar sus objetivos operativos y ofrecer una experiencia adecuada al usuario:

- **RF1:** El sistema debe capturar imágenes mediante la cámara integrada.
- **RF2:** Las imágenes deben normalizarse para obtener una representación estandarizada.
- **RF3:** El sistema debe pasar una secuencia de 10 imágenes al modelo para predecir la acción.
- **RF4:** Si se detecta la misma acción al menos dos veces seguidas y luego cambia, debe registrarse con hora y duración estimada.
- **RF5:** El sistema debe mostrar las imágenes y la acción detectada en la interfaz web en tiempo real.
- **RF6:** El sistema debe permitir consultar el historial de acciones detectadas desde el navegador.

5.4.3.2 Requisitos no funcionales

Los requisitos no funcionales especifican restricciones técnicas, de calidad y diseño que debe cumplir el sistema para garantizar su estabilidad, eficiencia y facilidad de uso:

- **RNF1. Rendimiento:** El sistema debe procesar en tiempo real.
 - **RNF2. Portabilidad:** Debe ser ejecutable en una Raspberry Pi sin requerir de hardware adicional.
 - **RNF3. Fiabilidad:** Debe continuar operando ante errores puntuales de lectura o escritura.
 - **RNF4. Usabilidad:** La interfaz debe ser sencilla y apta para personas mayores.
 - **RNF5. Seguridad:** El sistema debe funcionar localmente sin exponer información en redes externas.
 - **RNF6. Mantenibilidad:** Debe ser posible incorporar nuevas acciones o modelos sin reestructurar el sistema
-

5.4.3.3 Actores del sistema

Los actores del sistema representan los distintos roles que interactúan directa o indirectamente con la aplicación. La identificación de estos actores permite definir mejor los flujos de trabajo y los casos de uso asociados:

- **Usuario:** Persona que realiza acciones frente al espejo. No requiere conocimientos técnicos, ya que la interacción es completamente autónoma.
- **Sistema:** Conjunto de componentes hardware y software que capturan, procesan y predicen la acción del usuario a partir de una secuencia de imágenes.
- **Cuidador o familiar:** Persona encargada de supervisar que el usuario ha realizado las rutinas de higiene y adecuadas. Puede acceder al historial de acciones desde la interfaz web, sin necesidad de interactuar con el sistema a nivel técnico.

5.4.3.4 Casos de Uso

A continuación, se muestra en la Figura 5.34 el diagrama de casos de uso de la aplicación web.

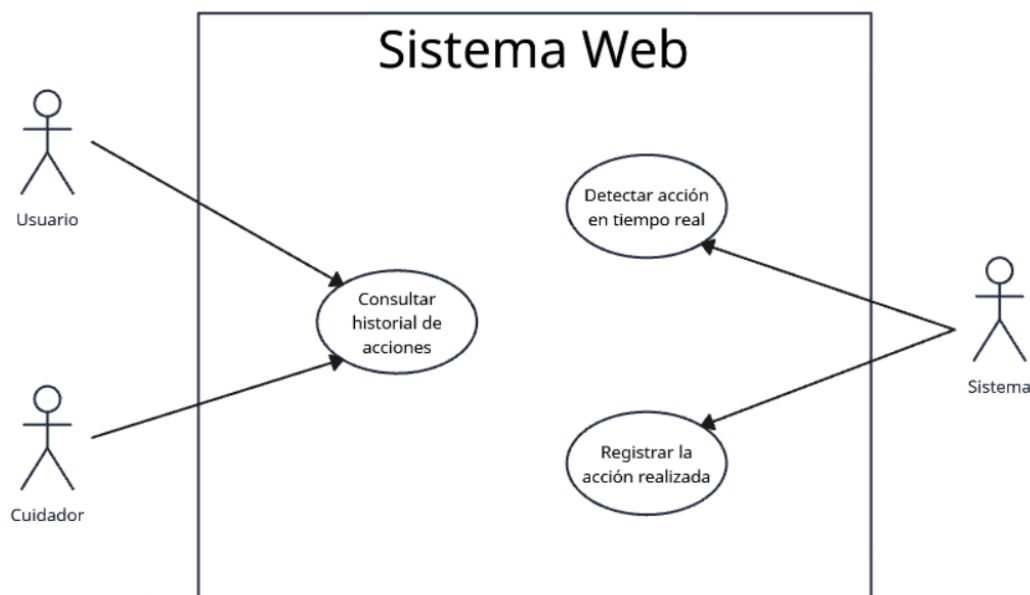


Figura 5.34: Diagrama de casos de uso.

Una vez definidos los requisitos del sistema, se detallan los casos de uso. Estos recogen las posibles interacciones entre usuarios y sistema, permitiendo identificar escenarios funcionales clave. Se recogen en la Tabla 5.7, la Tabla 5.8 y la Tabla 5.9.

Nombre	Detectar acción en tiempo real
Actores	Usuario
Descripción	El sistema captura imágenes, las normaliza y predice la acción realizada.
Requisitos Asociados	RF1, RF2, RF3
Precondiciones	El sistema está encendido y la cámara está operativa.
Postcondiciones	Se muestra la acción detectada en la interfaz web.
Flujo Principal	• Se capturan 10 imágenes. • Se normalizan los <i>keypoints</i>. • Se predice la acción con el modelo. • Se muestra el resultado en la interfaz.
Flujo Alternativo	Si no se detectan puntos clave suficientes, la imagen se descarta.

Tabla 5.7: Caso de uso 1: Detectar acción en tiempo real.

Nombre	Registrar acción realizada
Actores	Sistema
Descripción	Se registra una acción repetida cuando cambia a otra distinta.
Requisitos Asociados	RF3, RF4
Precondiciones	Se ha detectado una secuencia estable de acción.
Postcondiciones	La acción se guarda en un archivo CSV.
Flujo Principal	• Se detecta acción repetida. • Al cambiar la acción, se calcula duración. • Se registra hora, acción y duración.
Flujo Alternativo	Si no hay suficiente repetición, no se registra.

Tabla 5.8: Caso de uso 2: Registrar acción realizada.

Nombre	Consultar historial de acciones
Actores	Usuario, Cuidador o familiar
Descripción	El usuario o su cuidador acceden a una página web donde pueden consultar las acciones registradas, junto con su hora y duración estimada.
Requisitos Asociados	RF5, RF6
Precondiciones	Existe un archivo de historial con registros.
Postcondiciones	El usuario o cuidador visualiza la tabla de acciones registradas.
Flujo Principal	• Se accede a la ruta /registro. • Se carga el archivo CSV. • Se muestra una tabla HTML con los datos.
Flujo Alternativo	Si el archivo está vacío o no existe, se muestra un mensaje informativo.

Tabla 5.9: Caso de uso 3: Consultar historial de acciones.

5.4.4 Funcionamiento del sistema

El sistema desarrollado tiene como finalidad detectar en tiempo real las acciones que un usuario realiza frente a un espejo inteligente. Para ello, se sirve de una cámara integrada, la cual captura imágenes que son procesadas mediante algoritmos de visión por computador y una red neuronal previamente entrenada. Este proceso se compone de distintas fases que se describen a continuación: la obtención y normalización de los *keypoints*, la predicción de la acción, el almacenamiento del resultado y su visualización en la pantalla.

En la Figura 5.35 se representa el funcionamiento general del sistema mediante un diagrama de flujo. Este diagrama muestra la secuencia completa que sigue el espejo inteligente, desde su puesta en marcha hasta la detección e interpretación de acciones de higiene personal por parte del usuario, así como su posterior visualización e historial. A continuación, se detallan las distintas fases del proceso:

1. **Inicio del sistema:** El sistema comienza comprobando la disponibilidad de la cámara integrada en el espejo. En caso de que esta no esté operativa, se muestra un mensaje de error y se intenta reintentar la conexión. Si está disponible, se procede a la inicialización de los módulos necesarios: MediaPipe para la detección de puntos clave de la pose, las manos y la cara, y el modelo de inferencia entrenado previamente para el reconocimiento de acciones.
2. **Inicialización del entorno web:** Una vez preparados los módulos de visión e inferencia,

se pone en marcha el servidor web basado en Flask, encargado de actualizar la interfaz gráfica en tiempo real, mostrando tanto la acción reconocida como el historial de actividades.

3. **Captura y procesamiento continuo de imágenes:** En el bucle principal del sistema, se capturan imágenes de forma constante desde la cámara. En cada imagen, se evalúa si hay presencia de puntos clave reconocibles. Si no se detectan, se ignora la imagen y se pasa al siguiente. Si se detectan, se extraen los *keypoints* relevantes (cara, manos y cuerpo) y se normalizan mediante centrado y escalado. La imagen procesada se guarda temporalmente.
 4. **Formación de secuencia y predicción:** Una vez se han almacenado 10 imágenes consecutivas, estas se agrupan para formar una secuencia de entrada al modelo. Esta secuencia es enviada al sistema de inferencia, que devuelve una predicción sobre la acción que el usuario está realizando.
 5. **Comparación y validación de la acción:** El sistema compara la acción recién predicha con la acción inmediatamente anterior. Si ambas coinciden, se incrementa un contador. Si no coinciden, se evalúa si el contador anterior ha alcanzado un umbral mínimo (normalmente 2 repeticiones). Si se ha superado, se registra la acción anterior como válida y se calcula su duración estimada (número de repeticiones multiplicado por 10). Si no se alcanza el umbral, se descarta. A continuación, se reinicia el contador con la nueva acción detectada.
 6. **Visualización e historial:** La acción actual se muestra inmediatamente en la interfaz web del espejo. Si cumple los criterios de validación, se almacena en un historial interno con su *timestamp* y duración estimada. Finalmente, se eliminan las imágenes temporales usadas para esa secuencia, manteniendo optimizado el sistema.
-

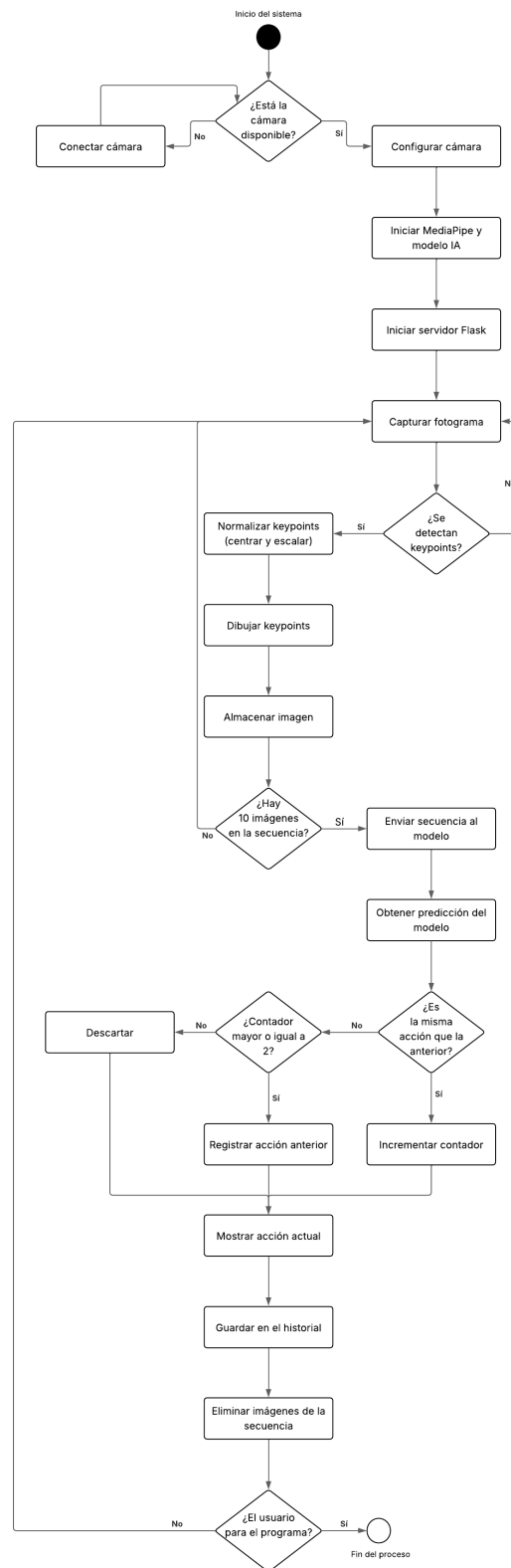


Figura 5.35: Diagrama de flujo del funcionamiento del sistema.

5.4.4.1 Escaneo e identificación de actividades

La detección de actividades comienza con la captura de imágenes desde la cámara integrada. Para ello, se emplea la herramienta *libcamera-vid*, que emite un flujo de vídeo accesible mediante el protocolo TCP local (Código 5.16). Este flujo es leído imagen a imagen por OpenCV.

Código 5.16: Captura de vídeo en tiempo real.

```
1 libcamera_process = subprocess.Popen([
2     'libcamera-vid', '-t', '0', '--inline', '--listen', '--nopreview',
3     '-o', 'tcp://127.0.0.1:8888',
4     '--width', '1640', '--height', '1232',
5     '--bitrate', '2000000', '--framerate', '1'
6 ])
7 cap = cv2.VideoCapture('tcp://127.0.0.1:8888', cv2.CAP_FFMPEG)
```

Es fundamental que la cámara tenga la misma configuración (resolución, tasa de imágenes y orientación) que la empleada durante la grabación del dataset de entrenamiento. Esto garantiza que las condiciones visuales del entorno —como el tamaño relativo del sujeto y su posición— se mantengan constantes.

Además, cada imagen capturada se normaliza mediante el mismo proceso que se aplicó a los datos de entrenamiento: centrado en el torso del usuario y escalado a una dimensión fija. Esto asegura que las entradas del modelo mantengan la misma estructura y proporción que las utilizadas para su entrenamiento, lo que resulta esencial para evitar errores de predicción. Esto se muestra en el Código 5.17.

Código 5.17: Normalización de puntos clave.

```
1 centro = (min_xy + max_xy) / 2.0
2 puntos_pose = np.array(pose_ordered) - centro
3 escala = POSE_BOX_SIZE / max(size_box)
4 px_pose = proyectar(puntos_pose.tolist())
```

En la Figura 5.36 se muestra un ejemplo de una imagen generada tras la normalización de los puntos clave.

Estas imágenes normalizadas se almacenan en una carpeta temporal. Una vez que se han capturado 10 imágenes consecutivas, se forma una secuencia lista para ser interpretada por el modelo.

5.4.4.2 Ejecución en tiempo real del modelo

El modelo de IA empleado es una red neuronal híbrida CNN-LSTM exportada en formato TensorFlow Lite para poder ejecutarse en la Raspberry Pi. Una vez almacenadas las 10 imágenes, se cargan como entrada del modelo y se obtiene la clase predicha (Código 5.18).

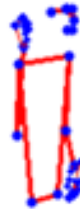


Figura 5.36: Ejemplo imagen normalizada.

Código 5.18: Inferencia del modelo.

```
1 input_seq = np.expand_dims(np.array(secuencia_imgs), axis=0).astype(np.float32)
2 interpreter.set_tensor(input_details[0]['index'], input_seq)
3 interpreter.invoke()
4 pred = interpreter.get_tensor(output_details[0]['index'])
5 clase_detectada = clases[np.argmax(pred)]
```

Si la acción se ha repetido al menos dos veces consecutivas y cambia posteriormente, se considera confirmada. Entonces, se registra junto con la hora y la duración estimada, como se muestra en el Código 5.19.

Código 5.19: Registro de actividad detectada.

```
1 if conteo >= 1:
2     segundos = (conteo + 1) * 10
3     hora = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
4     registrar_accion(anterior, hora, segundos)
```

5.4.5 Almacenamiento y recuperación de datos

Cada vez que se detecta una acción válida, esta se almacena en un archivo CSV con la siguiente estructura: nombre de la acción, hora de detección y duración (Código 5.20 y Figura 5.37). Este sistema permite registrar de forma eficiente el historial de actividad del usuario.

Código 5.20: Almacenamiento en archivo CSV.

```
1 with open(archivo, mode="a", newline="", encoding="utf-8") as file:
2     writer = csv.writer(file)
3     if not existe:
4         writer.writerow(["Acción", "Hora", "Duración (segundos)"])
5     writer.writerow([nombre, hora_actual, duracion_segundos])
```

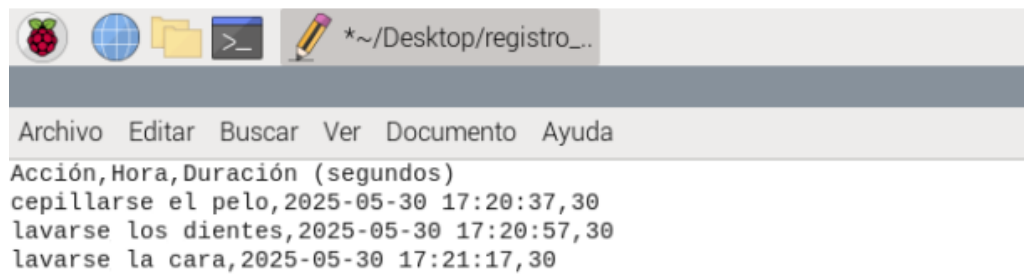


Figura 5.37: Almacenamiento en archivo CSV de las acciones detectadas.

Para visualizar el historial, el sistema dispone de una ruta web (`/registro`) que accede al archivo y lo representa en una tabla Lenguaje de marcado de hipertexto, por sus siglas en inglés *HyperText Markup Language* (HTML) usando Flask y Jinja2. Esto se puede observar en la Figura 5.38.

Acción	Hora	Duración (segundos)
lavarse la cara	2025-05-30 17:21:17	30
lavarse los dientes	2025-05-30 17:20:57	30
cepillarse el pelo	2025-05-30 17:20:37	30

Figura 5.38: Visualización del historial de acciones detectadas.

5.4.6 Muestra de actividades en la pantalla

La interfaz gráfica (Figura 5.39) ha sido diseñada para ser clara y accesible. Se muestran las acciones pendientes y realizadas, junto con la imagen del modelo con puntos clave en directo. El color del fondo de cada acción indica si ya ha sido realizada o no, siendo verde cuando sí se ha realizado y rojo cuando no.

Mediante JavaScript, el sistema consulta periódicamente el endpoint `/accion_detectada` para actualizar la acción en curso. Cuando una acción se mantiene durante 15 segundos, esta se marca como realizada y se muestra un vídeo de los movimientos del usuario (Código 5.21).

Código 5.21: Detección y confirmación de acción en el cliente.

```
1 setInterval(async () => {
2   const res = await fetch('/accion_detectada');
```



Figura 5.39: Interfaz web con lista de acciones y stream en vivo.

```
3 const data = await res.json();  
4 if (data.accion) actualizarAccionDetectada(data.accion);  
5 }, 1000);
```

Esta retroalimentación visual permite al usuario recibir confirmación inmediata de que ha realizado correctamente una de las tareas previstas, reforzando la autonomía y adherencia a rutinas diarias.

5.5 Presupuesto

A continuación, se detalla el presupuesto estimado para el desarrollo del presente (Tabla 5.10). Se incluyen tanto los componentes físicos necesarios para la construcción del espejo inteligente como el coste de la mano de obra empleada en su diseño, implementación y pruebas. El cálculo se realiza con precios aproximados de mercado y una estimación razonable del coste horario del trabajo.

El cálculo del coste de mano de obra se ha realizado considerando una tarifa estimada de 8,95€ por hora. Este valor hace referencia al salario mínimo interprofesional en España para el año 2024 en el ámbito del sector tecnológico, aplicado a perfiles sin titulación universitaria. Se ha estimado una dedicación total de 330 horas, abarcando todas las fases del proyecto: diseño del sistema, desarrollo de la aplicación, entrenamiento del modelo de IA, integración de hardware y pruebas funcionales.

El presupuesto total asciende a 3.193,27€, incluyendo tanto los materiales empleados como el coste del trabajo realizado. Esta cifra ofrece una estimación razonable del coste que supondría implementar un sistema similar en un contexto académico o de investigación.

Objeto usado	Cantidad	Precio Unitario (€)	Total (€)
Raspberry Pi 4 Model B con cargador	1 unidad	66,39	66,39
Cámara Módulo 3 NoIR para Raspberry Pi	1 unidad	20,99	20,99
Cable Flex CSI para cámara Raspberry Pi	1 unidad	2,91	2,91
Monitor HP LE2101W	1 unidad	82,00	82,00
Espejo acrílico semitransparente	1 unidad	51,86	51,86
Cable micro HDMI a HDMI	1 unidad	5,20	5,20
Marco para espejo	1 unidad	10,42	10,42
Mano de obra	330 horas	8,95	2.953,50
Total			3.193,27

Tabla 5.10: Resumen del presupuesto del proyecto.

Conviene destacar que algunos de los componentes hardware incluidos en la tabla ya estaban disponibles en el entorno de desarrollo, por lo que no fue necesario adquirirlos específicamente para este proyecto. Sin embargo, se han incorporado a efectos valorativos, con el fin de reflejar de forma más precisa el coste de reproducción del sistema en caso de que se quiera replicar en otro entorno.

6 Resultados

6.1 Desempeño del modelo

Esta sección presenta una revisión de los principales desafíos técnicos encontrados durante el desarrollo del sistema, así como una evaluación cualitativa de su funcionamiento en distintos contextos. Se analizan los aspectos que han condicionado el rendimiento del modelo, desde las limitaciones del hardware hasta las variaciones en las condiciones del entorno, con el fin de comprender su comportamiento real en situaciones representativas de uso.

6.1.1 Análisis mediante matrices de confusión

Para complementar la evaluación del modelo, se han generado matrices de confusión correspondientes a los conjuntos de entrenamiento, validación y test. Estas matrices permiten visualizar con mayor claridad los aciertos y errores del modelo al clasificar las distintas acciones, identificando patrones de confusión entre clases concretas.

En la Figura 6.1, se observa que el modelo ha aprendido adecuadamente durante el entrenamiento, con una tasa de acierto cercana al 100% en todas las clases. No se detectan errores significativos, lo que indica que la red es capaz de memorizar correctamente los patrones del conjunto de entrenamiento.

En la Figura 6.2 se presentan los resultados sobre el conjunto de validación. En este caso, ya aparecen algunas confusiones, especialmente entre las acciones *lavarse la cara* y *lavarse las manos*, lo que sugiere similitudes en los patrones de movimiento o interferencias debidas a la posición de las manos. En la Figura 6.3 se muestra un ejemplo de la similitud de ambas acciones. También se observa una ligera confusión en la clase *lavarse los dientes*, que podría deberse a la eliminación de los puntos faciales en la fase de integración, reduciendo así la información visual disponible para distinguir movimientos cercanos al rostro.

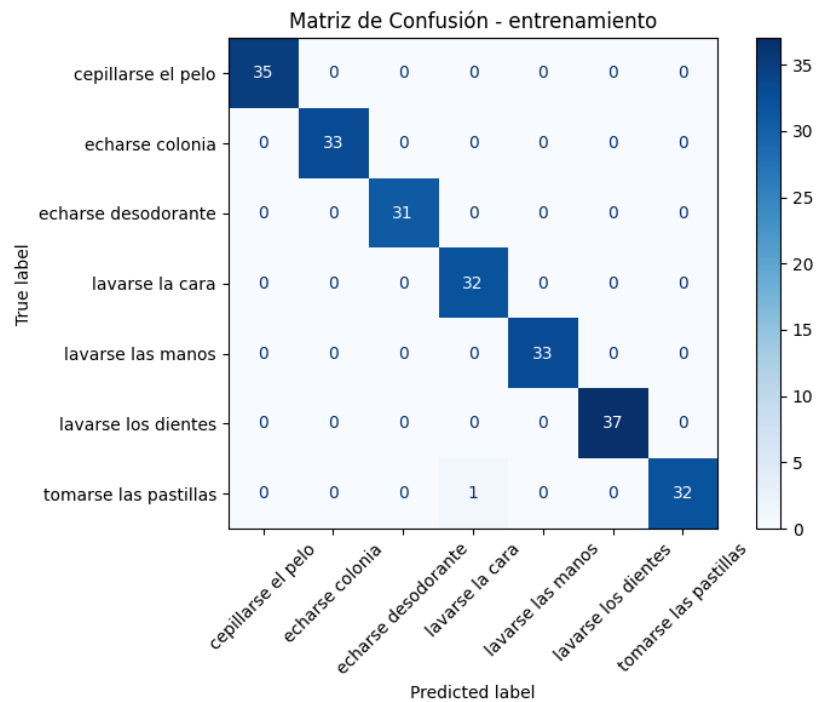


Figura 6.1: Matriz de confusión - conjunto de entrenamiento.

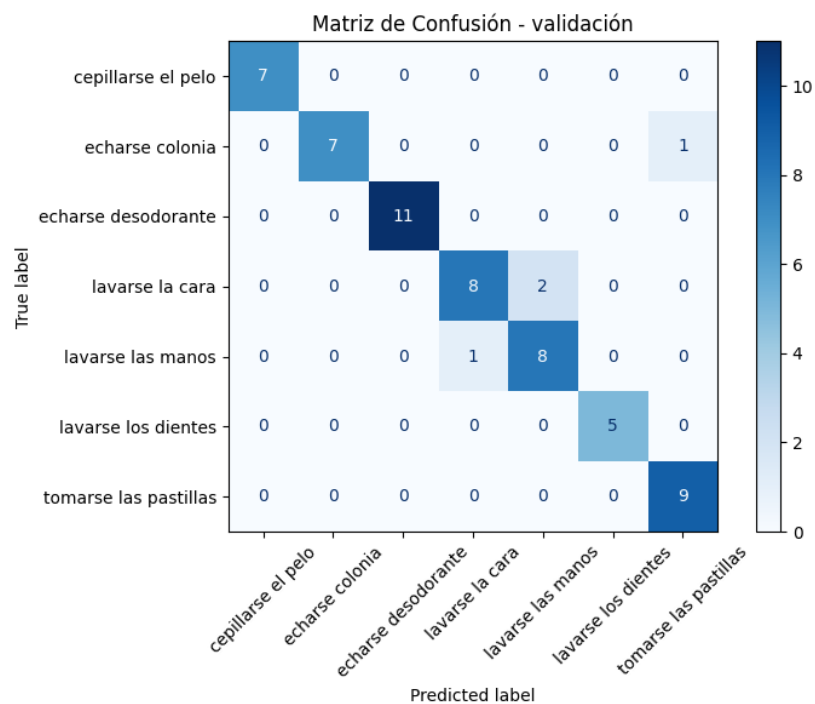


Figura 6.2: Matriz de confusión - conjunto de validación.

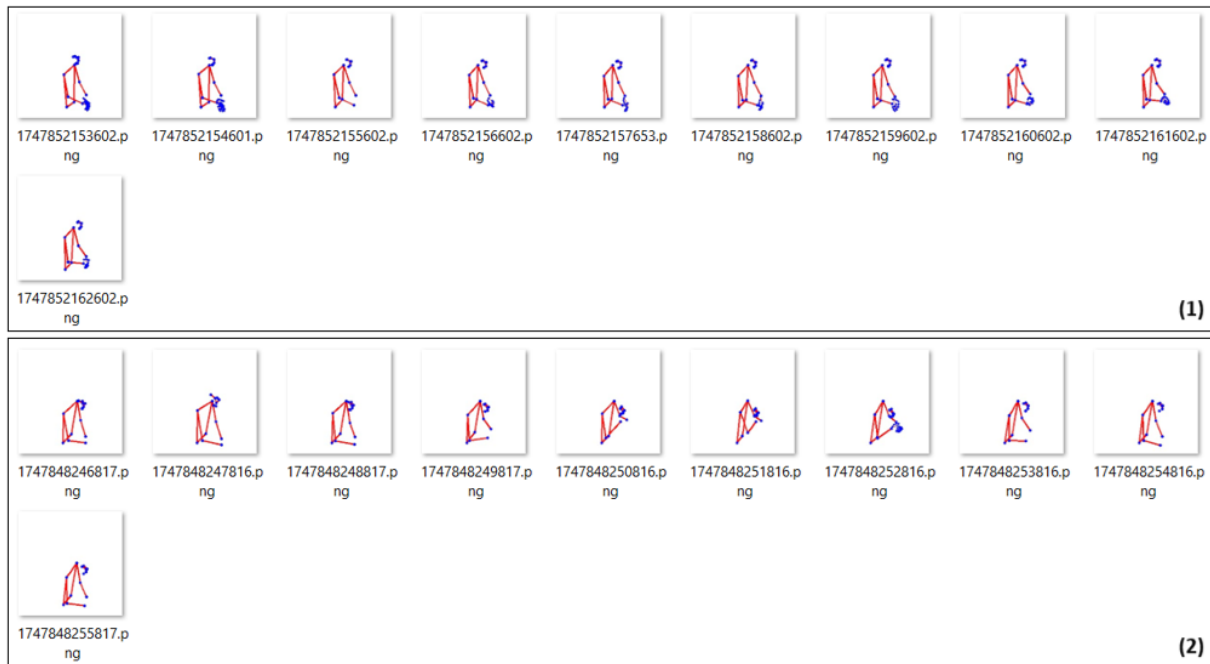


Figura 6.3: Similitudes entre lavarse las manos (1) y lavarse la cara (2).

Por último, la Figura 6.4 muestra los resultados obtenidos en el conjunto de test. Se observa una buena generalización del modelo, con la mayoría de las acciones correctamente identificadas. Las únicas confusiones relevantes ocurren entre acciones que comparten cierta similitud gestual, como *echarse colonia* y *cepillarse el pelo*, lo cual es comprensible dada la proximidad de las zonas del cuerpo involucradas. En la Figura 6.5 se muestran las similitudes del ejemplo.

En conjunto, estas matrices de confusión confirman que el modelo ha sido capaz de aprender los patrones de las distintas acciones con un alto grado de precisión, manteniendo una buena capacidad de generalización en condiciones de prueba. No obstante, también evidencian que algunas clases tienden a confundirse entre sí, especialmente cuando se basan en movimientos similares o afectan a regiones corporales próximas.

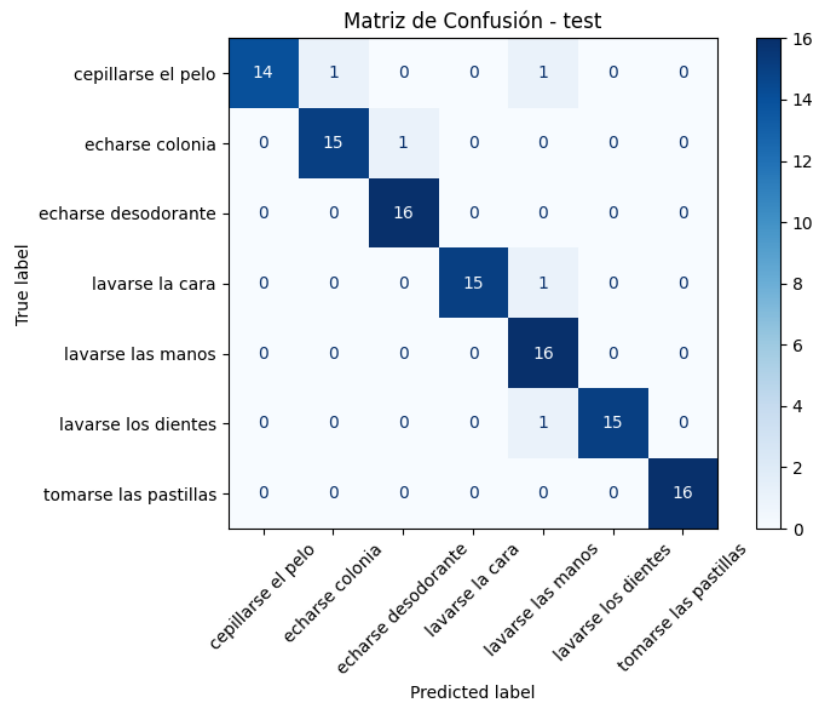


Figura 6.4: Matriz de confusión - conjunto de test.

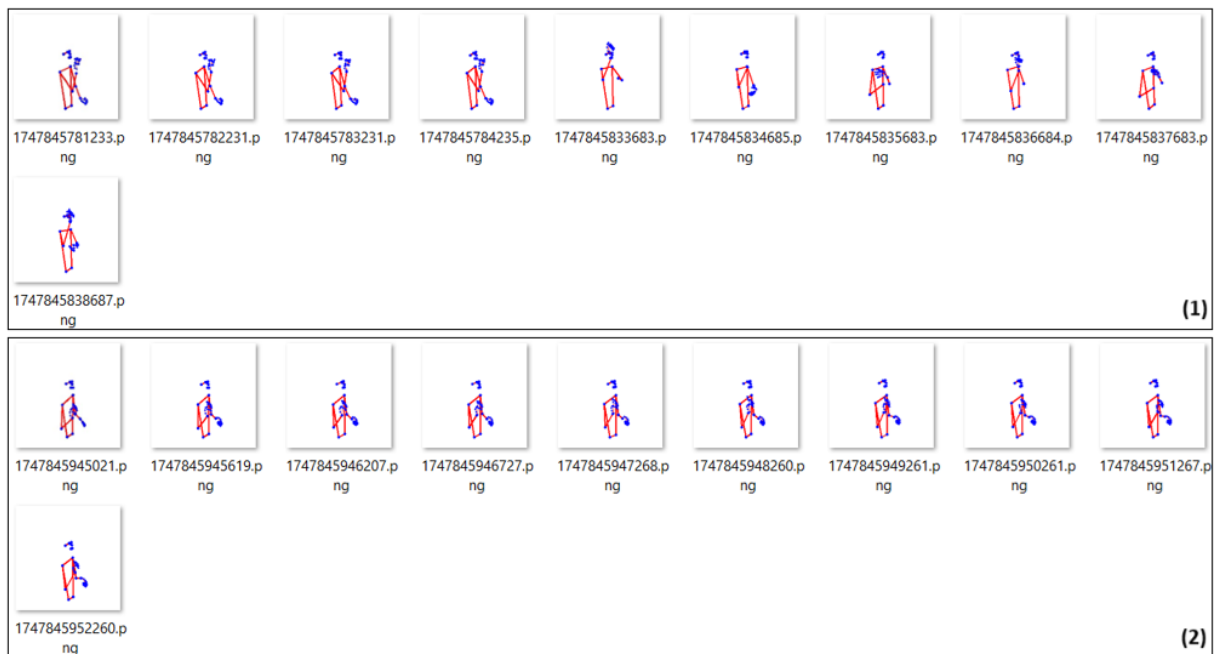


Figura 6.5: Similitudes entre cepillarse el pelo (1) y echarse colonia (2).

6.1.2 Dificultades

A lo largo del desarrollo del sistema se han presentado diversas dificultades que han requerido soluciones específicas y ajustes tanto en la planificación como en la ejecución de las pruebas.

Uno de los principales retos fue la incorporación de todo el sistema en una Raspberry Pi. Las limitaciones de procesamiento del dispositivo dificultaban la ejecución fluida de todas las tareas necesarias: captura de vídeo, detección de *keypoints*, inferencia del modelo, renderizado en la interfaz web y escritura de registros. Esta sobrecarga provocaba retardos significativos de hasta más de 2 minutos en la respuesta del sistema. Para mitigar este problema, se adoptaron medidas como limitar la tasa de imágenes a 1 FPS y reducir la resolución de las imágenes, lo que permitió estabilizar el rendimiento y reducir el retraso a aproximadamente 2-3 segundos por predicción.

También se presentaron dificultades durante la gestión y normalización de los datos de entrada. Como el modelo trabaja con secuencias de imágenes generadas a partir de *keypoints*, fue necesario diseñar un proceso que garantizara que las imágenes utilizadas durante la inferencia tuvieran exactamente el mismo formato que aquellas con las que se entrenó la red neuronal. Esto implicó normalizar los puntos clave mediante centrado y escalado, asegurando la coherencia en todos los pasos del pipeline, incluyendo la fase en tiempo real.

Asimismo, se detectaron errores relacionados con la ejecución del servidor Flask en la Raspberry Pi, como el bloqueo de puertos o conflictos entre instancias activas del proceso. Esto obligó a implementar controles adicionales para evitar instancias duplicadas del servidor y asegurar una inicialización limpia en cada ejecución.

Por último, durante la recolección del conjunto de datos se presentaron desafíos técnicos derivados del uso de MediaPipe. En situaciones con iluminación deficiente o posiciones atípicas del usuario, algunos *keypoints* no eran detectados correctamente, lo que comprometía la calidad de los datos. Para solventar este problema, se realizó una depuración de las imágenes y se añadieron validaciones para excluir aquellas que no cumplieran un mínimo de *keypoints* detectados.

Además, cuando el sistema estuvo completamente integrado y se desplegó en la Raspberry Pi para las pruebas finales, se observó que la captura simultánea de puntos clave de pose, manos y cara generaba una sobrecarga considerable en el procesamiento. Esto aumentaba drásticamente el retardo en la ejecución, haciendo inviable su uso en tiempo real. Como solución, se decidió eliminar la detección de los puntos de la cara, reduciendo así la complejidad computacional y permitiendo un funcionamiento mucho más fluido sin comprometer significativamente la precisión del modelo.

6.2 Simulación de uso

Para validar la utilidad del sistema en una situación real, se ha llevado a cabo una simulación de uso en la que un usuario ejecuta varias acciones de aseo frente al espejo inteligente para

obtener resultados en tiempo real.

En esta simulación, el sistema se encuentra en espera continua, capturando imágenes desde la cámara cada segundo. Cuando se han recogido diez imágenes consecutivas, éstas se normalizan y se pasan al modelo para inferir la acción. Si se detecta la misma acción en al menos dos bloques de imágenes consecutivas y posteriormente se identifica un cambio, el sistema registra dicha actividad junto con el tiempo estimado de duración.

Durante las pruebas, se simularon acciones como *echarse colonia* o *lavarse la cara*, observando cómo el sistema es capaz de detectar las transiciones entre actividades, mostrar en pantalla la acción actual y guardar un historial de acciones realizadas, junto con sus duraciones.

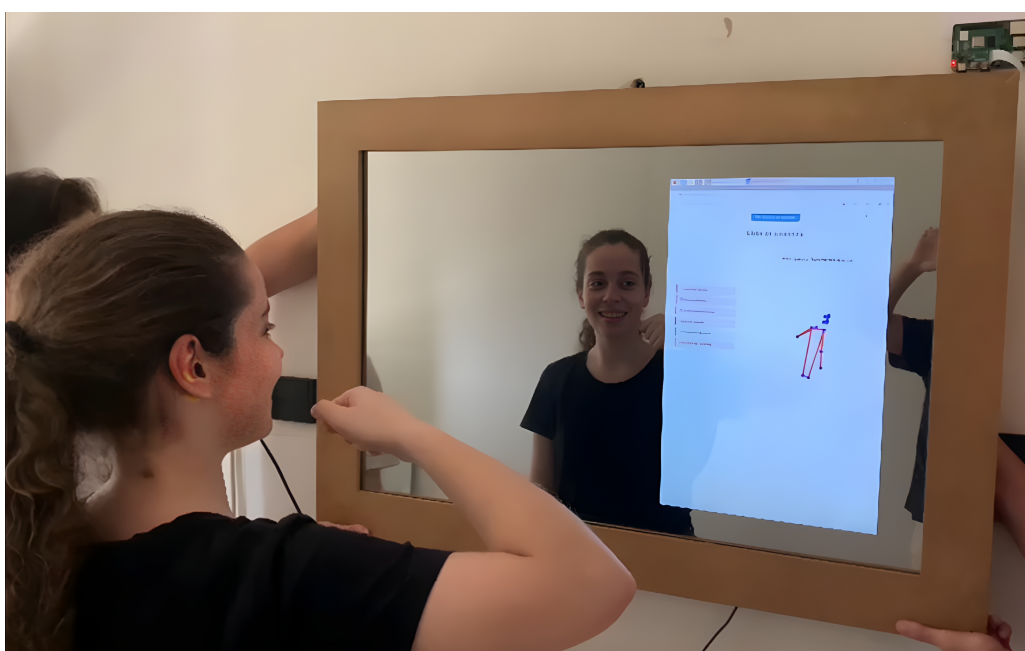


Figura 6.6: Prueba real del funcionamiento del espejo.

Estas simulaciones reflejan un posible escenario cotidiano (Figura 6.6) en el que el sistema ayuda al usuario a mantener su rutina diaria, a la vez que proporciona un registro consultable para familiares o cuidadores. Se muestran los resultados de la web en la Figura 6.7, Figura 6.8, Figura 6.9, Figura 6.10, Figura 6.11, Figura 6.12 y Figura 6.13.



Figura 6.7: Acción de cepillarse el pelo en la web.

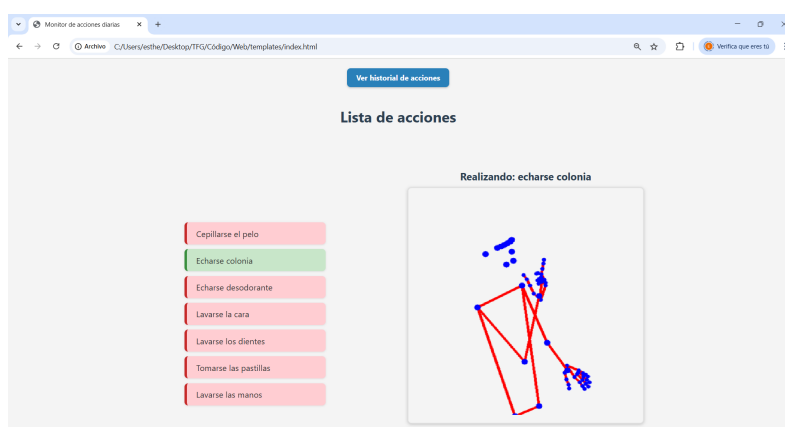


Figura 6.8: Acción de echarse colonia en la web.

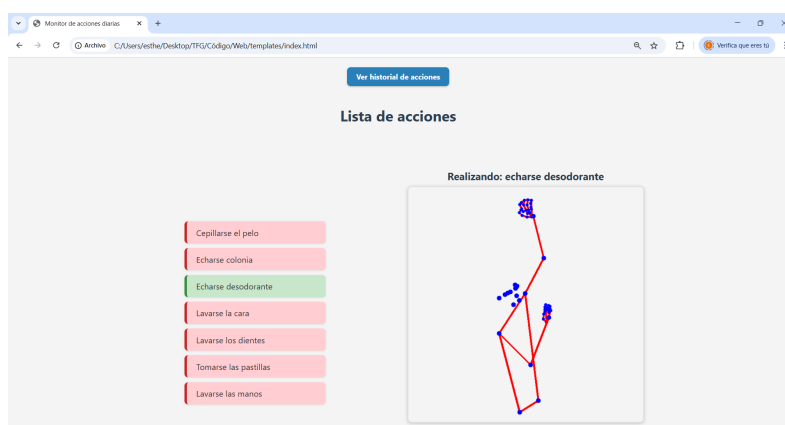


Figura 6.9: Acción de echarse desodorante en la web.

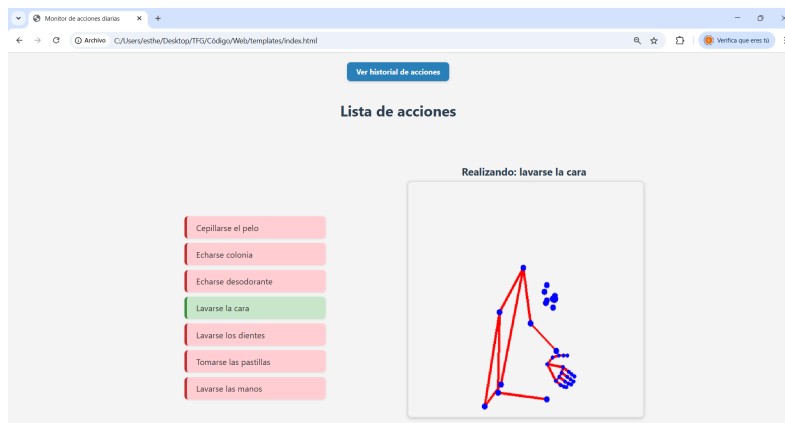


Figura 6.10: Acción de lavarse la cara en la web.

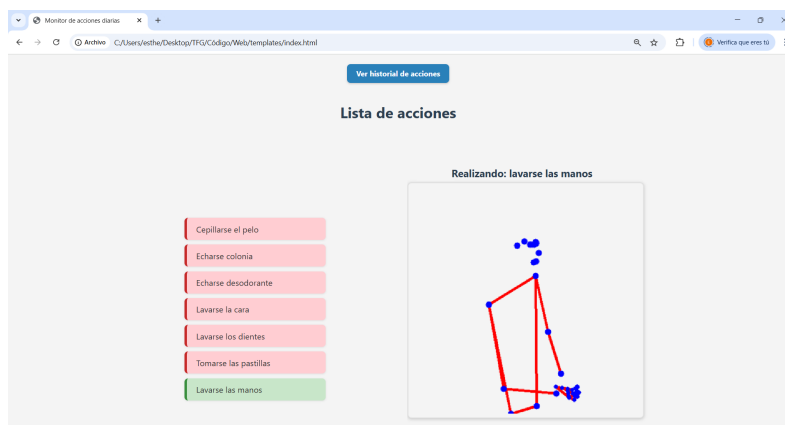


Figura 6.11: Acción de lavarse las manos en la web.



Figura 6.12: Acción de lavarse los dientes en la web.

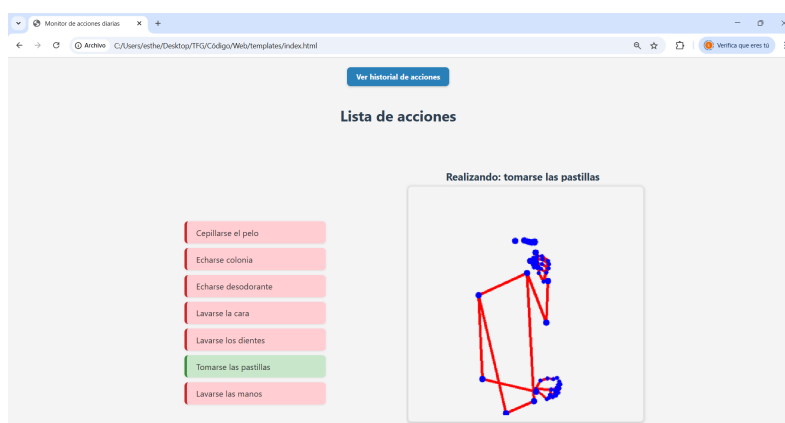


Figura 6.13: Acción de tomarse las pastillas en la web.

Una vez realizadas todas las acciones, el resultado en la web se corresponde con la Figura 6.14.



Figura 6.14: Todas las acciones realizadas vista desde la web.

6.3 Evaluación del modelo en diferentes escenarios

Con el objetivo de comprobar la robustez y la precisión del sistema en condiciones variadas de uso, se han diseñado y ejecutado diferentes pruebas prácticas. Estas pruebas permiten valorar el comportamiento del modelo ante factores que pueden afectar directamente a la calidad de los datos capturados y, por tanto, a la inferencia realizada por la red neuronal.

Cada prueba se ha realizado utilizando siete acciones distintas del conjunto original, aplicando dos intentos por acción, lo que da un total de 14 intentos en cada escenario de evaluación. Esta metodología permite observar posibles inconsistencias entre repeticiones y obtener una media representativa del rendimiento del sistema.

6.3.1 Precisión según el lado de ejecución

Se entrenaron tres modelos distintos para evaluar la influencia del lado del cuerpo con el que se realizan las acciones: uno con imágenes capturadas exclusivamente con la mano derecha, otro con la izquierda y un tercero con ambas manos. El objetivo era comprobar si la red aprendía correctamente los patrones cuando se alteraba la simetría de las acciones.

Los resultados muestran que el modelo entrenado con la mano derecha obtuvo 13 aciertos de 14 intentos, seguido por el modelo entrenado con la izquierda, que acertó en 12 de 14 casos. En cambio, el modelo entrenado con ambos lados presentó un rendimiento inferior, con 10 aciertos sobre 14. En la Figura 6.15 podemos ver gráficamente estos resultados.

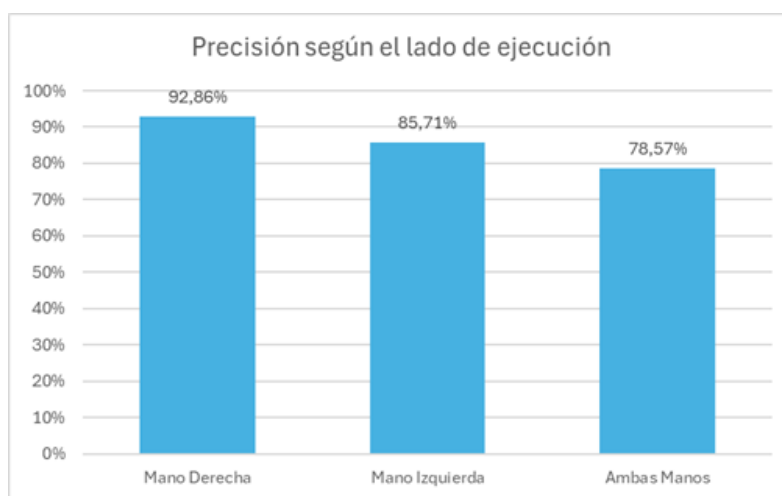


Figura 6.15: Porcentaje de acierto según la mano de ejecución.

6.3.2 Influencia de obstáculos en el campo visual

Para estudiar la tolerancia del sistema a interferencias físicas en la escena, se realizaron pruebas colocando objetos entre el usuario y la cámara. Se definieron tres niveles: sin obstáculos, pocos obstáculos (por ejemplo, una toalla o frascos parcialmente visibles) y muchos obstáculos (objetos grandes o múltiples elementos visuales que ocultan parcialmente al usuario).

Los resultados indican que en condiciones óptimas (sin obstáculos), el sistema alcanzó una tasa de acierto de 14/14. Con algunos obstáculos presentes, la precisión descendió a 10/14, y en los casos con una gran cantidad de elementos visuales interfiriendo, se obtuvo un rendimiento de 7/14. En la Figura 6.16 podemos ver gráficamente estos resultados.

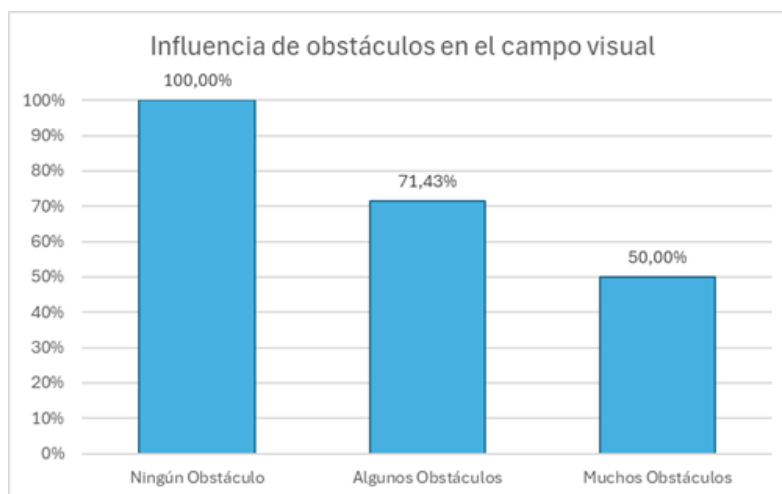


Figura 6.16: Porcentaje de acierto según los obstáculos en la habitación.

6.3.3 Variación según condiciones de iluminación

La iluminación del entorno también fue evaluada como factor crítico en el rendimiento del sistema. Se realizaron pruebas bajo tres escenarios distintos: buena iluminación (luz natural), iluminación media (luz ambiental difusa) e iluminación baja (condiciones nocturnas).

Los resultados fueron los siguientes: con buena iluminación, el modelo logró 13 aciertos sobre 14 intentos; con iluminación media, 11 aciertos; y en condiciones de poca luz, el rendimiento descendió notablemente a 5/14. En la Figura 6.17 podemos ver gráficamente estos resultados.

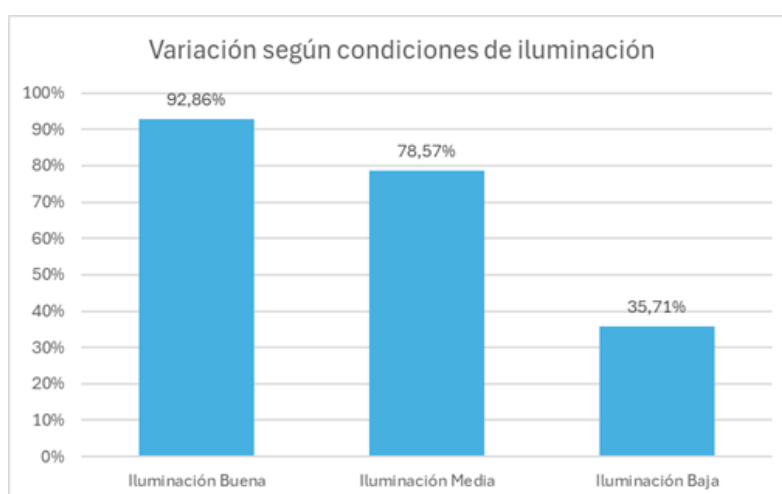


Figura 6.17: Porcentaje de acierto según la iluminación.

6.4 Conclusión de la evaluación

Los resultados obtenidos en las distintas pruebas permiten extraer varias conclusiones relevantes sobre el comportamiento del sistema. En primer lugar, se observa que un entrenamiento basado en una sola lateralidad (mano derecha o izquierda) tiende a generar modelos más consistentes y precisos, mientras que el uso combinado de ambas introduce una mayor variabilidad en los patrones de movimiento, lo que dificulta la generalización de la red neuronal.

Asimismo, se confirma la importancia de mantener un encuadre limpio y sin obstáculos visuales. La presencia de objetos entre el usuario y la cámara puede afectar significativamente a la detección de puntos clave por parte de MediaPipe, generando entradas erróneas al modelo y reduciendo la precisión global del sistema.

Por otro lado, aunque la cámara utilizada dispone de sensor infrarrojo, MediaPipe sigue dependiendo en gran medida de una visibilidad clara y adecuada de las extremidades para realizar la detección. Por tanto, las condiciones de iluminación continúan siendo un factor determinante para el correcto funcionamiento del sistema.

En conjunto, estas observaciones refuerzan la idea de que, si bien el sistema es capaz de operar con éxito en entornos reales, su rendimiento óptimo se alcanza en situaciones controladas en las que se cuida tanto la ejecución de las acciones como las condiciones del entorno.

6.4.1 Análisis por clase

Las matrices de confusión obtenidas para los conjuntos de entrenamiento, validación y prueba han permitido identificar qué acciones presentan un mejor rendimiento y cuáles tienden a generar errores de clasificación. Las acciones que se reconocen con mayor precisión son *tomarse las pastillas*, *echarse desodorante* y *lavarse las manos*, con tasas de acierto muy elevadas y sin apenas errores en los tres conjuntos. Esto sugiere que estas actividades poseen un patrón de movimiento distintivo, claramente diferenciable por el modelo.

En el caso de *cepillarse el pelo*, aunque existe una ligera confusión en el conjunto de prueba, la precisión se mantiene alta. Sin embargo, acciones como *lavarse la cara* y *lavarse las manos* presentan errores más frecuentes en validación, siendo confundidas entre sí en varias ocasiones. Esta ambigüedad puede deberse a que ambas comparten movimientos similares en la zona del rostro y el torso.

También se ha observado que la acción *lavarse los dientes*, que dependía en mayor medida de los puntos faciales, ha sufrido una ligera degradación en su precisión, especialmente en el conjunto de validación. Esto se explica por la eliminación de los *keypoints* de la cara durante la fase de ejecución en tiempo real, lo que dificulta distinguir correctamente esta actividad.

Por último, *echarse colonia* se confunde ocasionalmente con *echarse desodorante*, lo cual

resulta comprensible dada la similitud de sus gestos, en los que la mano se acerca a la parte superior del cuerpo.

En general, este análisis confirma que el rendimiento del modelo varía en función de la claridad gestual y la singularidad de los movimientos asociados a cada clase. Aquellas actividades con trayectorias más diferenciadas y sin interferencias tienden a ser reconocidas de forma más fiable.

7 Conclusiones y trabajo futuro

7.1 Conclusiones

Este TFG ha supuesto la culminación de un proceso de aprendizaje en el que se han integrado múltiples competencias adquiridas durante la carrera de Ingeniería Informática. El proyecto ha consistido en el desarrollo de un sistema inteligente, materializado en forma de espejo, capaz de reconocer acciones de higiene personal mediante visión por computador y técnicas de aprendizaje profundo.

El sistema implementado ha demostrado ser capaz de identificar acciones como lavarse los dientes, peinarse o echarse desodorante, gracias a la combinación de técnicas de extracción de puntos clave con MediaPipe y la posterior inferencia mediante un modelo entrenado con secuencias de imágenes. Este modelo ha sido optimizado para ejecutarse en tiempo real sobre una Raspberry Pi, con una interfaz web accesible que permite visualizar la actividad del usuario y almacenarla en un historial.

Durante el desarrollo, se ha seguido una metodología basada en el análisis de requisitos, identificación de actores y definición de casos de uso. Este enfoque ha sido posible gracias a los conocimientos adquiridos en la asignatura de *Ingeniería del Software*, donde se establecieron las bases para entender y aplicar técnicas de modelado funcional que aseguran que el sistema responda a las necesidades del usuario final.

La organización y gestión del trabajo han sido clave para abordar un proyecto con múltiples fases y dependencias técnicas. Para ello, han resultado especialmente útiles las herramientas y técnicas estudiadas en *Planificación y Gestión de Proyectos Informáticos*, que han permitido estructurar las tareas, estimar tiempos y recursos, y adaptar el plan de trabajo ante posibles desviaciones.

La obtención y tratamiento de datos visuales, especialmente los procesos de extracción, normalización y representación de *keypoints* sobre fondo blanco, se han apoyado en competencias desarrolladas en la asignatura de *Tratamiento Digital de Imágenes*. Estos conocimientos han sido fundamentales para preparar los datos de entrada del modelo de manera consistente y fiable.

La integración del sistema en un entorno físico real, utilizando la Raspberry Pi como plataforma de ejecución y gestionando la conectividad de la cámara y el flujo de datos, ha requerido aplicar conceptos de *Redes de Computadores*. Esta asignatura proporcionó los fundamentos necesarios para la configuración de puertos, direcciones IP y comunicación entre dispositivos.

El desarrollo de la interfaz web que acompaña al sistema, encargada de mostrar las acciones detectadas y registrar el historial de actividad, ha sido posible gracias a los conocimientos adquiridos en *Tecnologías Web*. El diseño y funcionamiento de esta aplicación ha sido pensado para ejecutarse de forma ligera y accesible desde cualquier navegador conectado a la red local.

Por último, la experiencia de uso del sistema ha sido cuidadosamente diseñada aplicando los principios de *Diseño de Interfaces de Usuario*. El objetivo ha sido ofrecer una interacción clara, con contrastes visuales adecuados, botones fácilmente reconocibles y navegación intuitiva, todo ello pensado específicamente para usuarios mayores con posibles dificultades tecnológicas.

Las pruebas realizadas en condiciones controladas han puesto de manifiesto que el modelo es capaz de distinguir con precisión razonable entre distintas acciones. No obstante, también se ha evidenciado que la fiabilidad del sistema depende de factores como la iluminación o la visibilidad de las extremidades.

En conjunto, este trabajo no solo ha confirmado la viabilidad técnica de un sistema de este tipo, sino que también ha puesto de relieve su utilidad potencial como herramienta de apoyo para personas mayores, contribuyendo a preservar su autonomía en tareas cotidianas del aseo personal.

7.2 Trabajo futuro

Aunque el sistema desarrollado cumple con los objetivos inicialmente propuestos, se han identificado diversas líneas de mejora que permitirían ampliar sus funcionalidades, aumentar su fiabilidad en entornos reales y reforzar su utilidad práctica.

Una de las principales propuestas consiste en la integración del historial de actividades con una base de datos centralizada. Actualmente, los registros se almacenan de forma local en archivos CSV, lo que limita las posibilidades de consulta remota y análisis a largo plazo. Al migrar estos datos a una base de datos en red, se abriría la posibilidad de conectarse con otros sistemas del hogar inteligente, permitiendo, por ejemplo, la consulta del historial mediante asistentes conversacionales.

En este sentido, una línea especialmente relevante sería la integración del sistema con el proyecto de Isabel Galindo, centrado en el uso de inteligencia artificial para ofrecer información a través de chatbots o asistentes de voz como Alexa. Esta conexión permitiría al usuario preguntar directamente al espejo inteligente cuestiones como: *¿Me he lavado la cara hoy?* y recibir una respuesta basada en los datos almacenados. De este modo, se facilitaría una interacción más natural y accesible, especialmente útil para personas con dificultades de memoria.

Otra mejora clave radica en la optimización del modelo de detección. Aunque el rendimiento ha sido satisfactorio en condiciones controladas, sería deseable incrementar su robustez ante situaciones más variadas, como la presencia de varias personas en el encuadre, la aparición de objetos que interfieren en la imagen o cambios de iluminación. Esto podría abordarse mediante

técnicas de aumento de datos, ajuste fino del modelo o incluso el uso de arquitecturas más sofisticadas adaptadas a entornos no estructurados.

También se contempla la posibilidad de adaptar el sistema para funcionar en dispositivos con mayor capacidad de procesamiento, como placas con unidades de procesamiento gráfico (GPU) o aceleradores dedicados como Coral o Jetson Nano. Esto permitiría incrementar la tasa de imágenes procesados por segundo, mejorar la fluidez general del sistema y escalar el modelo sin comprometer el rendimiento en tiempo real.

Otra línea de mejora consistiría en la incorporación de un sensor de presencia o proximidad que active automáticamente la captura de *keypoints* cuando se detecte a una persona frente al espejo. De este modo, se reduciría el consumo de recursos y se evitaría la captura innecesaria de datos, optimizando el rendimiento del sistema y prolongando la vida útil de los componentes. Esta funcionalidad resultaría especialmente útil en contextos donde el espejo no está siendo observado de forma continua, garantizando una activación eficiente y contextual del sistema.

La interfaz web actual, aunque funcional y accesible, podría enriquecerse con nuevas características orientadas a una mayor personalización. Entre las propuestas destacan: visualización del historial filtrado por fecha o tipo de acción, generación de alertas cuando no se detecte cierta actividad durante un intervalo prolongado, o incluso el envío de notificaciones automáticas a familiares o cuidadores en caso de comportamientos inusuales.

En conjunto, todas estas líneas de trabajo futuro apuntan a consolidar este proyecto como una solución extensible, conectada y centrada en el usuario. Su desarrollo progresivo permitiría no solo mantener, sino también mejorar la autonomía, seguridad y calidad de vida de personas mayores en su entorno doméstico.

Bibliografía

- [1] Luigi Atzori, Antonio Iera, and Giacomo Morabito. The internet of things: A survey. *Computer Networks*, 54(15):2787–2805, 2010.
- [2] Charith Perera, Arkady Zaslavsky, Peter Christen, and Dimitrios Georgakopoulos. Context aware computing for the internet of things: A survey. *IEEE Communications Surveys & Tutorials*, 16(1):414–454, 2014.
- [3] Dan Ding, Rory A Cooper, Paul F Pasquina, and Lavinia Fici-Pasquina. Sensor technology for smart homes. *Maturitas*, 69(2):131–136, 2011.
- [4] Chuango. La historia del hogar inteligente, 2024. Accedido el 28 de junio de 2025.
- [5] Frank K Aldrich. Smart homes: Past, present and future. In *Inside the Smart Home*, pages 17–39. Springer, 2003.
- [6] Marie Chan, Daniel Estève, Christian Escriba, and Eric Campo. A review of smart homes—present state and future challenges. *Computer Methods and Programs in Biomedicine*, 91(1):55–81, 2008.
- [7] George Demiris and Brian K Hensel. Technologies for an aging society: A systematic review of “smart home” applications. *Yearbook of Medical Informatics*, 17(01):33–40, 2008.
- [8] Liming Chen, Chris D. Nugent, Huiqiang Wang, and Cagatay Catal. Sensor-based activity recognition in the home: A review of datasets. *Journal of Ambient Intelligence and Humanized Computing*, 3(2):71–85, 2012.
- [9] Parisa Rashidi and Diane J. Cook. A survey on ambient intelligence in healthcare. *Proceedings of the IEEE*, 98(12):2190–2201, 2010.
- [10] Oscar D Lara and Miguel A Labrador. A survey on human activity recognition using wearable sensors. *IEEE Communications Surveys & Tutorials*, 15(3), 2013.
- [11] Alexander Mathis, Pranav Mamidanna, Kevin M Cury, Thiago Abe, Venkatesh N Murthy, Mackenzie W Mathis, and Matthias Bethge. Deeplabcut: markerless pose estimation of user-defined body parts with deep learning. *Nature neuroscience*, 21, 2018.
- [12] Parisa Rashidi and Alex Mihailidis. A survey on ambient-assisted living tools for older adults. *IEEE Journal of Biomedical and Health Informatics*, 17, 2013.

-
- [13] Feng Wang and Jing Wang. Daily activity recognition with wearable sensors for eldercare. *Pattern Recognition Letters*, 119, 2019.
 - [14] Ifigeneia Mavridou and Andreas L. Symeonidis. Smart home integration with human activity recognition. *Procedia Computer Science*, 113, 2017.
 - [15] Kun-Chih Chen, Wen-Huang Lin, Yuan-Ko Lu, Ying-Hsuan Chang, and Ming-Hsun Tsai. Human action recognition for physical rehabilitation using 3d pose estimation. *Sensors*, 21, 2021.
 - [16] Silvia Martínez, Alberto Pérez, Daniel Rodríguez, and Sergio Borromeo. Anomaly detection in human behavior for elderly monitoring using sensors in smart home environments. *Sensors*, 20, 2020.
 - [17] Chihiro N Harada, Miya C Natelson Love, and Kristen L Triebel. Normal cognitive aging. *Clinics in Geriatric Medicine*, 29, 2013.
 - [18] Denise C Park, Thad A Polk, Joseph A Mikels, Stephanie F Taylor, and Claudia Marshuetz. Aging and the functional architecture of episodic memory: a meta-analysis of neuroimaging studies. *Psychological Science*, 13, 2002.
 - [19] S M Korkki, F R Richter, P Jeyarathnarajah, and J S Simons. Healthy ageing reduces the precision of episodic memory retrieval. *Psychology and Aging*, 35, 2020.
 - [20] Ronald C. Petersen. Mild cognitive impairment: clinical characterization and outcome. *Archives of neurology*, 56, 1999.
 - [21] Donggeon Kwon, Jaehong Ryu, and Youngho Lee. Caremirror: A smart mirror for active aging. In *2018 International Conference on Smart Health (ICSH)*. IEEE, 2018.
 - [22] Daniel Besserer, Johannes Bäurle, Alexander Nikic, Frank Honold, Felix Schüssel, and Michael Weber. Fitmirror: A smart mirror for positive affect in everyday user morning routines. In *Proceedings of the 2016 ACM International Conference on Multimodal Interaction*. ACM, 2016.
 - [23] Andrea Gaggioli, Giuseppe Riva, Pietro Cipresso, et al. The wize mirror—a sensorized mirror for wellness and preventive healthcare. *IEEE Journal of Biomedical and Health Informatics*, 21, 2017.
 - [24] M. Gómez, J. Ferreira, and I. Domínguez. Shapes smart mirror: A multimodal interface for promoting active and healthy ageing. *Procedia Computer Science*, 198, 2021.
 - [25] S. López, C. Ramos, and P. Jiménez. Designing a responsive smart mirror for elderly care: User-centered insights and technical integration. In *Proceedings of the International Conference on Human-Computer Interaction*, pages 113–125, 2024.
 - [26] J. Park, D. Lee, and H. Kim. A computer vision-based system for hand hygiene compliance
-

- monitoring using depth cameras. *IEEE Access*, 8:130410–130421, 2020.
- [27] Y. Tian, W. Qin, and S. Li. Smart hospital environments: Monitoring hand hygiene with artificial intelligence. *Journal of Healthcare Engineering*, 2017, 2017.
- [28] A. Martínez, L. Zhao, and M. Thompson. Electronic hygiene monitoring systems in healthcare: A systematic review. *International Journal of Medical Informatics*, 153, 2021.
- [29] J. Fernández, F. Müller, and M. Ortega. Iot-based recognition of personal hygiene activities using machine learning in smart bathrooms. *Sensors*, 22(15), 2022.
- [30] Dataqu.ai. Beneficios de utilizar python en el machine learning. <https://www.dataqu.ai/beneficios-de-utilizar-python-en-el-machine-learning/>, 2023. Dataqu.ai.
- [31] Google. Mediapipe. <https://mediapipe.dev>, 2024. Último acceso: mayo 2025.
- [32] OpenCV Team. Open source computer vision library. <https://opencv.org>, 2024. Último acceso: mayo 2025.
- [33] Python Software Foundation. csv — csv file reading and writing. <https://docs.python.org/3/library/csv.html>, 2024. Último acceso: mayo 2025.
- [34] Python Software Foundation. os — miscellaneous operating system interfaces. <https://docs.python.org/3/library/os.html>, 2024. Último acceso: mayo 2025.
- [35] Python Software Foundation. time — time access and conversions. <https://docs.python.org/3/library/time.html>, 2024. Último acceso: mayo 2025.
- [36] Redacción ComputerHoy. Qué es latex, cómo funciona y por qué es una útil herramienta para crear documentos. <https://computerhoy.20minutos.es/tecnologia/latex-como-funciona-util-herramienta-crear-documentos-1165366>, 2023. ComputerHoy.
- [37] GitHub Docs. Acerca de github y git. <https://docs.github.com/es/get-started/start-your-journey/about-github-and-git>, 2024. GitHub Documentation.
- [38] Google Research. Google Colaboratory. <https://colab.research.google.com>, 2023. Último acceso: mayo de 2025.
- [39] Project Jupyter. Jupyter Notebook. <https://jupyter.org>, 2023. Último acceso: mayo de 2025.
- [40] Google. Mediapipe: Cross-platform, customizable ml solutions for live and streaming media. <https://mediapipe.dev>, 2023. Accedido en mayo de 2025.
- [41] Camillo Lugaresi, Jonathan Tang, Hartwig Nash, Chris McGuire, Chuo-Ling Chang, Michael Yong, and Joe Lee. Mediapipe: A framework for building perception pipelines. *arXiv preprint arXiv:1906.08172*, 2019.
-

-
- [42] Fan Zhang, Valentin Bazarevsky, Andrey Vakunov, Alexey Tkachenka, Grace Sung, Chuoling Chang, and Matthias Grundmann. Mediapipe hands: On-device real-time hand tracking. *arXiv preprint arXiv:2006.10214*, 2020.
 - [43] Valentin Bazarevsky, Yury Kartynnik, Andrey Vakunov, Alexey Tkachenka, and Matthias Grundmann. Blazepose: On-device real-time body pose tracking. <https://ai.googleblog.com/2020/08/on-device-real-time-body-pose-tracking.html>, 2020. Google AI Blog.
 - [44] Valentin Bazarevsky, Yury Kartynnik, Andrey Vakunov, Alexey Tkachenka, and Matthias Grundmann. Blazeface: Sub-millisecond neural face detection on mobile gpus. *arXiv preprint arXiv:1907.05047*, 2020.
 - [45] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. Capítulo 6: Feedforward Deep Networks.
 - [46] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521, 2015.
 - [47] Stanford University. Cs231n: Convolutional neural networks for visual recognition. <urlhttps://cs231n.github.io/convolutional-networks/>, 2016. Accedido en 2025.
 - [48] Chris Olah. Understanding lstm networks. <urlhttps://colah.github.io/posts/2015-08-Understanding-LSTMs/>, 2015. Accedido en 2025.
 - [49] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9, 1997.
 - [50] Yoshua Bengio, Patrice Simard, and Paolo Frasconi. Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5, 1994.
 - [51] Junyoung Chung, Caglar Gulcehre, Kyunghyun Cho, and Yoshua Bengio. Empirical evaluation of gated recurrent neural networks on sequence modeling. *arXiv preprint arXiv:1412.3555*, 2014.
 - [52] Wenpeng Yin, Katharina Kann, Mo Yu, and Hinrich Schütze. Comparative study of cnn and rnn for natural language processing. *arXiv preprint arXiv:1702.01923*, 2017.
-

Este Trabajo Fin de Grado presenta el desarrollo de un sistema inteligente capaz de reconocer acciones de higiene personal a través de un espejo inteligente dotado con cámara, una pantalla y un modelo de aprendizaje profundo. Utilizando técnicas de visión por computador y la extracción de puntos clave con MediaPipe, el sistema detecta en tiempo real actividades como lavarse los dientes o peinarse, mostrando la información en una interfaz web accesible e intuitiva. El objetivo es asistir a personas mayores con pérdidas de memoria, facilitando el seguimiento de sus rutinas diarias. El sistema se ha implementado sobre una Raspberry Pi, optimizando el rendimiento para garantizar un funcionamiento fluido con recursos limitados.

This Final Degree Project presents the development of an intelligent system capable of recognizing personal hygiene actions through a mirror equipped with a camera, a screen, and a deep learning model. Using computer vision techniques and keypoint detection via MediaPipe, the system identifies activities such as toothbrushing or hair combing in real time, displaying the information through an accessible and user-friendly web interface. The goal is to assist elderly people with memory loss by helping them monitor their daily routines. The system is deployed on a Raspberry Pi, with performance optimizations to ensure smooth operation on limited hardware.